

A SCALABLE SPARSE DISTRIBUTED NEURAL MEMORY MODEL

A THESIS SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF MASTER OF PHILOSOPHY
IN THE FACULTY OF SCIENCE AND ENGINEERING

September 2003

By
Joy Bose
Department of Computer Science

Contents

Abstract	11
Declaration	13
Copyright	14
Acknowledgements	15
1 Introduction	16
1.1 The motivation for the work	16
1.2 Aims of this research	17
1.3 Why hardware	17
1.4 A finite state machine	18
1.5 An artificial neural network as a pattern memory	18
1.6 Pattern encoding in a neural network	19
1.7 Scalability and online learning	19
1.8 Dynamics of spike propagation in neural systems	20
1.9 Storing a sequence of patterns	20
1.10 Research contribution made by this work	20
1.11 Thesis structure	21
2 Artificial neural models	22
2.1 Introduction	22
2.2 The human brain	22
2.3 Structure of the brain	23
2.4 Structure of a neuron	24
2.5 Mathematical model of a neuron	25
2.6 An artificial neural network	26

2.7	History of artificial neural network development	27
2.8	Issues in determining types of neural networks	27
2.9	Learning algorithms	28
2.10	Neural structures	29
2.11	An analysis of various neural algorithms and structures in the con- text of this research	31
2.12	Rate coding and temporal coding	31
2.13	Pulsed or spiking neural networks	32
2.14	Neural Networks research today	32
2.15	Applications	32
2.16	Conclusion	33
3	Spiking neuron models	34
3.1	Introduction	34
3.2	Neural Coding	34
3.3	Spiking neurons	35
3.4	Mathematical model of a single spike	36
3.5	Modelling populations of spiking neurons	39
3.6	Computing with spiking neurons	39
3.7	Other issues and simulation	40
3.8	Conclusion	40
4	Related Work	41
4.1	Introduction	41
4.2	Associative memories	41
4.3	RAM, BAM and CAM	42
4.4	Hopfield memories	43
4.5	Correlation matrix memories (CMM)	44
4.6	Willshaw memories	46
4.7	RAM based or weightless systems	47
4.8	Working Systems	49
4.9	Conclusion	52
5	Sparse Distributed Memories	53
5.1	Introduction	53
5.2	Overview	53

5.3	Kanerva's sparse distributed memory model	53
5.4	Development of the model	54
5.5	The final model	57
5.6	Reading and writing to the memory	58
5.7	Salient features of the sparse distributed memory model	58
5.8	How it works	61
5.9	Explanation of biological memory phenomena	62
5.10	SDM and the cerebellum	62
5.11	Comparison with a computer memory	63
5.12	Drawbacks of the memory	63
5.13	Conclusion	63
6	Sparse distributed memory using N of M codes	64
6.1	Introduction	64
6.2	N of M codes: an introduction	64
6.3	Motivation: Why N of M codes	64
6.4	Information capacity of N of M codes	65
6.5	Related work	65
6.6	Development of the memory model	66
6.7	The N of M address space	66
6.8	N of M implementation	66
6.9	An N of M coded sparse distributed memory structure: Structure and function	67
6.10	Reading and writing to the memory	69
6.11	Equivalence of the address space with that of Kanerva's model . .	71
6.12	Implementing N of M code with spiking neurons	71
6.13	Sorting	72
6.14	Processing all the neurons in a fascicle	75
6.15	Mathematical analysis of the memory model	76
6.16	Experimental analysis of the model	77
6.17	Capacity of the memory by varying the memory sizes	82
6.18	The convergence property	88
6.19	Feasibility to implement in hardware	89
6.20	Drawbacks of the model	89
6.21	Possible modification: Using an N Max of M code	89
6.22	Setting of the weights in writing	90

6.23	Use of the memory in the learning of a sequence	92
6.24	The memory as a finite state machine	93
6.25	Conclusions	94
7	Using rank ordered N of M codes	95
7.1	Introduction	95
7.2	Rank ordered codes	95
7.3	Related work	96
7.4	Why rank ordered codes	96
7.5	Ordered N of M Address Space and distance	98
7.6	Implementing rank ordered N of M codes through spiking neurons	98
7.7	Using an abstraction of the pulsed model: sensitisation factor . .	100
7.8	An analysis of the rank ordered N of M coded memory model . .	101
7.9	Sequence Recognition	107
7.10	Conclusions	107
8	Learning Sequences	108
8.1	Introduction	108
8.2	A neural finite state machine	108
8.3	Motivation: The importance of time	110
8.4	Previous approaches to building time into the operation of a neural network	110
8.5	Sequence Learning	114
8.6	Learning and retrieval of music tunes	114
8.7	Structure and role of the context layer	116
8.8	A modified rank ordered N of M model to learn sequences	117
8.9	The context weight factor	117
8.10	The sequence machine	118
8.11	Online and offline learning	120
8.12	Analysis of the sequence machine	122
8.13	Preliminary results for sequence recall	127
8.14	Recalling the sequence from the middle	128
8.15	Having dynamic context weightage	128
8.16	Application areas	129
8.17	Drawbacks of the sequence machine	129
8.18	Conclusion	130

9	Neurons as Dynamical Systems	131
9.1	Introduction	131
9.2	Motivation	131
9.3	Related work	132
9.4	A feedback control system	132
9.5	Propagation of spike waves around a feedback loop	133
9.6	Anti dispersion measures	136
9.7	Stabilisation of neural states and oscillation	136
9.8	A generalised neural system	136
9.9	Neural Engineering	138
9.10	Emergent behaviour	138
9.11	Conclusion	141
10	Conclusions and future work	142
10.1	Introduction	142
10.2	Summary of this research	142
10.3	Experimental results	143
10.4	Contributions made in this thesis	144
10.5	Future work: Towards better memory models	145
10.6	Future work: Possible applications	146
10.7	Conclusion	149
	Bibliography	150

List of Tables

6.1	Information capacity comparison of different coding schemes of same size	65
8.1	Offline writing of sequence ABC	120
8.2	Offline reading out of the sequence ABC on giving A as input . .	120
8.3	Online reading/writing of the sequence ABC	121

List of Figures

1.1	A generic finite state machine	18
2.1	Structure of a human brain	23
2.2	Structure of a neuron showing its component parts	24
2.3	A simple model of a neuron	25
2.4	An artificial neural network	26
3.1	Shape of a spike (taken from Gerstner: Spiking neural models) . .	35
3.2	Mathematical model of a spike (taken from Gerstner: Spiking neu- ral models)	37
3.3	Mathematical model of an integrate and fire neuron	38
4.1	A Hopfield Network	43
4.2	A Correlation Matrix Memory	45
4.3	A RAM node	48
4.4	ADAM network structure	50
4.5	WISARD structure	51
5.1	N-dimensional space of Kanerva's SDM	55
5.2	The sparse distributed memory model	57
5.3	An illustration of the convergence of the memory below, and di- vergence above the critical distance	59
6.1	The N of M neural memory model	68
6.2	Plot of the spread in w with address decoder threshold T	69
6.3	Feedback inhibition to impose N of M code	72
6.4	The pendulum model of a spiking neuron	73
6.5	Event driven processing in Pendulum model: Using time to sort .	75
6.6	Information content of N of M coded words with varying N	78

6.7	Capacity of the memory as the number of bit errors are increased	78
6.8	Incremental Capacity of the N of M memory	80
6.9	Capacity of the memory with varying threshold used to measure correctness	82
6.10	Average recovery capacity of the memory with error bits	83
6.11	A plot of the memory capacity with varying d greater than/ equal to 11	84
6.12	A plot of the memory capacity with varying d less than/ equal to 11	84
6.13	A plot of the memory capacity with varying d	85
6.14	A contour plot of the memory capacity with varying w (11 of 256 code) when matching threshold is 0.9	86
6.15	A 3D plot of the memory capacity, when exact matching is used .	87
6.16	A contour plot of the memory capacity with varying w (11 of 256 code) when exact matches are used	87
6.17	A plot of the memory capacity with varying W	88
6.18	A plot of the memory capacity with varying D	88
6.19	A plot of the memory capacity when different methods of varying weights are used	91
6.20	A finite state automaton for recognising the sequence ABCDE . .	94
7.1	Information content of ordered N of M coded words with varying N	97
7.2	Using feed-forward inhibition to impose input ordering	99
7.3	A fascicle to implement N of M code with rank order	99
7.4	A Screenshot of the memory simulator	101
7.5	Capacity of the rank ordered N of M memory for perfect match .	102
7.6	Capacity of the rank ordered N of M memory for matching thresh- old of 0.9	103
7.7	Memory capacity for ordered N of M memory with sensitivity for matching threshold of 0.9	104
7.8	Capacity of the rank ordered N of M memory with sensitivity=0.5	105
7.9	Capacity of the rank ordered N of M memory with sensitivity=0.9	105
7.10	Capacity of the rank ordered N of M memory with sensitivity=0.9	106
7.11	Average recovery capacity of the rank ordered memory	106
8.1	Using delay lines	111
8.2	Using feedback with delay	112

8.3	The Jordan network	112
8.4	An Elman network with context layer	113
8.5	The modified N of M Kanerva memory with context	118
8.6	Structure of the sequence machine	119
9.1	A feedback control system block diagram	132
9.2	A wave of input and output spike firings in a fascicle	134
9.3	A fascicle to implement ordered N of M code	135
9.4	A propagating spike wave and the two time constants	135
9.5	Using a second inhibitory fascicle to control wave dispersion . . .	137
9.6	Virtual environment	139
9.7	Virtual environment implementation with neurons	140

Abstract

For a number of years, artificial neural networks have been used for a variety of applications to automate tasks not suitable for the conventional computing model, such as pattern-recognition. Their inherent non-linearity and parallelism makes them suitable for approximating a variety of functions and graphs.

This work is an attempt to develop a scalable sparse distributed neural memory model, which is capable of storing and retrieving sequences of patterns. The model developed is robust, scalable, generic, self error correcting, with low average neural activity resulting in lower power usage, and capable of learning in one-shot. Patterns are encoded in the memory as an ordered code. The model is inspired by Pentti Kanerva's book on sparse distributed memories, in which he described a memory model where data are stored as addresses in the memory, and which could be implemented through neurons.

The basic neural model used is a modified version of Kanerva's model, which uses a special encoding scheme to encode information. N of M codes are used in this scheme, and increased information content is attempted through rank order coding. An experimental investigation is made into the capacity of such a memory, and its ability to store and retrieve sequences of characters and patterns. An attempt is made to incorporate some form of state information in the memory, so that it can store and retrieve multiple sequences having some similar components, and retrieve a whole sequence on being given a part of a sequence, or even one with errors.

In this work, a comparative analysis has been done of various memory models with respect to information bearing capacity, state space and ease of hardware implementation. A number of experiments have been done on the resulting memory model to determine its memory capacity and the ability to recognise sequences of characters. A discussion has been made of the issues involving the dynamics of neurons in large systems with feedback. Possible applications of the model have

also been explored.

The memory model developed has got certain similarities with the cerebellar cortex, a part of the mammalian brain specialising in sensory-motor coordination. Thus it is neurobiologically plausible. It tries to show it is possible to build a large scale neural network involving millions of neurons and utilise it to do useful tasks. The work should be of interest to neural network researchers as well as those seeking a new computational model having more robust and scalable characteristics.

Declaration

No portion of the work referred to in this thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institution of learning.

Copyright

Copyright in text of this thesis rests with the Author. Copies (by any process) either in full, or of extracts, may be made **only** in accordance with instructions given by the Author and lodged in the John Rylands University Library of Manchester. Details may be obtained from the Librarian. This page must form part of any such copies made. Further copies (by any process) of copies made in accordance with such instructions may not be made without the permission (in writing) of the Author.

The ownership of any intellectual property rights which may be described in this thesis is vested in the University of Manchester, subject to any prior agreement to the contrary, and may not be made available for use by third parties without the written permission of the University, which will prescribe the terms and conditions of any such agreement.

Further information on the conditions under which disclosures and exploitation may take place is available from the head of Department of Computer Science.

Acknowledgements

I would like to thank both my supervisors, Steve and Jon, for helping me in every way possible, apart from giving their valuable time in endless weekly meetings.

Thanks go to my advisor, Magnus for giving me a patient hearing.

Thanks to my co-researchers Li, Pete and especially Mike for helping me out and bearing my occasional bugging patiently. I am grateful to all IT302 occupants for their cooperation. Thanks to members of the APT group, especially Andrew, for their support and technical help at various times.

This work has been funded by the ORS studentship from Universities UK, and research scholarship by the University of Manchester. The author gratefully acknowledges the support of the same.

This thesis has been drafted using $\text{\LaTeX}$. The figures were made using `tgif` and `xfig`. Gnuplot and Matlab were using in plotting simulation results, and the simulations themselves were coded in Java.

Chapter 1

Introduction

The parallel processing and nonlinear capabilities of the human memory and its building blocks, the neurons, present great potential use in a variety of pattern recognition and related applications. Research in artificial neural networks is concentrated in finding mathematical models of the neurons in the brain and trying to seek insights into the working of the brain, as well as emulating some of the tasks considered trivial for the brain but not for the computer.

This thesis aims to build a memory built out of artificial neurons which is scalable and can be implemented in hardware. This work is part of a larger project whose aim is to build a large scale generic neural system in hardware. This research is concentrated on development of an appropriate software model to do the same.

This chapter is an introduction to the work. It includes an overview of the basic issues regarding the development of the memory model.

1.1 The motivation for the work

An artificial neural network seeks to model the functioning of natural neurons, which are the building blocks of our brains. Due to their nonlinearity and inherent parallelism, they are capable of a number of tasks not suitable for a conventional Von Neumann[39] computer model, which computes serially. They have many applications in pattern recognition and related fields. Neurons by themselves are unreliable, but a network made of such neurons can be very robust and capable of performing a variety of high end computing tasks. Information in a neural system is distributed among the component neurons, so they are quite robust to

failure of the individual components.

Building a neural system in hardware is useful in order to exploit the parallelism in hardware to build fast systems. Using a scalable artificial neural network model, a large scale system can be built. Such a system can have many possible applications, such as character or image recognition, robotics, time series prediction, machine vision, data mining, and many others. This work is basically an attempt to develop a suitable software model for building scalable hardware systems. These large scale hardware neural systems, when developed, would be robust and have high performance, and thus would be capable of being used in computationally intensive tasks.

There are also neurobiological aspects to this work. This work can help to get a better understanding of the working of the brain by modelling the neural structures found in the brain, especially those in the cerebral cortex. Neurons have emergent behaviour which can be observed only when large scale networks are used, and modelling which can develop a better appreciation for the way our brains work.

1.2 Aims of this research

The primary goal of this research is to develop a neural model which is scalable, built of spiking neurons, capable of one-shot learning and suitable for implementation in hardware. The model would function primarily as a pattern associator and be capable of storing and recognising patterns as well as sequences of patterns.

This work is part of a larger area of research, which is termed as Neural Systems Engineering. It is an attempt build reliable working systems out of unreliable component neurons. The long term goal of this project is to get a better understanding of the dynamics of neural behaviour in large systems of neurons.

1.3 Why hardware

Neural networks are by nature parallel connectionist architectures. However, software is run on inherently serial machines. A neural network implemented in hardware would, in principle, be faster than one implemented in software, since

parallel computation is possible in a hardware implementation. For large scale systems, speed becomes an important factor and the serial processor can quickly become a bottleneck in performance.

1.4 A finite state machine

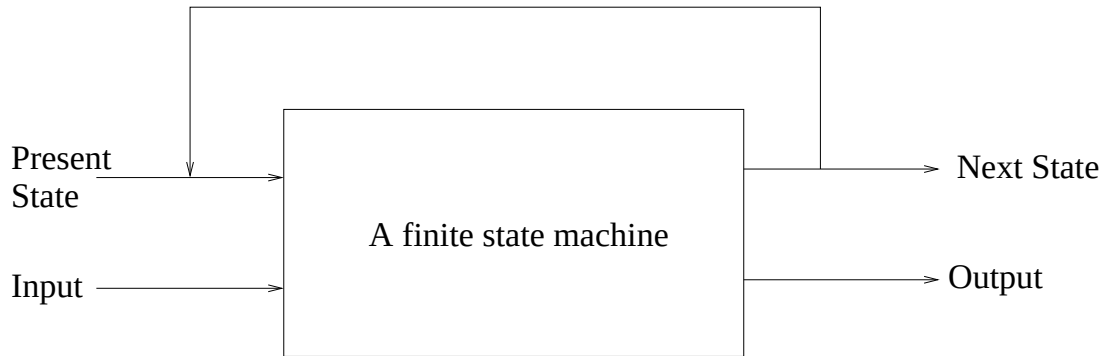


Figure 1.1: A generic finite state machine

A finite state machine is an automaton which has got a finite number of internal states and can change state depending on the input. The structure of a finite state machine is shown in the figure above. A Turing machine is an example of a finite state machine which can implement any algorithm (Church Turing Hypothesis [33]). A neural network can be modelled as a finite state machine if we include feedback and some neurons to represent state. One of the goals of this work is to explore whether it is possible to emulate the functionality of a general finite state machine by this model.

1.5 An artificial neural network as a pattern memory

An artificial neural network consists of a number of layers of artificial neurons with connections between them. There is an input layer, an output layer and one or more layers in between. An input pattern is provided in an encoded form to the input layer and the network outputs a pattern. The neural network can be trained to produce a particular output pattern given a specific input pattern. Thus it associates input and output patterns. Another way to look at it is like a memory

in which the output patterns(data) are stored at the input patterns(addresses) and to get the correct data all we need to do is to give the correct address pattern. The difference between a neural network and a conventional computer memory is that in a conventional memory we need to give the exact addresses to retrieve the exact data, but in a neural network even noisy addresses would be able to retrieve the correct or near-correct data. Besides, a neural network is robust to the failure of a few component neurons. That is because storage of information is distributed among several neurons rather than a single one. In a conventional memory if individual address locations fail, it would be impossible to retrieve the data stored in those locations.

1.6 Pattern encoding in a neural network

As stated in the above section, patterns are given to a neural network in an encoded form. The exact encoding of the input and output patterns used can influence how many patterns the memory is capable of storing, i.e. the capacity of the memory. In this research we have investigated the capacity of the memory when two encoding schemes are used: the N of M and the ordered N of M coding.

1.7 Scalability and online learning

One of the aims of this research is that the network model should be scalable. For this to happen it is necessary that the use of global control signals should be minimised. Communication between the individual components (neurons) of the network should be done as locally as possible. For online or one-shot learning in such a network to be possible, it is necessary that the neural learning algorithm used should be able to associate pairs of patterns after they are presented just once to the memory. The Hebbian learning algorithm, as described in Chapter 2, fits this description.

1.8 Dynamics of spike propagation in neural systems

Natural neurons communicate with the help of electrical pulses called spikes. For reliable propagation of information between in the form of spike trains between layers of neurons to be possible, the spike trains should not to be distorted as the signals propagate in feedback loops in the network. This necessitates a consideration of the dynamics of the neuronal systems. Some mechanism is needed to ensure that the separation in time between two different trains of spikes is sufficiently large to prevent interference between different spike trains.

1.9 Storing a sequence of patterns

In a previous section we saw how a neural network can be seen as a pattern memory, in the sense that it can learn to associate an output pattern for a given input pattern. In a sequence of patterns, different input patterns may have the same output pattern or vice versa, depending on the position of the pattern in a sequence. Thus, in order to store a sequence of patterns, there should be some form of storing of the context of every pattern in the sequence, so that the next output depends not only on the present input pattern, but also on its context or position in the sequence. Such a network might be capable of predicting the next output patterns in the sequence, and may even reproduce the whole sequence on being given the first pattern.

1.10 Research contribution made by this work

The main contribution of this work is to develop a scalable neural model using rank ordered spiking neurons, which is capable of storing and retrieving sequences of patterns with single shot learning. An analysis has been done of the information storage capacity of the memory, which has been compared to the capacity of related neural memories. Some application areas for the same have been explored.

1.11 Thesis structure

This chapter contained a brief introduction to the thesis work. The remainder of the thesis is organised as follows:

The second, third and fourth chapters provide a foundation to the field. The fifth, sixth and seventh chapters are about the development of the neural model and the analysis of its properties. The eighth chapter explores the use of the model to learn sequences.

Chapter 2: Artificial neural networks. This chapter contains an overview and comparison of various neural architectures and learning algorithms.

Chapter 3: Spiking neural models. This chapter contains introduction to the spiking neural model, which is used in this work.

Chapter 4: Related work. This chapter contains a brief overview of related work in neural associative memories and some examples of application-specific implementations. This forms the context for this research.

Chapter 5: Sparse Distributed Memories. This chapter discusses Kanerva's sparse distributed memory model in detail, which forms the basis for this work.

Chapter 6: Sparse distributed memory using N of M codes. This chapter discusses the use of N of M coding in the sparse distributed memory model and analyses the capacity and other properties of the resulting memory model through numerical experiments. It discusses both low level implementations as well as high level abstractions of the memory.

Chapter 7: Using rank ordered N of M codes. This chapter modifies the previous model by using N of M codes with rank order, and analyses the capacity of the resulting memory. It also evaluates the model as a state machine with feedback .

Chapter 8: Learning Sequences. In this chapter the previous model is modified to store context information in a sequence, thus enabling it to store and retrieve sequences of patterns.

Chapter 9: Neurons as Dynamical Systems. In this chapter, a discussion is made of the dynamics of propagating a neural wave front over a layer of neurons. This also explores a possible way of modelling emergent behaviour in neural systems.

Chapter 10: Conclusions and future work. This is the last chapter and concludes the thesis. It expounds the major results of the work done so far and discusses areas for future work, including possible applications.

Chapter 2

Artificial neural models

2.1 Introduction

This chapter contains an overview of neural network models and major learning algorithms. It also contains a comparison of rate codes and temporal codes and introduces the spiking model, which is discussed in more detail in the following chapter. The analysis of various neural architectures and algorithms in this chapter is done to give a broader insight into the wider field of neural networks and to discuss which suitable neural algorithm is to be used in this research.

2.2 The human brain

All neural network models are inspired by natural neurons, which are found in the nervous systems of most animal species. The brain is the best example of a natural neural network.

The brain is a robust, parallel, nonlinear, distributed information processor, which works mainly by associations and symbolic processing. The primary components of the brain are called neurons. The brain is made up of a complex network of these neurons. These neurons communicate by means of action potentials, or spikes of voltage, which are produced by a complex system of chemical and ionic reactions in the soma or body of the neurons.

The mammalian brain is the most advanced of all brains on this planet by virtue of its complexity and richness of function. The human brain is divided into two hemispheres, each of which are supposed to do a different function. There are three parts of the brain: The cerebrum, the cerebellum and the medulla

oblongata. The cerebrum is like a huge sheet coiled up. The brain is built up mainly of grey and white matter. The cerebrum is divided into regions, each of which performs a specific functions. Neuroscientists have attempted to isolate the functioning of various parts of the brain. The visual cortex and the retina are also considered an extension of the brain, because they too have neurons.

The structure of the brain is shown in the figure.

2.3 Structure of the brain

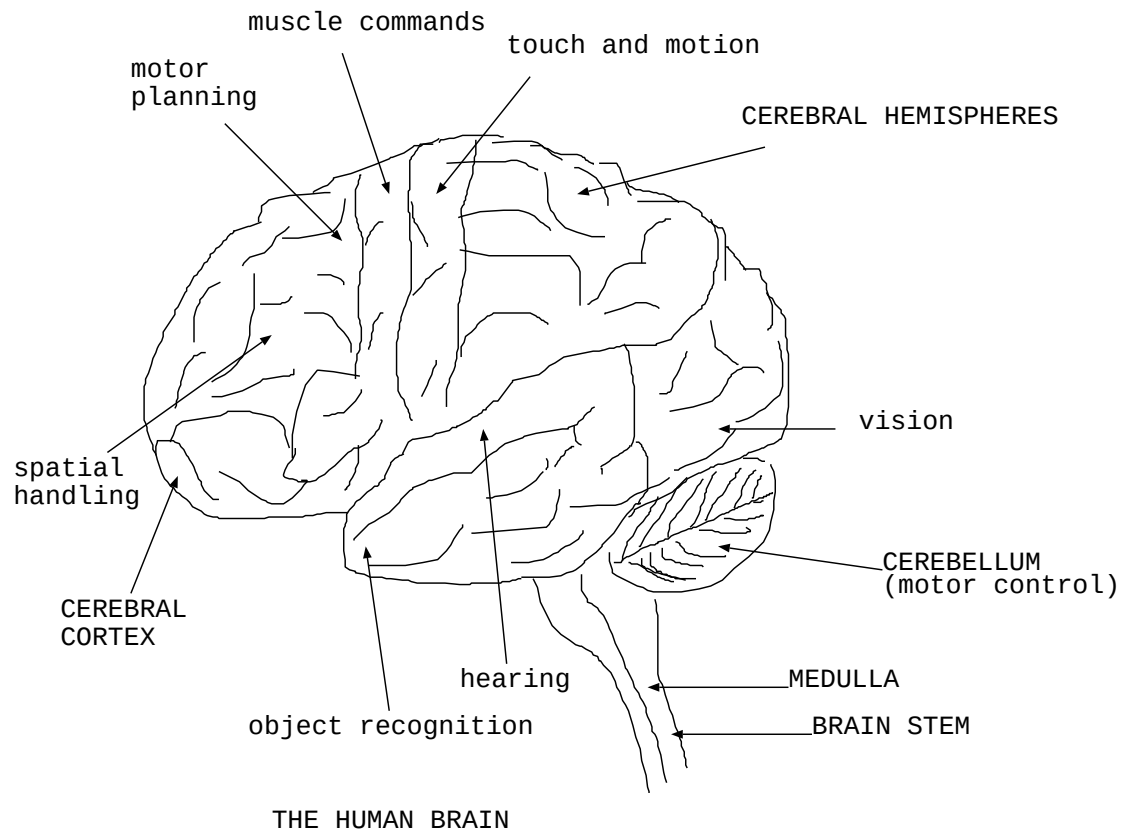


Figure 2.1: Structure of a human brain

Both neuroscientists and neural mathematicians have been interested in the functioning of the brain, the biologists for mainly medical purposes, and the mathematicians in order to exploit it to enable the computer to perform the traditional brain tasks. Present day computers can perform millions of accurate computations at an incredible pace and store vast amounts of data, yet are inhibited when told to do a number of tasks which would be considered trivial for

the brain. The brain is mainly made up of cells which are called neurons.

2.4 Structure of a neuron

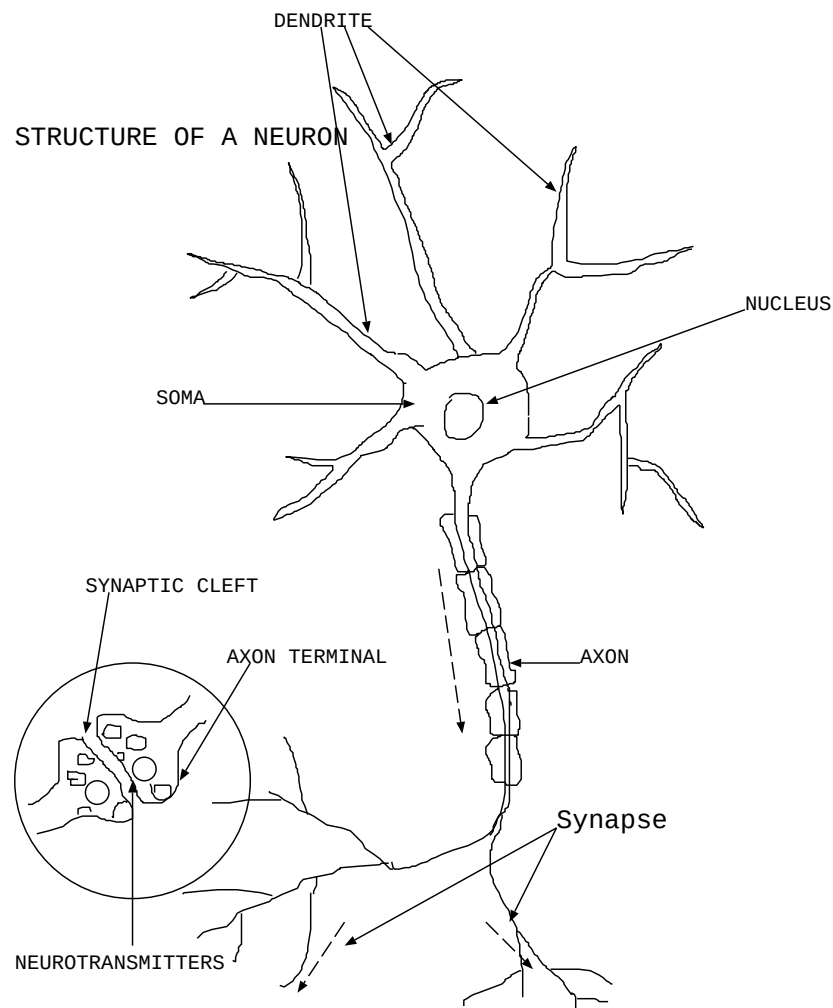


Figure 2.2: Structure of a neuron showing its component parts

The neuron is like a long cell, having three main parts, namely the cell body or soma, the long axon and covered with the myelin sheath. At the ends of the axon are the dendrites, a mesh of very fine fibres which act like the sensory organs. The spikes or action potentials, through which the brain computes, are produced in the soma, travel through the axon and are finally transmitted to the next neuron by the dendrites. The length of the neuron varies, but the average neuron is quite long. The neuron communication and the transmission of spikes

takes place through neurotransmitters, which are chemicals having ionic charge.

2.5 Mathematical model of a neuron

Neurons function in a very complex way, with information being stored and transmitted through synapses, the complex chemical and ionic reactions happening at a cellular level almost all the time, and the resulting action potentials that are transmitted in the form of spikes through the axon and synaptic clefts. The mathematical model of the neuron aims to model its function and working in the simplest possible manner, by abstracting out all but its most essential components.

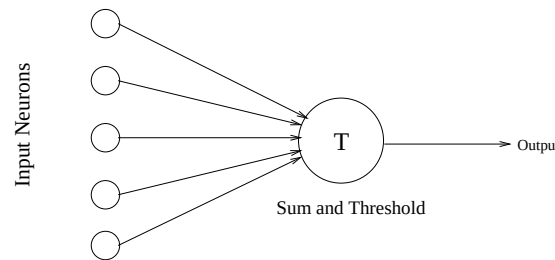


Figure 2.3: A simple model of a neuron

The neuron cell body is modelled by a nonlinear threshold function, the dendrites by a number of connections, each having weights representing the synaptic strength. And the average rate of firing of the pulses is represented by a number value which forms either the input or the output.

An artificial neuron is the building block of an artificial neural network, just like the actual neuron is the building block of the brain. The simplest model of the artificial neuron, as shown in the figure, consists of weights representing connections from other neurons, a summing mechanism to sum all the activations, and a threshold mechanism that makes it fire if the total value of the activation exceeds the threshold. Each of these models parts of a real neuron: the weights representing the strength of connections between neurons and the summing and thresholding abstracting the task of the cell body, soma.

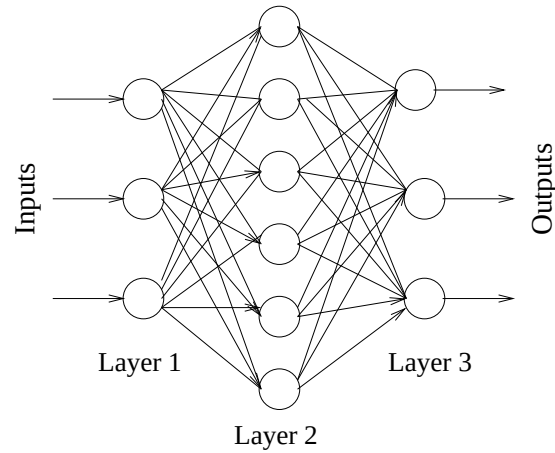


Figure 2.4: An artificial neural network

2.6 An artificial neural network

An artificial neural network consists of many of these artificial neurons connected together. These neurons are grouped together in layers. The connections between different neurons of one layer or of different layers are represented by weights. The value of these weights, which may be positive or negative, represent the strength of the connections. The activation of a neuron are transferred to the neurons connected to it, when it fires.

2.6.1 The computer memory compared to a neural network

The conventional sequential computer stores data at fixed addresses. There is no scope for error. Once put, data stays at the address until overwritten. Thus, it learns in 'one pass'. One more characteristic is that it does not forget, unless the memory location is overwritten, when it forgets abruptly. Neither can it recognise noisy data. If the address is incorrect even one bit, it would never recall the correct data. On the other hand, a neural network is distributed, which means that one piece of information is stored at a number of places rather than a single memory location, as in a computer. The weights in a neural network store the information. A typical neural network has to be 'trained' to associate the input address with the data, which means that learning the association of patterns, usually takes more than in one pass. Rather, it takes a number of passes with different data sets to train a network. Besides, forgetting is gradual rather than

abrupt.

2.7 History of artificial neural network development

The development of artificial neural networks is not a very recent trend. In 1943, McCulloch and Pitts[29] developed a model of neurons, along with their synaptic connections. They showed that such a network could compute any computable function. Hebb, in 1949, wrote a book called *The Organisation of Behaviour*[20], in which he gave his famous postulate of learning. Rosenblatt in 1958, proposed the perceptron[34] and gave the famous perceptron convergence theorem. Many other neural architectures, including associative memories and least mean square algorithm, were developed in the 50's and the 60's. In 1972, Kohonen[27] gave the idea of correlation matrix memories. Neural network research was stunted for a decade after that. In the 1980's, Hopfield[23] again raised interest in neural network research with his Hopfield networks. Architectures such as Boltzmann machine, the backpropagation algorithm, reinforcement learning etc were developed after this time

2.8 Issues in determining types of neural networks

In comparing various neural systems, a few factors need to be considered.

- Number of layers or weight matrices
- Learning rule used
- Neural connectivity architecture used
- Offline or online learning
- Number of iterations taken to learn a sequence: single or multiple
- Global or localised learning
- Feedback, feed forward or lateral connections

- Excitatory or inhibitory connections
- Unipolar or bipolar weights used
- Linear or nonlinear activation function
- Neural coding scheme used
- Sparse or dense weight matrices

2.9 Learning algorithms

This section contains an overview of the major neural learning algorithms as have been developed so far. All the learning algorithms are to govern how the parameters of the neural network, mostly the weights, should be changed so that the projected output should be closer to the desired one.

2.9.1 Unsupervised and supervised learning

Unsupervised or self organised learning refers to the case where there is no external teacher signal. Instead, it is specified how accurately the network is expected to model the output space, and the parameters like learning constants, weights etc. are fine tuned with respect to this specification. Competitive learning or Self Organised Maps (SOM) are examples of unsupervised learning.

In supervised learning, on the other hand, there is an external teacher signal which checks how close the network predicted output is to the desired output, and the parameters of the network are modified as per this signal. The effort is made to tune the parameters so as to minimise the error in the future. Gradient descent is one such type of learning algorithm.

2.9.2 Hebbian Learning

This is based on Hebb's postulate of learning, which states that if the firing of one neuron induces the firing of a connected neuron, the strength of the connection between them is increased, and vice versa. This learning is highly localised and hence, scalable.

2.9.3 Competitive learning or self organising learning

In this type of learning, the output neurons of a network compete among themselves, and the neuron with strongest activation fires first. Such neurons typically have lateral inhibitory connections between neurons in the same fascicle. This is an unsupervised way of sorting the activations of neurons in a layer.

2.9.4 Minimum error learning

This type of learning is supervised as it is necessary to know the exact value of the error, which is the difference between actual and desired output of a neuron. This type of learning seeks to minimise the error value, typically by an adjustment of synaptic connections or weights.

2.9.5 Probabilistic learning

In this type of learning, a neuron states are changed according to a probability when depends on, among other things, some measure of the energy of the network. Boltzmann learning is an example of such probabilistic learning. These are inspired by statistical mechanics.

2.10 Neural structures

This section gives a brief overview of some major neural architectures.

2.10.1 Two layered and multiple layered

All neural networks have an input and an output layer of neurons. Some networks can have one or more neural layers in between the input and output layers. Such layers are called hidden layers. Each layer may contain one or more neurons.

2.10.2 Feedforward, feedback and lateral connections

In feedforward networks, the signals are fed forward from each layer to the next. The flow of information is in one direction only. Feedback connections feed the output signal back into the input. Such networks are called recurrent networks. Lateral connections connect neurons within a layer.

2.10.3 Associative memories

These associate the input patterns with the output patterns. Information is stored in the connections between neurons. In fact, all neural networks can be grouped as a special case of associative memories, although those neural structures commonly referred to as associative memories simply learn the exact associations. The weight matrix is the combined product of the input and output patterns. They try to minimise the total energy of the network.

They are called memories as they store or memorise associations. Correlation matrix memories are a type of associative memories, as also are the BAM and the Hopfield memories.

2.10.4 Hierarchical networks: Backpropagation

These networks use error correcting learning. These have an input and output layer and one or more hidden layers. Backpropagation networks first propagate the input forward through the network to get the projected output, find the error by comparing it with the desired output, and propagate the error backward through the layers by changing the weights so that the error gradient is minimised.

2.10.5 Radial Basis Function Networks

These networks have one hidden layer apart from the input and output layers. Each of the neurons in the hidden layer computes a particular function and the final output is the weighed sum of the hidden layer outputs. Each of the hidden neurons is sensitive to a radial sphere in the state space.

2.10.6 Self organising feature maps

These are unsupervised and based on Hebb's rule. These transform the input pattern into a two dimensional map. Each input pattern corresponds to a region in the two dimensional space represented by the map. These have lateral inhibitory connections in the layers. These are inspired by cortical maps in the brain.

2.10.7 Support Vector Machines

These are feedforward networks that cast the input pattern into a high dimensional feature space and try to construct an optimal hyperplane where it is easy to separate the features.

2.10.8 Content addressable memory

These were proposed by Willshaw. This is a type of associative memory where the address in the state space represents the data stored at that address.

2.11 An analysis of various neural algorithms and structures in the context of this research

The research aims to build a scalable learning system. It is clear that Hebb's rule is the best as far as scalability is concerned, as it is localised. As the aim is to build a neural memory, it will be a type of associative memory. A content addressable memory is a good candidate for the same, as it uses Hebbian learning and learns in one pass. Backpropagation is unsuitable as it does not scale well and there is no online learning possible. The idea of casting the input pattern into high dimensional space and then linearly separating it can be implemented by a support vector machine. Lateral connections are very helpful in sorting activations and may be useful. Feedback and feedforward connections shall both be used. Probabilistic learning is another candidate for setting the weights in learning.

Content addressable memories and other models directly relevant to this work are discussed in greater detail in a later chapter.

2.12 Rate coding and temporal coding

The neural architectures discussed above assume that the inputs and outputs of neurons shall be represented as numbers. True neurons communicate by means of spikes, which are identical voltage pulses. These spike are fired by a neuron when its activation voltage exceeds a certain threshold. Only the time of firing

of these spikes matter. Temporal coding takes into account the time of firing of spikes. On the other hand, rate coding takes into account the rate of firing of these spikes. The numbers representing the inputs and outputs can be treated as the firing rates. Thus temporal coding is a biologically more realistic model of neural activity.

2.13 Pulsed or spiking neural networks

In pulsed or spiking neural models, firing of neurons is modelled as a sequence of spikes fired at specific times. These spikes are identical and the only information conveyed is in their time of firing. They use temporal coding. Any of the above neural architectures or learning algorithms to set the weights can be used in these networks. This research is based on the spiking model of neural activity. This has been discussed in more detail in the next chapter.

2.14 Neural Networks research today

Neural networks is a vast and interesting field. Research is continuing on finding better neural structures and learning algorithms, hardware modelling and use in applications such as pattern recognition. Neural networks, along with genetic algorithms and cellular automata, can be grouped under the broad field of evolutionary algorithms, which forms a part of Artificial Intelligence research.

Neural networks are autonomous and asynchronous by nature. Apart from the considerable uses by way of applications, they are of interest to biologists and people interested in knowing how our brains work, how we behave and how evolution takes place. Neural networks can provide a mathematical insight and a convenient platform to model the same.

2.15 Applications

The distributed information storage, parallel processing and robustness of the neural networks make them suitable for a variety of applications in the fields of speech or image recognition, conventional artificial intelligence applications and many others.

2.16 Conclusion

In this chapter, we have had a comparative analysis of the various artificial neural structures and learning algorithms. We have seen the two main types of coding, namely rate codes and temporal codes. In the following chapter we analyse the spiking model in greater detail as it is relevant to the work.

Chapter 3

Spiking neuron models

3.1 Introduction

In the previous chapter, we had an overview of various ways to model neural networks mathematically. This chapter is an introduction to the spiking neuron model. Since this work is built up based mainly on this model, it is worthwhile to discuss it in some detail.

3.2 Neural Coding

Neurons transmit information through short electrical pulses called action potentials or spikes, which originate in the soma (cell body) of the neuron and travel through the dendrites and axon.

All the spikes look alike. There are two different theories about how the information is conveyed by neurons in the brain: rate codes and temporal codes.

3.2.1 Rate codes

Rate codes assume that the information is transmitted by the neurons through the mean rate of firing of spikes. However, even the mean firing rate could mean average over time, or over several runs, or over several neurons.

3.2.2 Temporal codes

Temporal or time codes assume that it is only the number and timing of the spikes which carries useful information, and the rate of firing has nothing to do

with it.

Thorpe[35] showed that studies in the primate visual cortex proved that neurons could respond selectively to complex visual stimuli only about 150 ms after the stimulus. They argued that the information had to cross about 10 layers of neurons in this time, so each individual processing stage would need to operate in about 10 ms. Such a time period was sufficient for firing at most one spike, since cortical neurons typically fire at rates of about 100 Hz. This implied that there was no time for the neurons to compute the firing rates from the interspike interval, and so rate coding could not explain the phenomenon. This proved temporal coding schemes to be more plausible in the explanation of visual recognition.

In the last few years, temporal spiking models have received greater attention. A number of books([42], [43], [16]) have been recently published which analyse these models in detail.

3.3 Spiking neurons

3.3.1 Shape of a spike

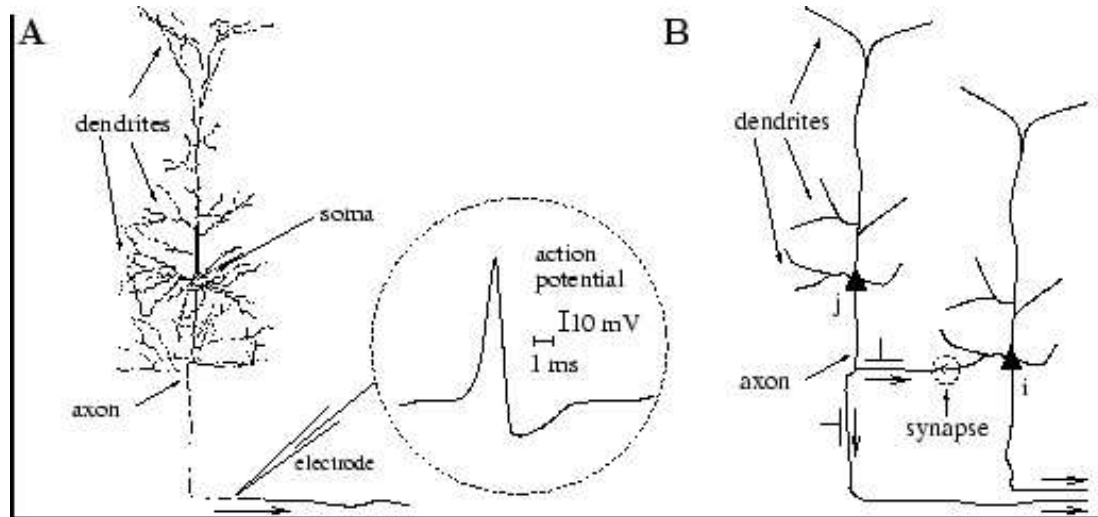


Figure 3.1: Shape of a spike (taken from Gerstner: Spiking neural models)

The shape of a spike is shown in the figure. When a spike gets an input (presynaptic) spike, its potential increases, till it exceeds a firing threshold. As soon as the potential becomes above the threshold, the neuron fires a spike. Its potential shoots up momentarily, but comes down almost instantly. For a certain

period of time called the reset period, the neuron is unable to fire any other spike. For some time after the period too, the neuron can fire a spike, though it would need a higher than average presynaptic pulse to do so. This period is called the refractory period, and this property is called the refractoriness of the neuron.

The firing time of a spike is defined as the time when the neuron membrane potential crosses the threshold from below. The sequence of firing times by a neuron i is defined as the spike train of neuron i .

3.3.2 Information in a spike

The spikes also called action potentials or pulses, are largely identical in shape and size. The timing of the pulses conveys information. In most mammalian neurons, pulses are the actual means of communication. So this model is biologically more realistic than the other rate based codes discussed in the last chapter. This model follows temporal coding, only information conveyed is through their relative timing.

3.4 Mathematical model of a single spike

By abstracting the complex ionic equations that go on inside a neuron to produce a spike and model its behaviour, we obtain a mathematical model of a spiking neuron.

3.4.1 Simple spiking model

Also called the spike response model, Let the activation of the neuron at time t be represented as $u_i(t)$. The neuron is said to fire when the activation reaches a threshold. i.e when $u_i(t) \geq T$. The neuron fires spikes at times t_i , where $i=1,2,..$. After each firing, the activation $u_i(t)$ reduces with time. Hence the activation at time t would be $\eta_i(t - t_i)$, where η is a constant.

For a network of neurons, the spiking behaviour may be modelled in a similar way. Consider the i th neuron. It gets input spikes from input neurons connected to it. Its activation is the sum of the product of activations due to these spikes and the weight of the connections. Hence the equation governing the activation

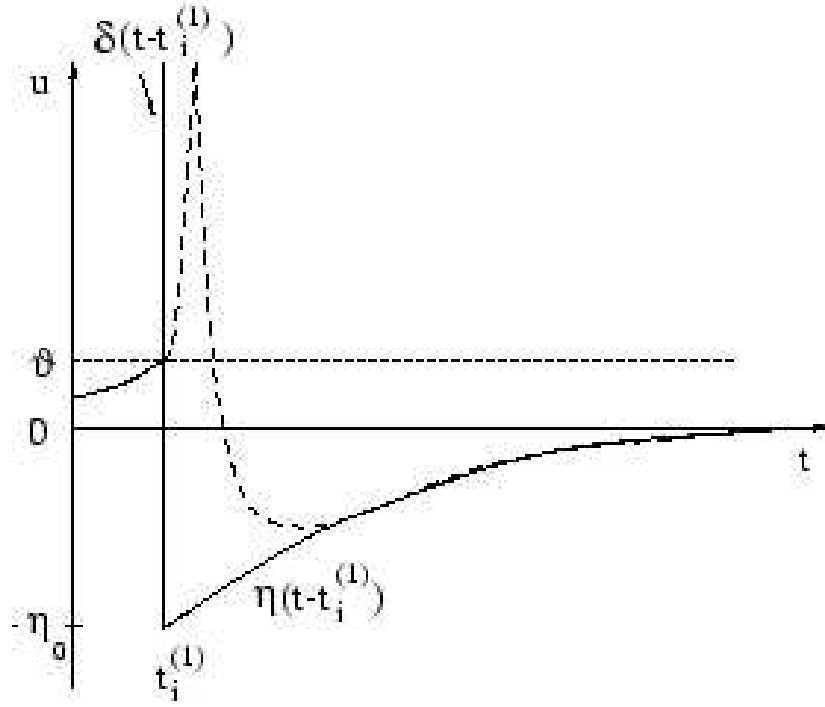


Figure 3.2: Mathematical model of a spike (taken from Gerstner: Spiking neural models)

of this neuron at time t is:

$$u_i(t) = \sum_{t_i \in S_i} \eta_i(t - t_i) + \sum_{j \in \eta_i} \sum_{t_j \in S_j} w_{ij} \eta_{ij}(t - t_j) \quad (3.1)$$

Here η_i represents the refractoriness of the neuron and ϵ_{ij} the postsynaptic potential. The refractoriness η_i resets the potential of the neuron to 0 immediately after it fires and itself decays back to 0 gradually. It may be represented as:

$$\eta_i(t) = -\mu \exp -\frac{s}{\tau} H(s) \quad (3.2)$$

where τ is a time constant and $H(s)$ is the Heavyside function, which is 0 for $s \leq 0$ and 1 for $s > 0$. The kernel η_{ij} gives the response to presynaptic spikes.

The spike response model described above is a generic framework to describe spiking behaviour.

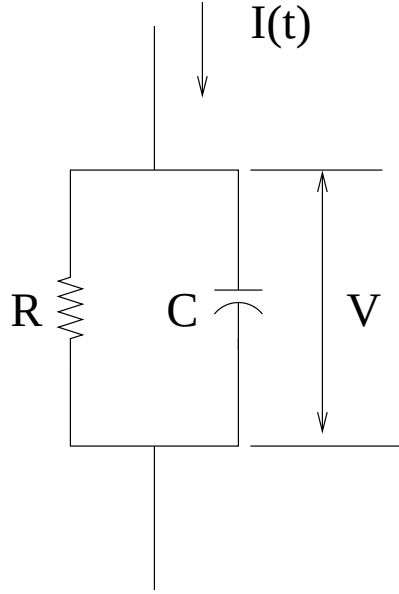


Figure 3.3: Mathematical model of an integrate and fire neuron

3.4.2 Leaky Integrate and Fire Model

The neuron can be modelled by an electrical circuit, with capacitors and resistors. The basic circuit of the integrate and fire model contains a capacitor C in parallel with a resistor R driven by a current I . The equation of the current is:

$$I(t) = \frac{u(t)}{R} + C \frac{du}{dt} \quad (3.3)$$

where u is the voltage across the capacitor C , which models the membrane potential of the neuron. Let $\tau = RC$ be the time constant of the model. Thus we get:

$$\tau \frac{dm}{dt} = -u(t) + RI(t) \quad (3.4)$$

This model can be mapped to the spike response model.

3.4.3 Other models

The Hodgkin Huxley model[22] is a low level model of spiking behaviour which seeks to model the ionic currents that cause the spikes. Sodium and Potassium are the main ionic currents which are modelled, along with a small leakage current that represents mainly negative chlorine ions.

The compartmental model[43] seeks to model the spatial structure of the

dendrites and the neuronal dynamics in more detail. The rate model describes the neuronal activity in terms of rate.

3.5 Modelling populations of spiking neurons

Normally, the behaviour of populations of spiking neurons is described by a quantity called population activity, which is the total number of spikes fired by all the neurons in that population in a given time interval divided by the number of neurons. Gerstner [43] has studied large homogenous populations of spiking neurons and modelled them with the help of population density equations, which are partial differential equation that describe the probability that an arbitrary neuron in the population has a specific internal state. In some cases, these density equations can be integrated to represent the whole population.

3.6 Computing with spiking neurons

Maass [42] has shown that it is possible to build networks of spiking neurons that can approximate most common computations performed in the temporal domain. A few examples are given below:

3.6.1 Threshold gate

A threshold gate can be simulated by a spiking neurons. Suppose we need to simulate a threshold gate with boolean inputs, which either fire or do not.

The requirement is for the neuron to fire if the total activation exceeds the threshold. If a neuron i has presynaptic neurons which fire at different times, synchronization problems arise with the timing of firing of the different inputs. Some mechanism is needed to synchronise the firing of the input neurons. Maass showed that such a mechanism is possible by using some auxiliary spiking neurons. Some auxiliary inhibitory neurons are connected to the input neurons whose presence or absence of firing encodes the input bits. Thus any boolean circuit, or in the absence of noise even any Turing machine, can be simulated by spiking neurons.

The driving force for the firing of the neuron however, comes from the input independent excitatory neurons. Another way to obtain synchronisation of firing

times is by some common background excitation.

3.6.2 Coincidence detection

A spiking neuron acts as coincidence detector for incoming pulses[1]. So if the arrival times of incoming pulses encode numbers, the spiking neuron can detect if some of these numbers have almost equal value.

3.6.3 Storing and retrieving information: Synfire chains

Synfire chains[1] are models for networks of spiking neurons that are suitable for storing or retrieving information. They are called so as an almost synchronous firing of neurons in one pool triggers an almost synchronous firing of neurons in the next pool in the chain, which are connected to those in the first pool by a rich diverging or converging pattern of feedforward connections.

3.7 Other issues and simulation

Spiking neurons are of information to neurobiologists as well as computer scientists. How information is encoded in spikes is a subject of interest. Issues regarding oscillations and synchronisation of spike trains, modelling of noise, hardware modelling etc. have also generated a lot of interest. GENESIS and NEURON are simulators of spiking neurons based on detailed ionic models like the Hodgkin Huxley model.

3.8 Conclusion

Thus in this chapter, we have briefly introduced the spiking model of neural activity. This is the basic neural model on which this work is based. The next chapters shall introduce the sparse distributed memory and related models. While discussing these memories, we shall also discuss how to implement them using spiking neurons.

Chapter 4

Related Work

4.1 Introduction

The main objective of this research is the development of a scalable associative neural memory model. This chapter gives an overview of the related work done in this field. Any further related work is stated along with the relevant chapters. Various other work done in the field of associative neural nets is examined in this chapter,

4.2 Associative memories

Associative memory is the name given to the general class of neural networks whose primary function is to associate patterns. On being given any input pattern, an associative memory retrieves as output the stored pattern that matches the input pattern most closely. These memories can retrieve the closest pattern for noisy inputs too.

Associative memory has a very broad definition. Almost all types of neural nets, such as correlation memories, hopfield memories or even backpropagation can be considered types of associative memories. Any neural network which can store associations can be considered as 'memory'.

The associative memory works by storing the correlations between patterns. All the information is stored in the weights of the network.

The most common types of associative memories are linear and feedforward in nature, and use some form of Hebbian learning. They commonly have one-shot learning, which means that they learn in a single pass. The weight matrix is

created by summing the outer products of all the input output pairs.

In any associative memory, perfect retrieval is possible if the input vectors are mutually orthogonal, else cross talk creates errors in the output. When an input pattern is given to the memory, it calculates the output by taking the dot product of the weight matrix and input vector, and thresholding them. The output may have errors. That is basically the same as finding the stored pattern whose correlation with the input pattern is maximum.

Associative memories are distributed and content addressable. There are two main kinds of associative memories:

1. Auto-associative: These associate patterns with themselves. They can retrieve associations of erroneous (noisy) or incomplete patterns.
2. Hetero-associative: These associate two different types of patterns.

All the other memories studied in this chapter are forms of the associative memory.

4.3 RAM, BAM and CAM

4.3.1 Random Access Memory (RAM)

The Random Access Memory is the way computers use to store information. It is also a type of associative memory. It has zero tolerance for input errors. On being given a previously stored pattern (address), it retrieves the data associated with that pattern.

4.3.2 Bidirectional Associative memories (BAM)

These are the same as associative memories, except that they also use bidirectional weights. If an association XY is stored in the memory, pattern X can be retrieved on presenting Y , and vice versa. The structure of a BAM is similar to an associative memory, with two layers of fully connected linear neurons. The capacity of a BAM with N neurons in each of the two layers is $0.1998N$ [38].

4.3.3 Content addressable memory (CAM)

In such memories the address at which the data was stored is a function of the data value, so the data itself was used as a retrieval cue in finding the stored value at that address.

4.4 Hopfield memories

Hopfield [23] introduced a model of associative memory which all transitions and change of weights were such that they decreased a quantity, which he called the energy of the network. The energy was calculated using a special function of the inputs and output vectors, called Liapunov function.

He defined the energy of the network as being a function of the input vector, weight matrix and the output vector. The weight change was till the local energy minima was found.

The weights used were bipolar (+1 and -1). Each neuron could be in one of two states: on (+1) or off (-1).

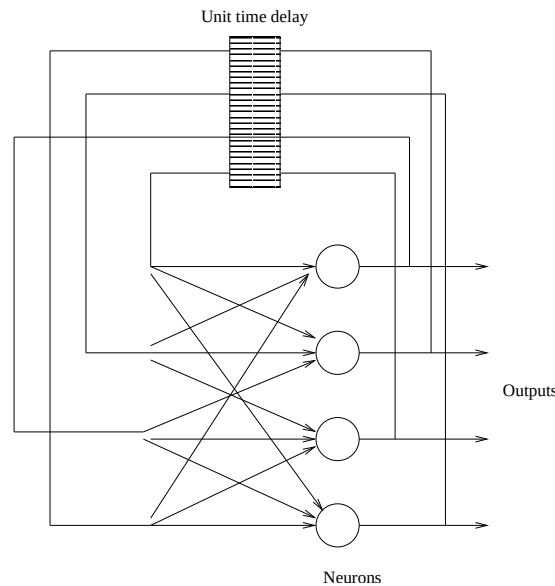


Figure 4.1: A Hopfield Network

4.4.1 Structure

The Hopfield memory was a fully interconnected, one neural layer model of memory. The neurons of that layer had feedback connections to all the other neurons.

Thus it is a single layer recurrent network.

4.4.2 Learning rule

Learning in the Hopfield model was Hebbian. The weight matrix was symmetrical (weight of connection from a to b is equal to the weight from b to a), and was calculated as before, by taking the sum of the outer products of the input and output vectors to be stored.

In recall, the neurons are updated asynchronously, i.e. one at a time, until the network converges to stable states.

4.4.3 Memory capacity

The number of random associations that can be stored in the Hopfield memory has been found to be $0.138N$ [13], where N is the number of neurons. Randomly generated patterns are likely to be mutually orthogonal. If the stored patterns are orthogonal to each other (which means that their dot product is zero) the Hopfield memory gave perfect recall of the pattern most closely matching the input pattern, as long as the total number of patterns stored is within the limit mentioned above. If the patterns are not orthogonal, the retrieved output pattern may be noisy. However, if the number of patterns exceeds the limit of $0.138N$, the memory forgets all the patterns it previously stored and is unable to recall any of them. This drastic memory degradation is one of the major drawbacks of this type of memory.

These memories are more better at pattern completion (if the input pattern is incomplete) than at error correction.

4.5 Correlation matrix memories (CMM)

The CMM is also called linear associative memory. The CMM is a type of associative memory where the outer product of the input and output vectors is calculated and this forms the weight matrix.

These memories were developed by Kohonen[27] in 1972. These can store autoassociations as well as heteroassociations. The data vectors can be continuous.

As the name shows, these memories store the correlation between patterns. The weight matrix is the sum of these correlations. The learning rule is typically

Hebbian.

4.5.1 Structure

The linear associator is the simplest type of associative memory. It has two layers of neurons, fully connected by feedforward connections. The structure of

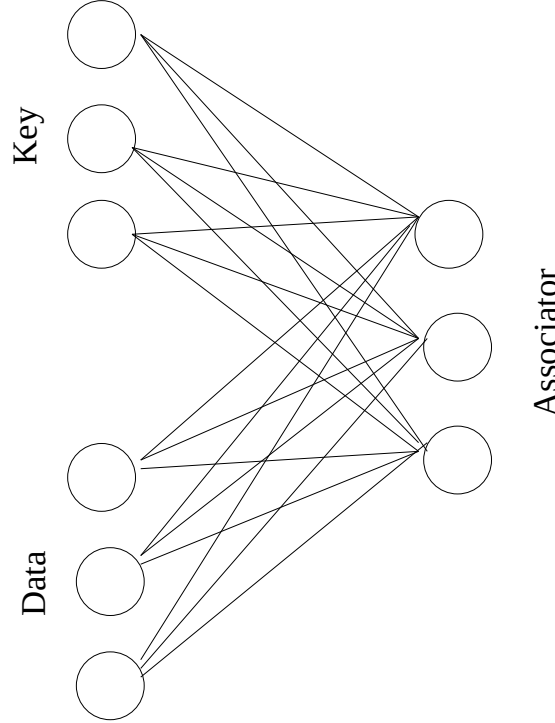


Figure 4.2: A Correlation Matrix Memory

a Correlation Matrix is shown in Figure 4.2.

4.5.2 Learning rule

The learning rule is Hebbian. The connection weight matrix is found by superimposing the correlations of the input and output vectors.

The weight matrix is given by $w_{ij} = \sum_{i=1}^n x_i y_j$, where W_{ij} represents the (i,j) component of the weight matrix and x_i and y_j represent the i th and the j th component of the data and key pattern respectively.

The same can also be written as

$$W = YX^T$$

where X is the matrix whose columns contain the input vectors and Y is the matrix whose columns contain the output vectors.

The output vector y is found by simply taking the product of the weight matrix w and the data input vector x .

$$y = W.x$$

As in other associative memories, the output is accurate if the inputs are mutually orthogonal.

Austin and Turner[5] have done a mathematical analysis of the storage capacity of binary CMMs.

A number of types of CMMs like the Exponential Correlation Associative memory (ECAM)[40] have been developed.

4.6 Willshaw memories

These memories were developed by Willshaw[10] in 1969. Willshaw memory is a type of Correlation Matrix Memory having binary weights (0 and 1). The input and output data vectors are binary as well.

4.6.1 Structure

It consists of a layer of input units connected to a layer of output units by feed-forward connections.

4.6.2 Learning rule

All the input and output vectors are binary. The weight matrix is created by superimposing the outer products of input-output pairs, by using the logical OR operator.

$w_{ij} = \sum_{j=1}^n y_j \cdot x_j^T$ where Σ implies ORing over all the associations stored. x_j and y_j are the data vectors whose associations are to be stored.

This is a form of Hebbian learning, called Clipped Hebbian Learning, which changes a connection weight from 0 to 1 if both the input unit and the output unit are active for the same connection pair.

In retrieval, the weight matrix is multiplied by the input data vector and thresholded to give the output vector. Output = $\theta(W.x)$ where x is the input vector and θ is the thresholding function.

The threshold can be chosen in a number of ways, including Willshaw thresholding, in which the threshold is equal to the modulus of the input binary pattern.

For optimal performance, the data input vectors should be as orthogonal as possible.

4.6.3 Memory capacity

The memory capacity of the Willshaw net has been found to be 0.69 bits per synapse([10], [32]) for a fully connected net with noise free input cues. Jagota[4] and Henson[21] also studied the information capacities of the Willshaw model and related models.

4.7 RAM based or weightless systems

They are also called n-tuple networks. They are suitable for implementation in hardware.

As the name indicates, a weightless system is an associative memory which uses memory nodes whose function is altered by not changing the weights, as in conventional neural networks, but by changing the contents of a memory device. In such networks, instead of information being stored in the weights, it is stored in RAM nodes. Pentti Kanerva's Sparse distributed memory and Igor Alexander's WISARD are examples weightless systems. The main drawback of such systems is that these cannot generalise or interpolate. The function they implement has to be a function of their inputs.

RAM based nets are also associative memories. The input pattern is termed as the 'address' and the output as 'data', to show the similarity to the standard hardware RAM.

4.7.1 RAM nodes

Figure 4.3 shows the structure of a RAM node. A RAM node, like the actual Random Access Memory of a computer, consists of an address decoder and memory registers to store the contents of the address decoders. A given input address activates the corresponding RAM node, and the input data is stored at the memory register corresponding to that address. Such memories are also local and

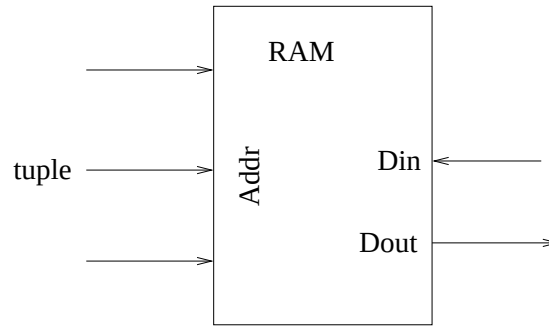


Figure 4.3: A RAM node

hence, scalable. RAM based systems use binary weights and use only a single layer of neurons.

4.7.2 Discriminators

A group of RAM neurons is called a class discriminator. They are called so because they can be trained to discriminate between classes of inputs. We need one discriminator for each class. Each class is taught to its own discriminator only, although while reading from the network, firings of all the discriminators are counted. The pattern belongs to the class of the discriminator with the largest number of firings.

4.7.3 Structure

The network consists of a single layer of RAM nodes. In structure, it is not much different from the conventional computer RAM. N bits are randomly sampled from the input patterns which are otherwise binary, thus forming an N -tuple.

4.7.4 Learning rule

Initially, all the locations of the RAM are cleared. The input pattern is presented on the input address lines of the RAM. The RAM nodes that are connected to active input address bits are set to 1. Thus, the RAM nodes serve as lookup table entries. Learning in a RAM node usually takes place by writing into the corresponding lookup table entries. RAM nodes deal only with binary values. The address and data patterns are usually N tuples.

While writing to the memory, a value is written to the specified address. While reading from the memory, the number of RAM nodes firing is counted. The output is 'read' from the addressed location.

4.7.5 Memory capacity

Adeodato and Taylor[2] studied the storage capacity of RAM based nets.

4.8 Working Systems

The following subsections give an overview of a few systems being researched which are also scalable and have been implemented for useful applications on large scale databases.

4.8.1 ADAM

The Advanced Distributed Associative Memory (ADAM) neural network was developed by Jim Austin[6] at the University of York. It was a CMM based associative memory, originally built for image analysis. It used weightless nodes in N-tuple pattern recognition methods to recognise images. There were two correlation matrices. The correlation matrices were binary and used to store N tuple states. The input image was first pre-processed by a set of N Tuple units, which selected N random bits of binary data to form an N tuple state which was stored in the CMM.

The model used weightless or RAM based nets, described in the previous section. All the matrices, inputs and outputs were binary.

4.8.2 AURA

The Advanced Uncertain Reasoning Architecture (AURA) was also developed by Jim Austin and his team. It was basically an improved form of the earlier ADAM along with symbolic reasoning methods for matching strings in large databases. It is more generalised than ADAM, being applicable not only for images alone, but also for text and others. It is also based on correlation matrix memories and has been used for high performance pattern matching. The correlation memory matrix was formed by taking the cross product of the sum and data vectors

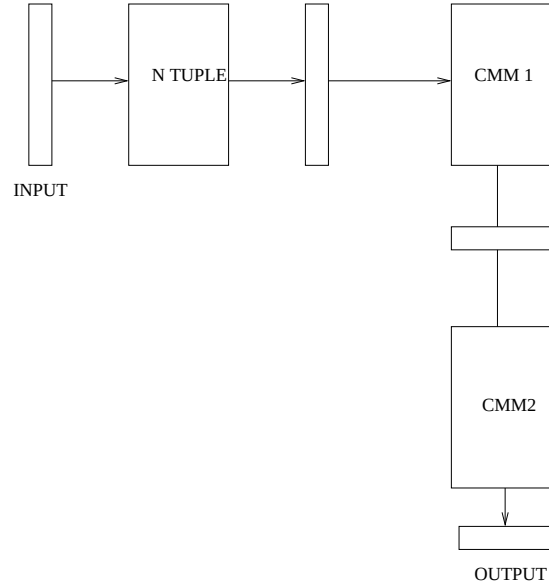


Figure 4.4: ADAM network structure

over the training set. Recall was achieved by taking the product of the correlation memory matrix and the key. It had the characteristics of partial match, online training, scalability, parallel processing, partial matching, and data compression. It has been used on large scale databases, in a variety of applications including a postal address matching system, based on preliminary work done in Lomas'[28]Msc thesis. AURA uses symbolic reasoning to encode data stored in the CMM. In AURA N tuples were replaced by preprocessing.

4.8.3 The WISARD system

Aleksander and Stoneham[3, 24] developed a weightless system called WISARD. This was a RAM based system implemented in hardware. It was a multi discriminator system, which meant that each of its discriminators (RAM like single memory neurons) was trained to respond to a different class of object. It functioned as a general purpose image processing system. The structure of a WISARD system is shown in Figure 4.5.

4.8.4 SpikeNET

SpikeNET is a neural simulator inspired by the human visual system. It is an image recognition system, capable of being ported on to software or hardware. It

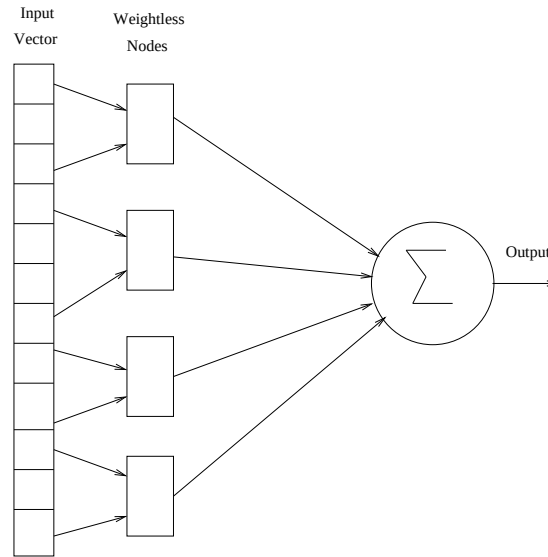


Figure 4.5: WISARD structure

is a simulator for large networks of asynchronously spiking neurons, using rank ordered codes to encode information[35]. It used ordering of the inputs as a way to encode temporal information. It used simple integrate and fire neurons, which had step changes in membrane potential when the inputs fired. It used a sensitivity parameter to modulate the effect of incoming action potentials. The processing was event based. Every firing generated an event, and all the events were processed after each time step.

The neural architecture closely modelled the human visual system, with on centre, off surround connections. It processes images in real time and is highly adaptive, being able to deal with a wide variety of environments and applications. It has been used successfully in reverse engineering of the visual system using on centre, off surround receptive fields, and in face recognition in large databases. It claims to be able to do real time processing of up to 10 frames per second.

SpikeNET was developed by Thorpe[12] and others at Toulouse in France. It has been implemented in parallel hardware and is claimed to be scalable as well. It claims to do robust image recognition for any kinds of images, with tolerance towards noise and light conditions. Natural scenes could be recognised in real time by the software, even though at most one spike could be fired per neuron in that time.

The architecture of the network was modelled on the human visual system. There were a number of layers, consisting of two dimensional arrays of leaky

Integrate and Fire neurons. The input image was first fed to a layer of on centre, off surround units. Neurons of the second layer acted like receptive fields for different angles of rotation of the image. The next layer identified components like the mouth, right or left eye in the face and the final layer identified the whole face. The receptive property to a particular orientation was emergent in those neurons.

Rank order coding was used by desensitising the inputs which came later. The processing model was event driven, with each firing of spikes generating an event. A number of optimisations were used to enable faster processing.

4.9 Conclusion

In this chapter we have had a brief overview of existing literature in this field. We have analysed various software and hardware attempts to create a scalable model. The following chapters shall concentrate on the approach used in this research, namely Kanerva's sparse distributed memory and its modifications.

Chapter 5

Sparse Distributed Memories

5.1 Introduction

This chapter is an introduction to a scalable neural memory model called sparse distributed memory, which was originally developed by Pentti Kanerva. This model forms the primary basis of this work. This chapter contains a brief analysis of the memory model, the address space and its properties.

5.2 Overview

Pentti Kanerva, in his book *Sparse Distributed Memory*[26] developed a memory model in which contents of the memory locations themselves served as the addresses to the memory. His memory had an enormous address space of 1000 bits but required a much less number of storage locations, which he called hard locations. Thus it was sparse. The memory was distributed in nature, as an item of data was stored in multiple addresses. He conducted a study into the mathematical feasibility of this sparse distributed memory model and showed how it could be built out of neuron-like components.

5.3 Kanerva's sparse distributed memory model

Kanerva's model of memory was in many ways similar in functioning to a conventional computer RAM memory. This memory could be implemented in hardware as well as through neurons. The memory had two layers, which were named similar to those of a hardware memory, being called the address decoder layer and

the data memory layer.

Kanerva used the terminology 'data' and 'address' to describe the associated patterns, as it gave the impression of being like a hardware memory. The model was a type of neural associative memory, as it could associate 'data' patterns with 'address' patterns. It differed from a conventional hardware memory in that even if the address given was noisy or partially incorrect, the data retrieved by the memory was more similar to the original data than the noisy address was to the original one. On being given a random 'address' pattern, the memory retrieved the data pattern closest in distance to that address.

In the memory, each data item was stored in many hard locations. So it was distributed in nature. In the same way, many of these hard locations participated in the retrieval of each data item. This made it much more robust to errors. Each of these hard locations stored data in a number of counters, each counter standing for a bit of data. The total number of counters each hard location had was equal to the number of bits in the data, which was equal to the dimensionality of the address space. The counters were like the weightless nodes of the RAM based memories, described earlier.

5.4 Development of the model

5.4.1 The SDM address space

Kanerva explored the properties of a high dimensional address space having n coordinates for every point. He assumed all points in the space to be binary. Thus all the vertices were those of a n dimensional unit cube.

The total number of points in the space could be 2^n . The address space modelled was high dimensional. Metrics of distance, similarity etc were defined on this address space. Distance between two points in this space was defined as simply the number of bits in which they differed, which is the hamming distance. Kanerva used a boolean space of 1000 bits for the address space. The number of hard locations out of this was much less than the total size of the address space, which was 2^{1000} . These hard locations were chosen randomly.

He visualised all the vertices as lying on the surface of a sphere with radius $\frac{\sqrt{n}}{2}$. This is because for every point x , there existed another point y such that the whole surface lay between x and y . That point y lay on the other side of

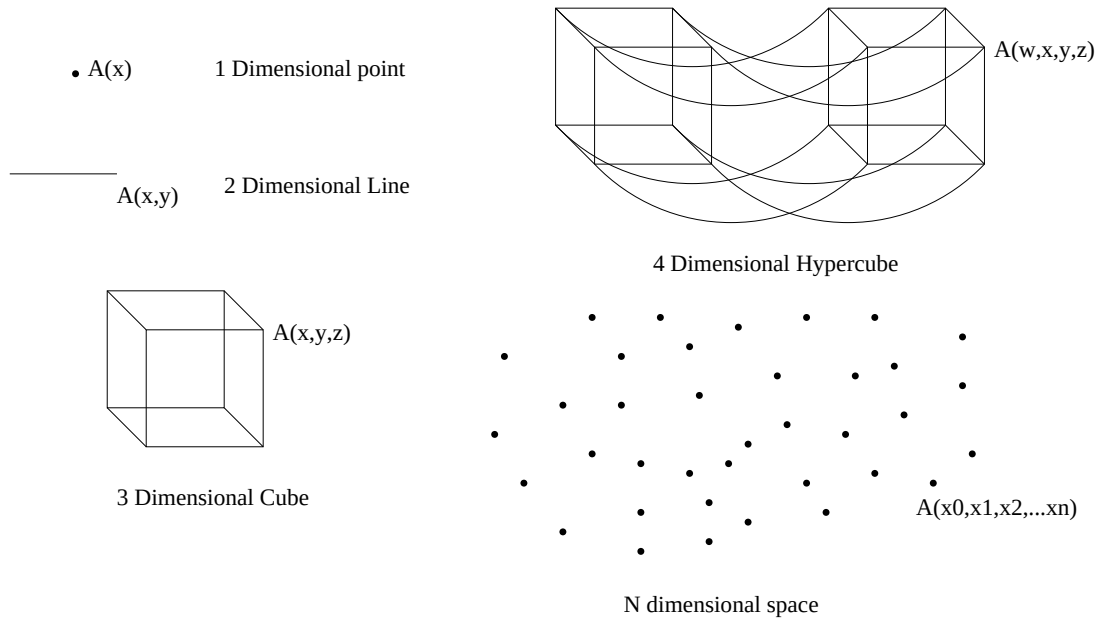


Figure 5.1: N-dimensional space of Kanerva's SDM

the longest diagonal of the n dimensional cube passing through x . He found statistically that most of the space was nearly orthogonal, i.e lay at the mean distance $\frac{n}{2}$ to any given point.

The space functioned as a memory, in the sense that each address and each data was represented as a point in the space. The memory stored each data at a specific address. However, the address at which the data was stored depended on the data itself. Thus it was a Content Addressable Memory as the content of the memory determined what its address would be.

5.4.2 Address decoder neurons

Kanerva visualised the address space could be implemented by neurons alone. The n bit address was input to the neurons. He postulated that a neuron could function as an address decoder for a storage location in that space, if its weights were chosen so as to make it fire on receiving a specific address and the weights were kept fixed. As per the choice of the threshold, a neuron could fire at giving a particular address. By lowering the threshold for the number of address bits needed to match to make a particular neuron fire, a number of address decoders could be activated on receiving one input address.

The total number of address decoder neurons necessary to represent each

possible address in the space was, thus, 2^n . However, If the threshold was lowered, an address decoder neuron could fire for a number of input addresses within a distance from that address, and multiple address decoders could fire for each input address. Thus the address decoders represented storage locations in the memory.

5.4.3 The best match machine

Kanerva proposed that if the number of words to be stored in the memory were much less than the possible total number of storage locations, and if each word was stored at or around the same address location, the memory could be constructed as the words are stored. Thus it would need only one address decoder for each word in the data set, leading to a considerably less number of address decoder neurons being necessary.

Thus for a sparse memory, where the number of storage locations was considerably less than the possible address space, the model performed reasonably correctly. An n bit word was written to a free storage location and an address decoder neuron was set to respond to that storage location, or any location around it up to a specific distance. Once a neuron's weights were set, they did not change.

5.4.4 Sparse memory

However, the above arrangement was impractical with real neurons. So Kanerva proposed the storage locations to be fixed from the start, and distributed randomly in the space. The thresholds of the address decoder neurons representing the storage locations was allowed to vary and thus a memory could be searched for the nearest storage location for a word to be stored. Only the contents of the locations could be modified. The data inside a location was stored using a set of counters. Kanerva called such a memory a sparse random access memory. It is 'sparse' because the number of actual 'hard' locations is much less than the possible address space. The memory was associative in nature, and functioned as a lookup table.

5.4.5 Distributed storage

If the thresholds of the address decoder neurons were also made fixed, each of them would respond to any address within a fixed distance from its original

address. Kanerva conducted a thorough mathematical analysis of the feasibility of such a distributed memory. He found that there was a distance such that, if an address within this distance of the address required was given to the memory, and the data output of the storage locations was fed back as the address again till it converged, the system always converged to the correct data. However if the address given was further than this distance, it did not converge ever. He called this distance critical distance. Thus the memory could predict correctly for a noisy input, as long as the noise level was within the critical distance.

5.5 The final model

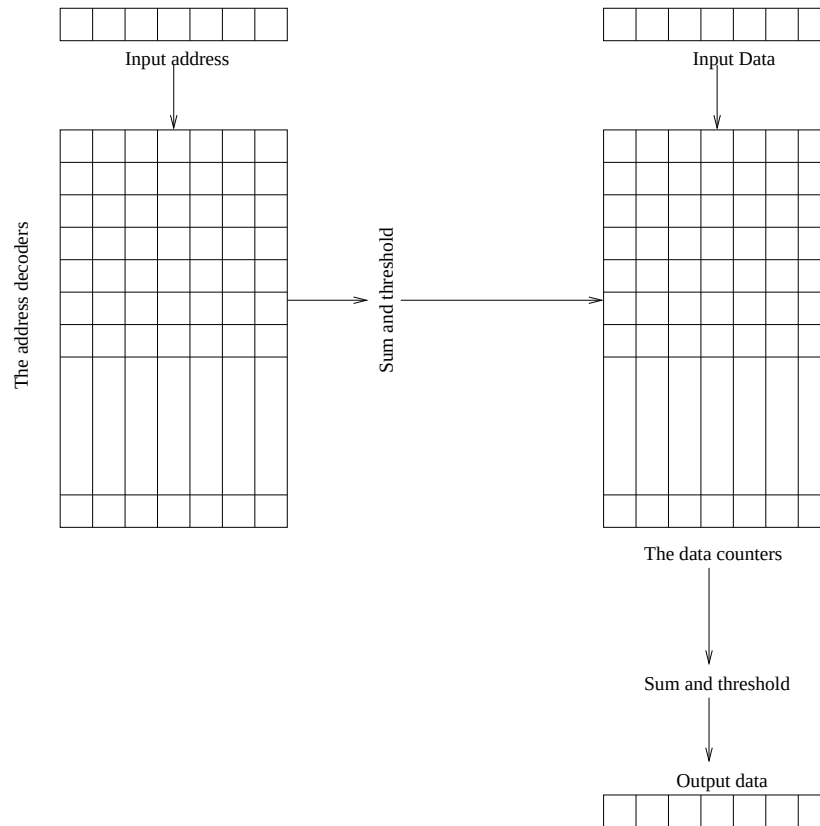


Figure 5.2: The sparse distributed memory model

The final memory model developed by Kanerva had a number of address decoder neurons, whose weights and thresholds were set randomly, so that they responded to addresses within a fixed radius of their weights. Each such neuron had a n bit counter to store the data contents of the address. Each time a neuron

fired on receiving an address, its counter was incremented by 1. The number of address decoders was large though much less than the total address space. Writing a data at an address in the memory implied incrementing the counters of all the address decoder neurons that responded to the input address. To read the data stored at a particular address in the memory, the counters connected to the activated address decoders were summed and thresholded to get the data output.

5.6 Reading and writing to the memory

To access a memory item stored at a particular address, all the address decoders (hard locations) within a specified distance from the address were accessed. This distance was called the access radius of the memory, and all those locations formed an access circle.

5.6.1 Writing

In writing an n bit vector to an address, all the hard locations in the access circle of that address were accessed, and their counters were modified. If the bit being written was a '1', the corresponding counter was incremented, otherwise it was decremented.

5.6.2 Reading

If the data at a particular address was to be read, all the hard locations in the access circle of that address were accessed, as before. Their corresponding bit counters were added and thresholded to get a n bit data value.

5.7 Salient features of the sparse distributed memory model

Thus the sparse distributed memory model as developed by Kanerva had a number of features, such as scalability, robustness and error tolerance and can be modelled in hardware as well as by a neural network. Some of its features are discussed in more details in the following subsections.

5.7.1 Sparse and distributed

The address space of the memory was very sparse, since the number of hard locations were much less than the total address space. Most of the space is at a distance greater than $\frac{n}{2}$ from any given point in the space, where n is the dimensionality of the space. The memory is distributed because each data item is stored at a number of address locations. Similarly, to retrieve any address, a number of locations around that address have to be accessed. Each input address activates a number of storage locations and each storage location is activated by a number of input addresses.

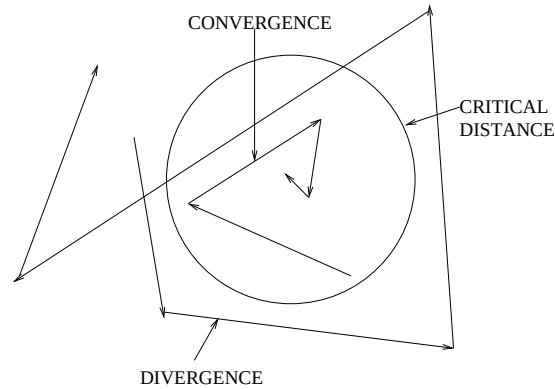


Figure 5.3: An illustration of the convergence of the memory below, and divergence above the critical distance

5.7.2 Convergence property

There is a critical distance for the memory. If the input address is within that critical distance from the actual address, the memory output would converge to the correct data. This means that if the output of the memory is fed back to the input, the successive outputs would converge to the stored input data item. Once the memory converged to the stored data item, it would stabilise and successive iterations would output the same data item. However, if it is beyond this distance, it would diverge. In divergence, the successive recalled data items were orthogonal to each other.

5.7.3 Scalability property

In the memory, learning is done by incrementing or decrementing the counters representing the data. This can be done locally as each address decoder has its own counter. Different components of the memory can communicate asynchronously. Thus it can be claimed to be able to scale up.

5.7.4 Robustness and error tolerance

Since the memory is distributed, each data item is stored in a number of memory locations, which are within the threshold distance from that data item. So it is robust to failure of individual locations. It is also error tolerant, as it can retrieve correct data even if the address given has errors, as long as the number of errors is below the threshold.

5.7.5 Multiple writes

If an address data pair was written a number of times to the memory, it was able to recall the data faster. So the speed of convergence was increased if the data was written to the memory multiple number of times.

5.7.6 Memory capacity

The memory capacity is defined as the maximum number of associations the memory can store and recover correctly. As more and more associations are stored in the memory, it starts to forget the originally stored associations due to crosstalk. A point comes when increasing the number of stored associations did not increase the number of associations the memory recovered, but rather starts to forget more associations than it learns new ones. That point is defined as the capacity limit of the memory. Kanerva found that for the address space of 2^{1000} , the memory capacity was 1/10 th of the number of hard locations.

In a study of the memory capacity, it is important to consider the capacity as a ratio of the memory size, which is the number of bits in the memory. The capacity of the memory is constrained by the number of hard locations but it does not constrain the size of the address space.

Chou[8] did an analysis of the theoretical capacity of the SDM. He found that the capacity of the Kanerva memory had an exponential growth rate with the

dimensionality of the address. This exponential growth rate was comparable to the best in information theoretic codes. However, this growth of capacity was at the cost of an exponential growth in hardware.

5.7.7 Online or one pass learning

There is a separation required between the reading and writing modes to the memory. However, the learning can be done in one pass as the only activity is the incrementing or decrementing of the data counters corresponding of activated address decoders. Thus it is a form of Hebbian learning using weightless nodes.

5.7.8 Learning sequences

A sequence of patterns could be stored and read from the memory. For example, if the sequence of patterns is ABCDE, the associations A-B, B-C, C-D, and D-E were stored in the memory. While retrieving a stored sequence, any of the patterns in the sequence was presented to the memory, and the output was fed back to the input. For example, if A was presented, the memory would read B, which would then be fed back at the input to retrieve C, D and E in the same way.

Instead of associating each output with the present input, the outputs could be associated with the previous inputs too. For example, in the sequence ABCDE, the outputs can be associated with the inputs delayed by two time units, giving the associations A-C, B-D and C-E, which the memory could be trained for.

Another way could be to define a time window, and the output could be associated with an encoding of all the inputs within this time window. In the above example, if a time window of length 2 is used, the associations stored could be AB-C, BC-D and CD-E.

5.8 How it works

Kanerva's sparse distributed memory is basically an associative memory with the address decoder neurons casting the input address pattern into high dimensional space, where there is much more probability of it being linearly separable. This is done in the first layer, the address decoder. The second layer is the main layer used for learning, and it is in this layer that the weights are trained. The

incrementing and decrementing of the counters serve as the weight training and can be done locally asynchronously. Thus it is a form of Hebbian learning.

5.9 Explanation of biological memory phenomena

Kanerva found that the sparse distributed memory model could model accurately many of the phenomena in human memories, in the way we remember and recall things.

5.9.1 Tip of the tongue phenomenon

This could be explained as an item to be recalled being near the critical distance to the nearest stored data item. So a number of iterations were needed before the memory could converge.

5.9.2 Knowing that one knows

The convergence property could be the explanation of this phenomenon. If the item was well within the critical distance, the memory converged rapidly to the stored data item.

5.10 SDM and the cerebellum

The sparse distributed memory was found to have structural similarity with the cerebellum in the brain. The cerebellum takes sensory stimuli as input and coordinates the smooth movement of muscles through the motor neurons. The cerebellum contains a number of Purkinje cells and granule cells. The axons of the granule cells form a dense network of parallel fibres with the dendrites of the Purkinje cells. These parallel fibers are similar to the address decoders in Kanerva's memory. Climbing fibres wrap round the Purkinje cells and form akin to the address inputs. They give very strong inhibitory inputs and when they fire, the Purkinje cell is guaranteed to fire. Thus an SDM has structural similarity to the cerebellum, and this makes it possible to model the function of the cerebellum through an SDM-like model.

5.11 Comparison with a computer memory

The SDM can be compared to a Random Access memory. The difference is that it works for sparse addresses, where the number of address locations is considerably less than address space. The model is also error tolerant, distributed and robust, things that a computer RAM does not have. It also forgets gradually. However RAM is much faster to write and read things from. It is also suitable for the non sparse case, whereas the performance Kanerva's memory would degrade once it starts filling up. Also, the efficiency per bit of RAM at maximum capacity is higher than Kanerva's memory, since RAM stores 1 bit per bit of memory.

5.12 Drawbacks of the memory

Having counters for storing data is not efficient with respect to computation required as well as hardware implementation. The memory is good only for the sparse case and the performance degrades rapidly if the number of items to be filled up grows.

5.13 Conclusion

Thus the Sparse Distributed Memory can be a very useful model to store things in, especially when the number of hard locations is considerably less than the address space, and when there may be errors in the input. The next chapter describes a modification to the sparse distributed memory by using a special encoding of data.

Chapter 6

Sparse distributed memory using N of M codes

6.1 Introduction

In the last chapter we discussed Kanerva's sparse distributed memory model and its characteristics. This chapter introduces a modification to Kanerva's Sparse Distributed memory using N of M codes and analyses the properties of the resulting memory model experimentally. The resulting neural model has higher storage density than Kanerva's original model, low average neural activity and self timing property, which makes it more feasible to implement in hardware.

6.2 N of M codes: an introduction

A valid word in the N of M code has got a total of M components, out of which only N are active, where $N < M$. In the neural case, it translates to saying that there are M neurons in the fascicle, out of which only N neurons fire at any one time. The code is formed by the choice of the N firing neurons out of M. The total number of choices are $(N!)/(M! * (N - M)!)$.

6.3 Motivation: Why N of M codes

N of M codes have a number of advantages. They are self error correcting, as a word is valid if N of the M components are active, else it is invalid. So it is

easier to detect if a codeword is valid or it contains errors. Since the memories are sparse in nature, there is low average neural output. This results in good power efficiency when such codes are implemented in hardware. The use of N of M codes in this model has been inspired by the use of such codes in asynchronous hardware, particularly at the APT group at the University of Manchester[41].

6.4 Information capacity of N of M codes

According to Shannon's information theory[37], the information content of a code word is defined as the logarithm to base 2 of the total number of words that can be represented in that code. In this table the amount of information in an N of M code is compared with binary and other coding schemes. From the above

Coding Scheme	Bits of information
1 of 256	8
Binary 256 bit	256
10 of 256	58
Ordered Binary 256 Bit code	1684
Ordered 10 of 256	79

Table 6.1: Information capacity comparison of different coding schemes of same size

table we can see that the N of M code has a vocabulary larger than a 1-Max code (1 of M code) but smaller than a pure binary code. If order of the bits is also considered, we get a much increased information content in each word.

6.5 Related work

The use of N of M codes as a coding scheme for associative neural networks has been studied by Casasent and Tefler[7]. Davidson[11]has analysed a number of alternative approaches and has developed a neural model for symbolic processing using N of M codes. N tuples have been used in ADAM and AURA systems discussed previously. Associative memories have been discussed in Chapter 2. The Kanerva memory model has been discussed in the Chapter 3. A correlation matrix memory has been used in this model. Nadal and Toulouse[32] have made detailed study of memory information capacities. Sparse coding for autoassociative memories has been studied in detail by Meunier et al[30].

6.6 Development of the memory model

This section discusses the development of the N of M coded implementation of Kanerva's model. One drawback of Kanerva's model was that it used counters to count the neurons, which is expensive to implement. This drawback was removed in the memory by using a correlation matrix memory or a Willshaw memory, which used binary weights rather than counters.

The first layer of the network using the N of M code was identical to Kanerva's. It was an address decoder layer which cast the input address into high dimensional space.

The second layer of the model did not use counters like in Kanerva's model. Instead it used a much simpler max or OR function to increase the weights. The motivation behind this was the ease of implementation in hardware, as it is much more efficient to implement an OR function rather than to keep counters for every bit. The second layer of the fascicle was, thus a form of correlation matrix.

Like in Kanerva's model, the second layer is the actual memory layer, which stores the associations of the patterns. The first layer is simply to cast the input pattern into high dimensional space in order to facilitate linear separability of patterns.

6.7 The N of M address space

The N of M address space is a subset of Kanerva's binary address space, because of the added restriction that each point must have exactly N of the total M binary coordinates set to 1. This gives a total of possible valid N choose M points in the space. This forms a sphere in the original Kanerva's space, with each point at an equidistant Hamming distance from the centre of the sphere.

For example, if a 2 of 5 code is used, a vector [3 1] would be translated to the coordinates [0,1,0,1,0] in the 5 dimensional memory space.

6.8 N of M implementation

The ultimate objective of the research is to implement the memory in hardware, and so the software model has to be modified to make it more efficient with respect to hardware implementation. Neurons are inherently parallel. Hardware

has definite advantages over software in parallel processing. Furber[36] proposed a modification to Kanerva's memory model which used N of M codes instead of the binary codes that Kanerva's model used. Of course, the drawback is that the validity of the code in a hardware implementation has to be checked to ensure that exactly N of the M bits are active at any one time.

N of M coding implies that exactly N of the M neurons are 'on' at any given time to give a valid codeword, and the choice of the N out of M neurons stores the information. The N of M coded Kanerva model had two layers. The first layer was similar to the address decoder layer used in Kanerva's model, while the second was a correlation matrix memory. Other than the coding difference, the memory is similar to that in Kanerva's model.

6.9 An N of M coded sparse distributed memory structure: Structure and function

It is assumed that the fascicles of neurons get spiked inputs and fire spikes as outputs. The presence or absence of the spikes in the neuron makes the code. N of the total of M neurons in the fascicle fire spikes at approximately the same time, and the input code is formed by the choice of the N firing neurons.

Furber's model was similar to Kanerva's model, with a few differences and optimisations. It used N of M coding throughout in the inputs, outputs and weights, whereas Kanerva's model used binary coding. It did not use counters as in Kanerva's model, instead using a very simple rule: each memory location was 1 bit, and the contents of the memory were simply overwritten if a new data word was written to the same memory location. It did not write 0's, decrease or normalise the weights, but used only a OR function to overwrite the contents, which was much easier to implement in hardware.

In structure, the model was similar to Kanerva's. It had two layers. The first layer was the address decoder layer, which decoded the input addresses into high dimensional space. The second layer was the memory, where the associations were learnt and stored.

Figure 6.1 shows the structure of the memory model.

The address decoders to activated by the input could be chosen in two different ways:

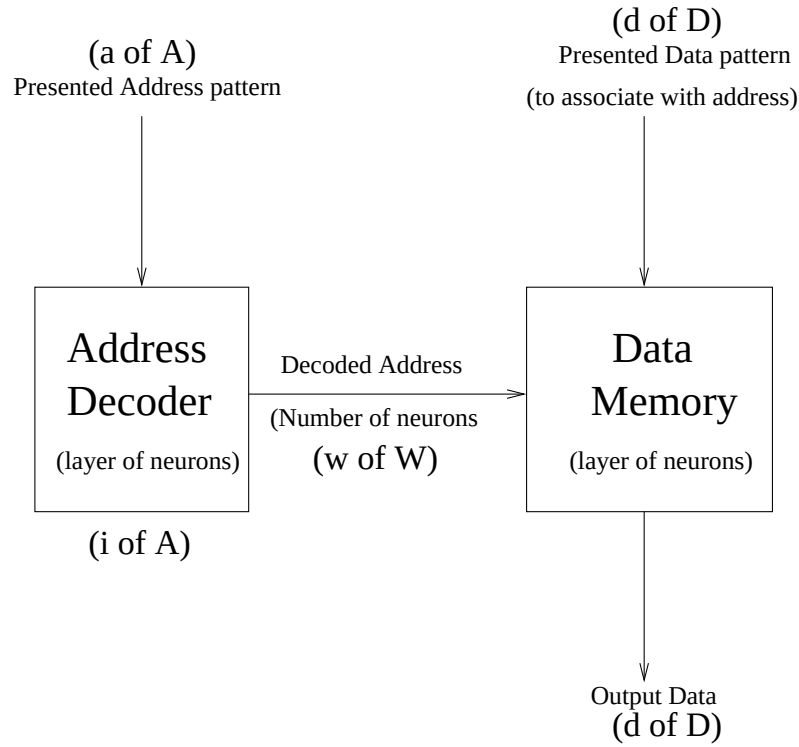


Figure 6.1: The N of M neural memory model

- Choosing a suitable threshold
- Sorting the activations of the address decoders to select the top few.

Furber showed through statistical methods that the number of address decoders firing could be roughly predicted as a function of the threshold chosen.

Figure 6.2 plots the number of address decoders firing (w) as a function of the address decoder threshold (T) as a result of experimental simulation. From the graph we see that for any value of threshold, the number of address decoders firing increased with increased number of bits set, till virtually all of them started firing. This is reached much quicker when the threshold is small.

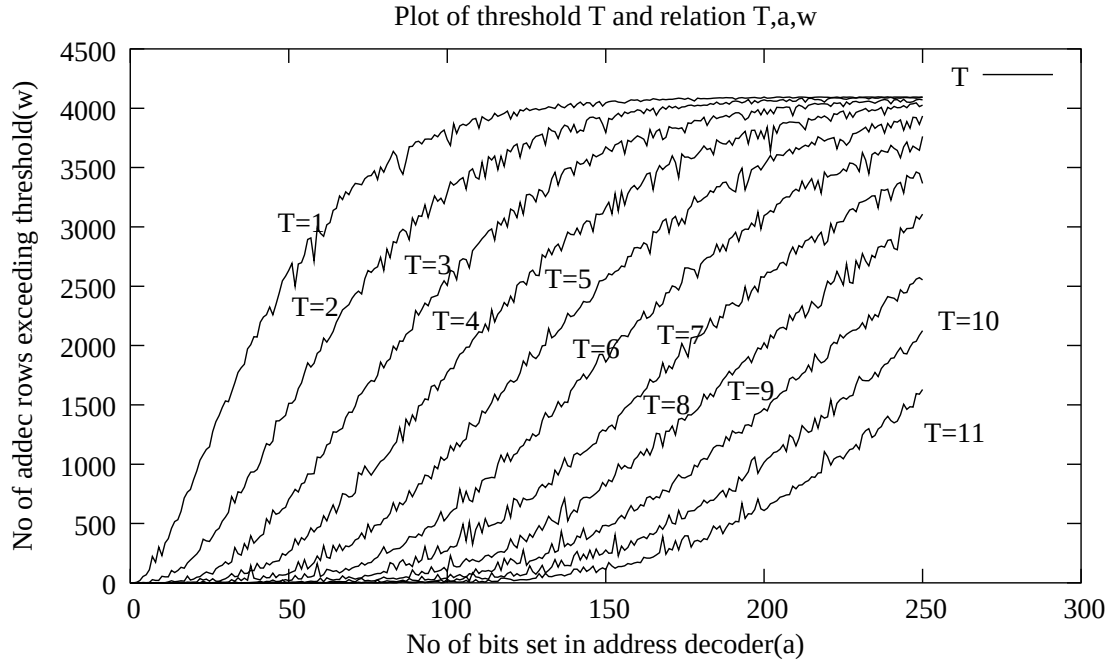


Figure 6.2: Plot of the spread in w with address decoder threshold T

6.10 Reading and writing to the memory

6.10.1 Writing

Assume that the memory consists of W address decoders and the input data is a d of D coded pattern. There are D neurons in the data memory. To cast the address into high dimensional space, W is chosen to be much larger than D . The input address is an i of A code. Here i may or may be assumed equal to d and A to D , if the addresses and data are similar in structure, as in Kanerva's memory.

In writing, an address pattern was to be associated with a data pattern. The i of A coded address pattern was input to the address decoder layer, whose weights were each fixed to a random a of A code. This resulted in w out of the total of W address decoders firing, with w as a function of the threshold chosen, T . Initially the data memory, which was the second layer in the model, was empty, so its weights were set to 0. The d of D coded input data was presented to the D data memory neurons. Writing to the memory took place by the following simple rule: if the k th data memory neuron was 'active', meaning that if the k th bit of the data input had been

set, and the i th address decoder neuron fired, the i th weight of the k th neuron was set to 1.

Basically this implied taking the cross product of the write data array with the address decoder firing array, and using it to set the weights in the data memory weight matrix. Thus writing was like in any correlation matrix memory.

6.10.2 Reading

In reading from the memory, the input address was presented to the memory and the data corresponding to that address was supposed to be read out. We assume that the associations had been written to memory earlier. Reading proceeded as follows: first the i of A input address was presented to the W address decoder neurons, resulting in w of them firing, the same way as in writing. The w neurons to fire could be chosen either by sorting their activations or by choosing a threshold such that the number of neurons firing would be approximately equal to w , although exact w cannot be guaranteed. The data memory output was taken by taking the dot product of the address decoder firing array with the data memory weight matrix, as is normal for a CMM associative memory. Since the output data had to be a d of D code as well, the resulting activations were sorted and the d maximum activations chosen.

6.10.3 Hebbian Learning

The learning in this system is Hebbian, as the weights in the data store fascicle are increased only if both the address decoder and the data input neurons fire together. Thus the connection strength is increased if the pre and postsynaptic neurons fire simultaneously. The resulting learning is Hebbian and localised in nature. This making the model scalable.

6.11 Equivalence of the address space with that of Kanerva's model

The N of M address space is like a subset of Kanerva's memory space, because of the extra restriction that each point must have exactly N of the total M binary coordinates set to 1. This gives a total of possible valid N choose M points in the space. This forms a sphere in the original Kanerva's space, with each point at an equidistant Hamming distance from the centre of the sphere.

6.11.1 Distance, or similarity between two codes

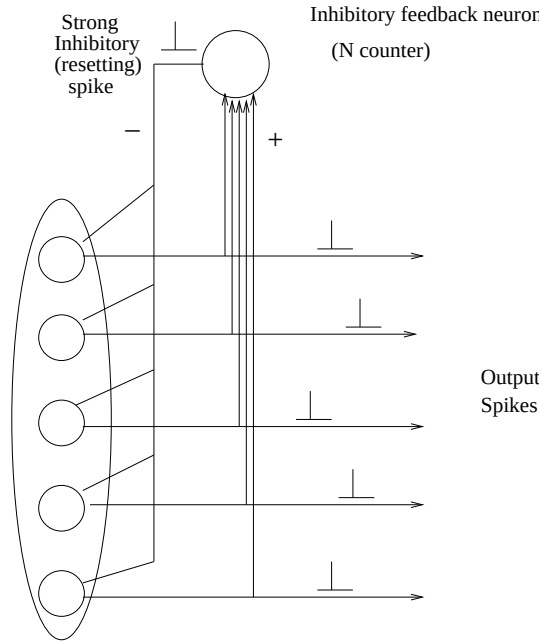
In comparing two binary codes, the Hamming distance is a good measure. It was used in Kanerva's original model. It is defined as the number of bits which differ in the two codes. In N of M codes, the number of 1's used is fixed. So the Hamming distance between any two codes would be twice the number of bits set in the first code but not in the other, or vice versa. A way to measure the similarity would be to take the dot product of the binary vectors of the two codes to be compared.

6.12 Implementing N of M code with spiking neurons

Implementing N of M codes in a fascicle (a group of neurons having similar properties) with spiking neurons requires that exactly N of the total M neurons in that fascicle should fire spikes at approximately the same time to form a valid codeword. The requirement that N neurons fire can either be strictly enforced, or approximately. One way of doing it is to choose the threshold such that approximately N spikes can fire. Another is to choose the N neurons with maximum activation and firing them. This involves sorting the activations suitably to pick out the N neurons with maximum activation.

The neurons are assumed to be self resetting, which implies that a neuron

which has fired resets itself immediately, so that it may not fire again in the same time interval, so as not to give an invalid word.



Imposing N of M code in spiking neurons

Figure 6.3: Feedback inhibition to impose N of M code

Imposing N of M code in a layer of neurons implies forcing N of the total M neurons in the fascicle to fire, and resetting all the rest. This can be done by having a feed forward inhibitory neuron, having a threshold of N and inputs as all the neurons in the fascicle connected with unit weights. Whenever a neuron in the fascicle gets a spike, the activation of the inhibitory neurons gets incremented by 1. When N neurons have fired, and the activation has reached N, the inhibitory neuron sends a strong inhibitory spike to all the neurons. This inhibitory spike resets the activation of the neurons in the fascicle to the default level.

6.13 Sorting

Sorting is a problem when it comes to In the N of M code, it is required to sort the activations of the D address decoder neurons to find the d neurons with maximum activations. Sorting is computationally expensive. One way

to reduce the time spent in sorting is to use lateral inhibitory connections, as in the unsupervised competitive learning algorithm.

Another way is to use time as a global variable to sort the neurons. Mike Cumpstey devised a pendulum model of a spiking neuron and used discrete time to sort the N neurons with maximum activations.

6.13.1 Pendulum model

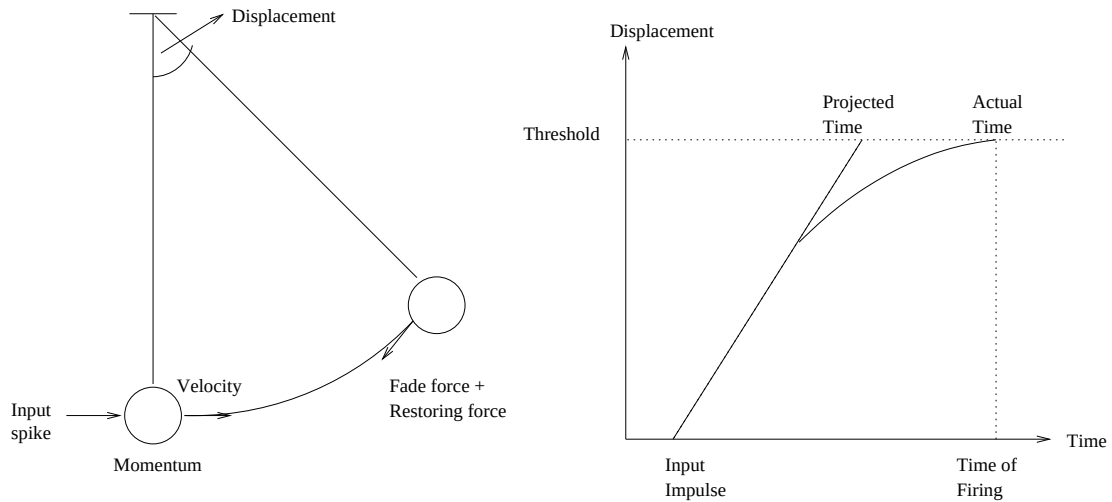


Figure 6.4: The pendulum model of a spiking neuron

In this model, the spiking neuron was modelled in the form of a damped pendulum. Every input spike acted like an impulse, imparting additional momentum to the pendulum. The pendulum starts swinging on getting an impulse. In time, it will swing enough to reach a threshold angle, when it is said to have 'fired'. However, there is also resetting force, modelled by the earth's gravity, which makes the pendulum swing back onto the opposite side after it has swung to its maximum on one side, and a fade force, modelled by the air friction, which tends to bring the pendulum to rest on reaching the state of zero displacement. The restoring force is directly proportional in magnitude and acts in opposite direction to the displacement, while the fade force is proportional to the velocity and also acts in opposite direction to the direction of motion.

6.13.2 Dynamics of the Pendulum model

The dynamics of the pendulum can be described by the following equations:

(Angular) Displacement= x , Velocity= $v = x'$, Acceleration= $a = x''$

At any given time t , the angular displacement of the pendulum is x , say. On getting an input spike the pendulum gets an impulse which increases its velocity. Let the new velocity be $v(new) = v(old) + i$

Let f be the fading constant, which decreases the velocity each unit of time
Therefore, fading force= $f * v$

Let r be the restoring constant, which decreases the displacement
Therefore, restoring force= $r * x$

Hence, after each time step, the parameters of the pendulum need to be updated as follows:

New Displacement = Old displacement + Velocity - Restoring force

$$x = x + v - r * x$$

New Velocity= old velocity - fading force

$$v = v - f * v$$

If a neuron gets a presynaptic (input) spike, its velocity increases by an amount equal to the weight of its connection with the neuron firing the spike.

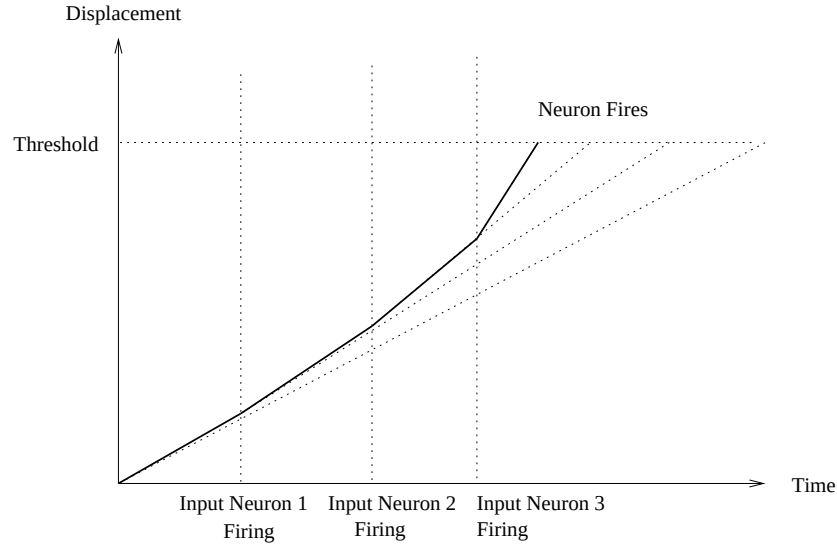
$$v = v + w$$

If the displacement x exceeds a threshold T , the neuron is said to fire. It is immediately reset and all the neurons connected to it have their activations updated.

The parameters of damping(fading) constant and restoring constant need to be chosen suitably so that the system is overdamped.

6.13.3 Using the pendulum model to sort activations

The pendulum model is a technique to sort the activations in linear time. Assume time to be discrete and measured in time steps. The neurons of a fascicle get the input spikes and this increases their momentum. After



Event Driven Processing

Figure 6.5: Event driven processing in Pendulum model: Using time to sort

each time step, the activations of the neurons is recomputed based on their present momentum and adjusting for the effect of the fading and restoring force since the last time step. If any of the neurons has fired, it is immediately reset within that time step. A counter (which can also be modelled by a neuron as shown in the previous section) keeps track of how many neurons have fired. As soon as the counter reaches N , the entire fascicle is reset by sending a very strong inhibitory spike to all the neurons in that fascicle.

6.14 Processing all the neurons in a fascicle

The neurons of a fascicle can be processed in two ways: by having time steps and processing all the neurons in each time step, or by using an event queue.

6.14.1 Discrete time

In the above model, time can be handled either as continuous or discrete. Having discrete time involves time steps. At the end of each time step, all

the neurons in a fascicle are processed and their activations updated. If the activation of any neuron exceeds the threshold, it is said to fire. A counter is used to count the number of neurons that have fired in that time step. If N neurons have fired, the counter signals and fires an inhibitory spike resetting all the neurons in the fascicle.

6.14.2 Continuous time and the event queue model

Continuous time can be implemented using an event queue model. The neurons are processed asynchronously in the order of their firing. Each neuron firing generates an event. All the neuron firing events are stored in a queue. Each node of the queue contains the information about the neuron number and the firing time in the future, calculated by the pendulum model as the time when the activation exceeds the threshold. The event queue is processed serially in order of the time occurrence of events. When a neuron reaches the top of the queue, it is processed. All the neurons which are connected to it have their activations updated and their projected firing times correspondingly changed. Their event nodes are searched in the queue, removed and reinserted in the order of firing times. Thus, the three tasks that are performed in a queue are:

- Insertion of a node.
- Deletion of a node from the middle of a queue.
- Removal of a node from the top of the queue followed by processing all the neurons which are connected to it.

6.15 Mathematical analysis of the memory model

An analysis of the theoretical capacity of the memory using statistical methods has been dealt with in detail in the original paper[36].

6.16 Experimental analysis of the model

In order to analyse the performance of the memory, a number of address-data associations were stored, or written in it. Then only the address was given and the memory output compared with the corresponding data. If both of them were sufficiently similar, which is decided by a threshold on the number of bits which matched, a counter was incremented. In this way the total number of words successfully recalled by the memory was plotted against the number of words fed into the memory. This memory capacity experiment was repeated with a number of memory parameters, including memory dimensions, code dimensions, writing method, etc. being varied, and their results compared to study the effect of varying the parameters.

6.16.1 Default parameters used in the experiments

In all the experiments, the default parameters used to construct the memory were as follows:

- Coding Scheme used was an 11 of 256 code. The words stored were each 256 bit long, 11 of which were 1 and the rest, 0's.
- The size of the address decoder was 4096, 20 of which fired. The 20 neurons to fire were decided by sorting and taking the 20 neurons with maximum activations
- To measure correctness of the recovered output code, it was compared with the originally stored codeword. If both matched exactly, which means that the same neurons were active in both code words, then only was the word deemed to have been successfully recovered.

There are a number of ways to measure the performance of the memory:

- Information Content: This is the total amount of information the memory can hold (and recall). This is the maximum number of words that can be fed in the memory before it starts to forget those it has previously learnt. Figure 6.6 shows a plot of the information content in bits in an N of M codeword with varying N, M being fixed at 256. We see that the maximum information in a word is obtained when a 128 of 256 code is used.

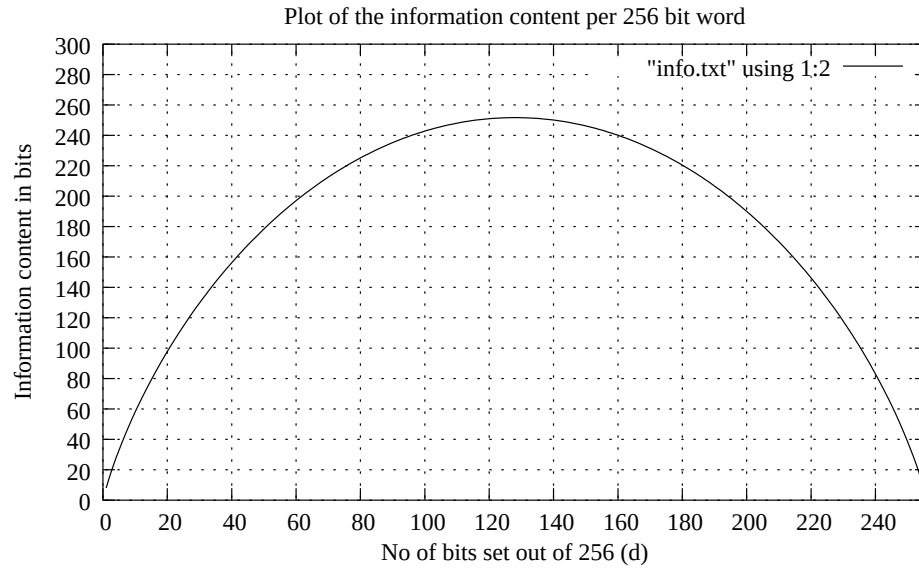


Figure 6.6: Information content of N of M coded words with varying N

- Error tolerance: This measures how much the memory can retrieve the correct word (the recovered word exactly matches the stored word) if the address contains errors or random noise. This is a measure of how the performance of the memory degrades as the number of bit errors increases in the input address. Figure 6.7 shows the results of

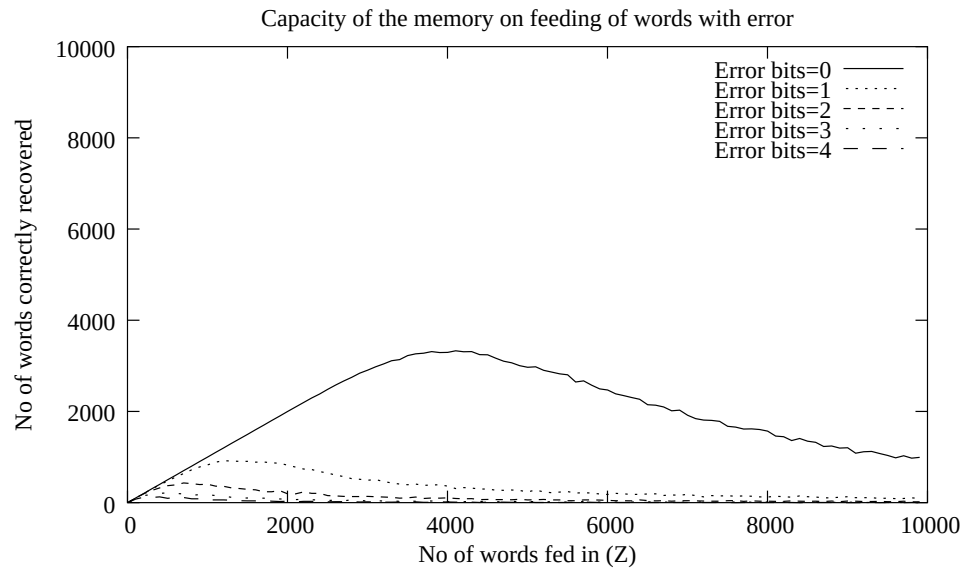


Figure 6.7: Capacity of the memory as the number of bit errors are increased

experiments to measure the error tolerance of the memory. We see that the performance of the memory degrades appreciably as the number

of bit errors are increased, implying poor error tolerance.

However, when a threshold of 0.9 is used (allowing for at most 1 bit error for a recall to be judged correct) rather than an exact match, the capacity degradation is much less.

- Critical distance: Experiments were also done to measure the critical distance, which is the minimum number of error bits permissible before the memory can converge the original stored word after a few iterations. If the number of error bits is more than this, the memory never converges and keeps hitting orthogonal points. The critical distance was found to be 3 bits on using the given parameters.
- Learning and forgetting speed: This measures how long the memory must take to memorise an association, or after how many runs does it start forgetting what it had previously learnt. In this model, one iteration is enough to learn an association. Forgetting takes place when the total number of associations learnt increases beyond a limit.
- Information efficiency (or density) as bits per synapse: The maximum number of words that the memory can hold is found out, either experimentally or statistically. The information content of each word is a function of the parameters chosen and is calculated. This is multiplied by the number of words to determine the total information stored in the memory in bits. This is divided by the size of the memory, given by the size of the weight matrix in bits. This gives the information efficiency in bits per synapse.

In the memory used, the total size of the memory is 4096 times 256, which is the total number of weights. Now from the capacity graph for the default parameters (from the 0 bit error or perfect recall case of the previous graph), we have seen that a maximum of around 4096 words can be successfully stored in the memory (seen from the peak of the graph), which is approximately equal to the number of address decoders. Since an 11 of 256 code is used, the information content in each codeword is $\log_2 256$ choose 11, which is approximately 62.43 bits. This gives an information density of approximately 0.24 bits per bit of information stored.

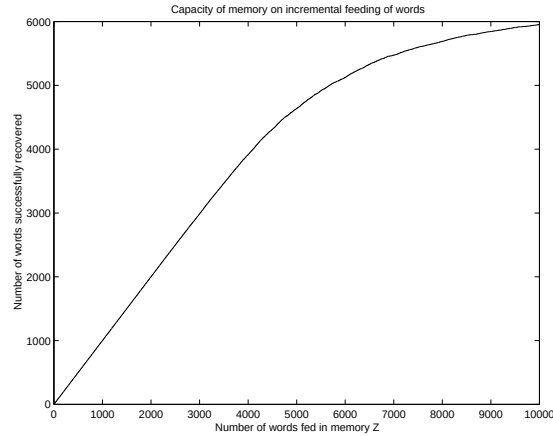


Figure 6.8: Incremental Capacity of the N of M memory

Figure 6.8 is a plot of the incremental memory capacity. It has been obtained experimentally, by storing an additional address-data pair in the memory, and then trying to retrieve the data corresponding to a random given address, which is representative of all the words in the memory. As seen from the graph, the memory recovers correctly (graph is a straight line) for the first few words. Soon there comes a stage when the memory starts to forget the original words it has stored, when the graph starts to flatten out. The derivative of this graph would give the expected percentage of the words correctly recovered as the memory is filled with words. The point at which the graph ceases to be a straight line and starts to bend is the point where errors start appearing. The point where the derivative of the graph starts falling is the point of maximum useful capacity of the memory, as then it can no longer retrieve associations it has learnt previously, and starts forgetting faster as the memory fills up more.

Another way of measuring the capacity is to fill up the memory with a certain number of words and then trying to retrieve them all to see how many words are correctly retrieved. This process is repeated for an increasing number of words. If the number of words being retrieved correctly starts falling, we have reached the capacity of the memory, which can be measured by the peak of that graph.

There are three ways to measure the capacity based on errors:

6.16.2 Capacity without errors:

The capacity is measured by the maximum number of words that can be stored in the memory, such that all of the words can be recovered correctly without errors. In the capacity graphs, this is the point where the slope of the curve showing number of associations retrieved versus those stored, drops from 1.

6.16.3 Measuring capacity allowing for the presence of errors:

The capacity is measured by the maximum number of words that can be stored in the memory before it starts forgetting faster as new words are put in. This is the peak of the capacity graphs shown above.

6.16.4 Allowing for errors and measuring the similarity of the recovered word with the stored word:

The similarity between two words can be measured by the Hamming distance between them. This can be obtained by taking the dot product of the recovered word vector and stored word vector. A threshold can be set here too, say 0.9, and the word may be said to be recovered correctly if the dot product exceeds the threshold, i.e. if the recovered word is sufficiently similar to the original stored word. The memory capacity is measured in the same way as in earlier cases. Figure 6.9 shows the influence of threshold used to measure correctness on the memory capacity obtained. We can see that a lower threshold implies a lesser number of bits matching in order for a word to be deemed correctly recovered, which implies more number of words recovered.

Another way would be to see which stored word the recalled word is most similar to. However, since the points are distributed randomly in the space, there can be no efficient way of finding this except to select the most similar stored word and claiming the word recovered to be that.

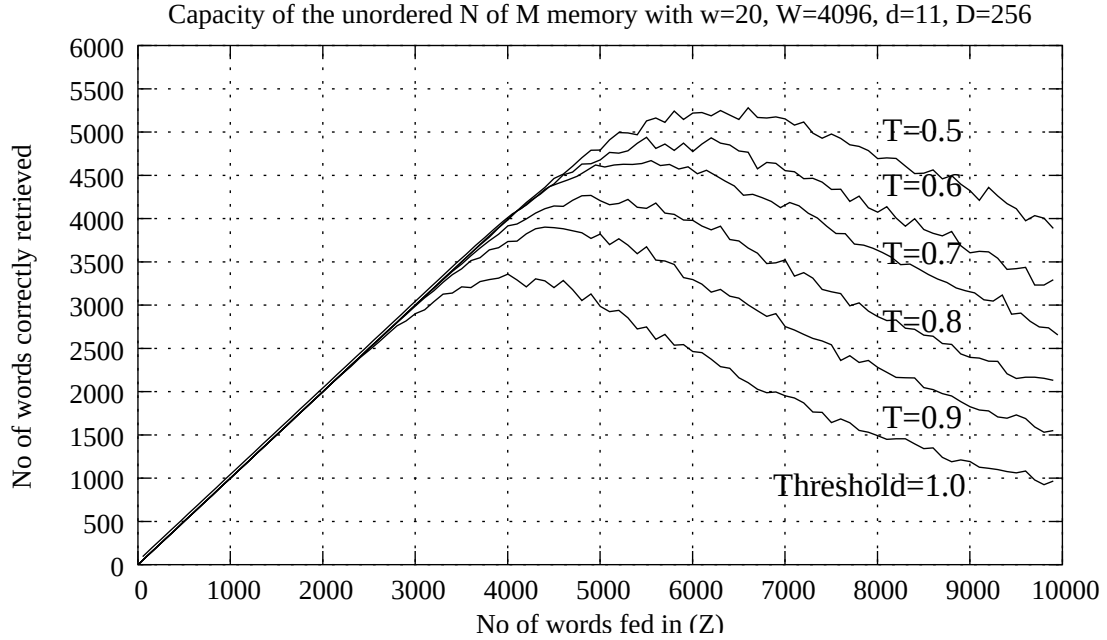


Figure 6.9: Capacity of the memory with varying threshold used to measure correctness

6.16.5 Average recovery capacity:

This measures the average number of bits in the word which are recovered correctly divided by the total number of stored bits. in a way, this is opposite to the above similarity measure, and also more accurate, as threshold used to measure correctness is no longer a factor.

Figure 6.10 plots the recovery capacity of the memory, when the number of error bits is increased. The degradation of the memory performance with error bits is confirmed again here.

6.17 Capacity of the memory by varying the memory sizes

The primary parameters of the memory are d (the number of bits set in the codeword), D (total number of bits in a codeword), w (number of address decoder neurons firing) and W (total number of address decoders).

This section shows the results of experiments made to plot the capacity of the memory when each of the above parameters is varied.

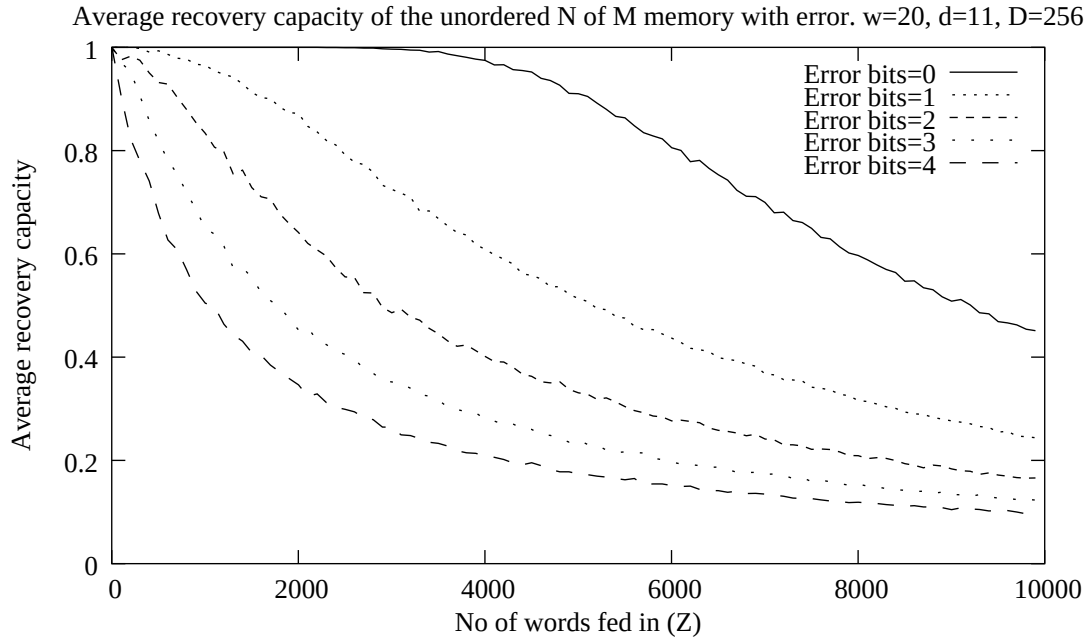


Figure 6.10: Average recovery capacity of the memory with error bits

6.17.1 Capacity of the memory with varying d , with D constant at 256

For the experiments, in the default case an 11 of 256 code was chosen. Now we experimentally check the capacity of the memory with different values of d , and try to find out the optimal value for maximum capacity.

Figure 6.11 compares the capacity when $d=11$ with other values of d . We find that the capacity decreases monotonically when $d \neq 11$. A reason for this could be that with greater value of d , more number of weights are set, and so the memory rapidly fills up.

Figure 6.12 is the plot of the memory capacity with d being less than 11. The capacity, as we see, does seem to increase, but the increase in capacity is not monotonic. A reason for this could be the fact that for very low values of d , for example a 1 of 256 code, the vocabulary is small and this limits the different number of words being fed to the memory. The increased cross talk between words can be one reason for a decrease in capacity for very small values of d . We see that $d=4$ gives the best results (as far as the peak is concerned). However, if we also consider the information in each word, which is d choose D , we find that $d=11$ is not such a bad choice.

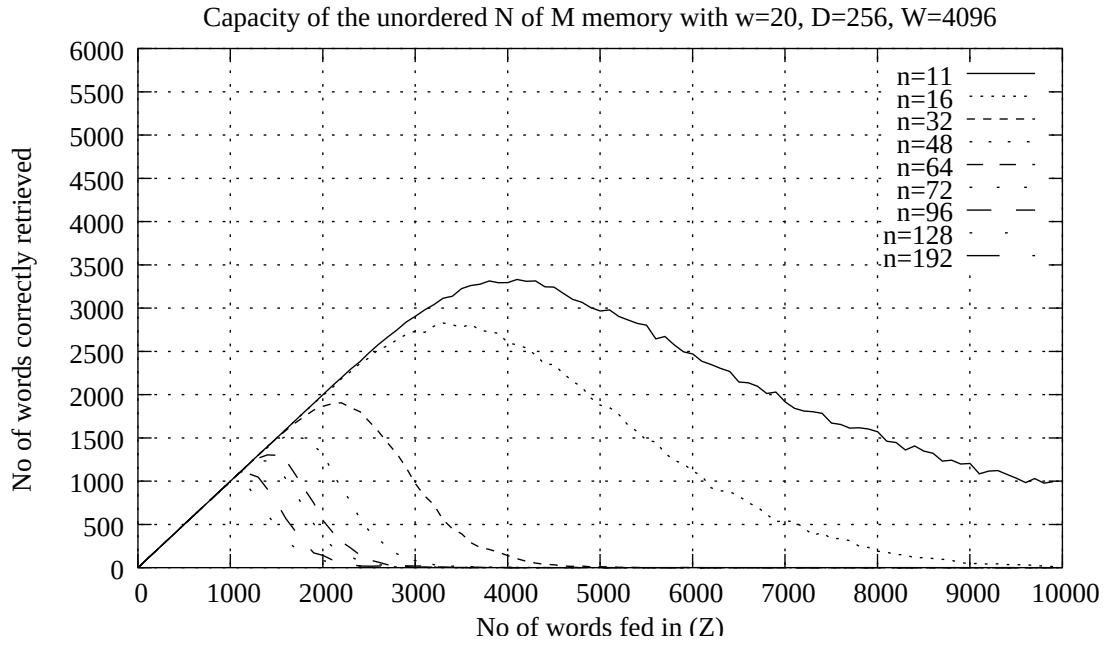


Figure 6.11: A plot of the memory capacity with varying d greater than/ equal to 11

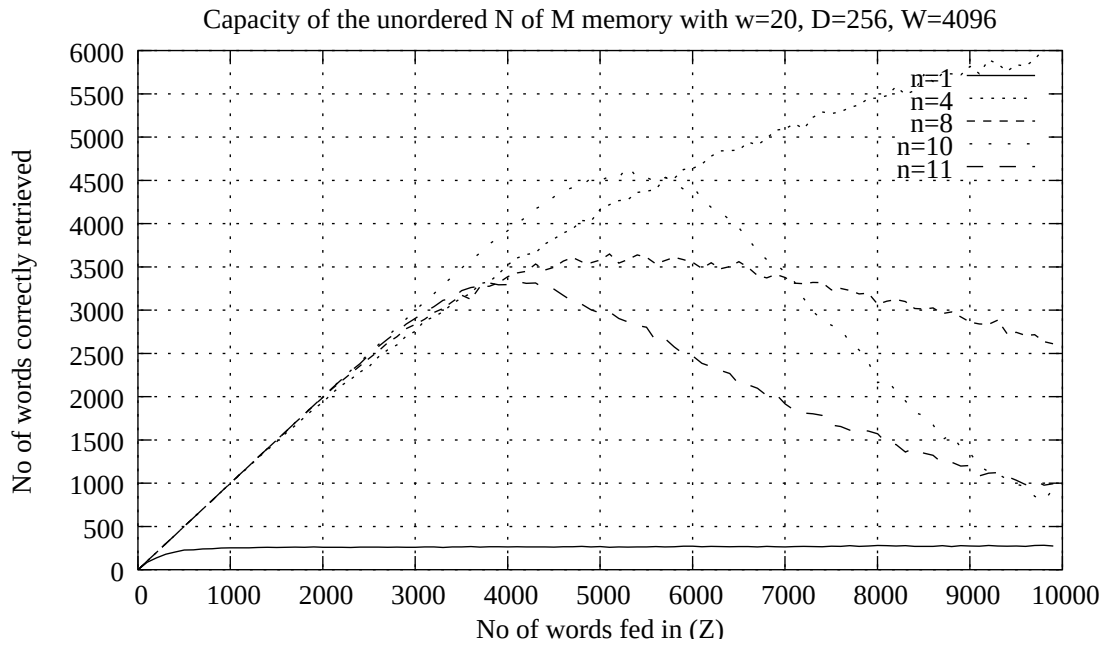


Figure 6.12: A plot of the memory capacity with varying d less than/ equal to 11

Figure 6.13 is the same as the earlier two plots, except that the maximum number of words retrieved is plotted here against d .

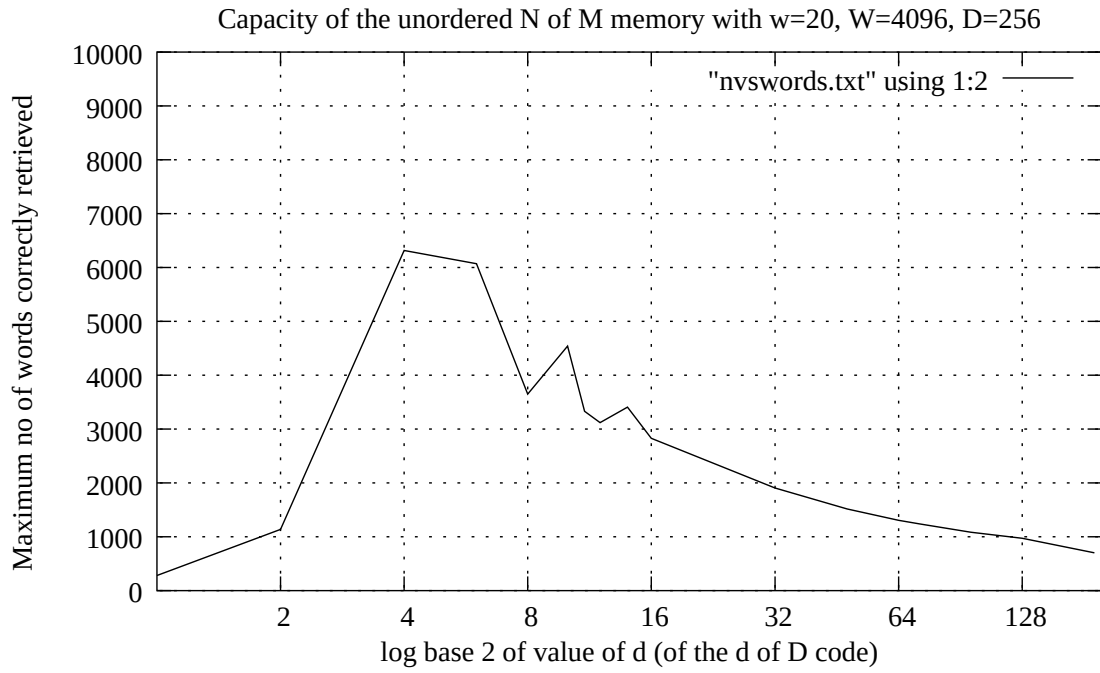
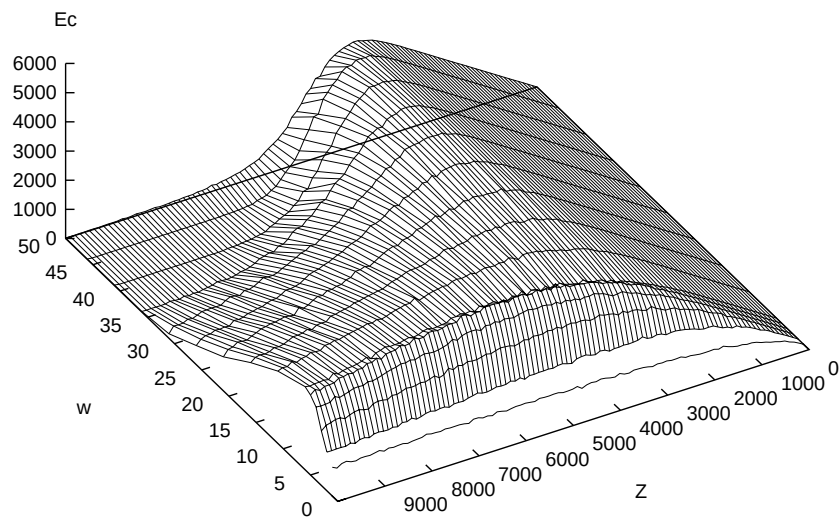


Figure 6.13: A plot of the memory capacity with varying d

6.17.2 Capacity of the memory with varying w , with W constant at 256

A 3D plot of memory capacity ($d=11, D=256, W=4096$)



The 3 Dimensionsl plot in the figure shows the performance of the memory as the number of stored associations successfully retrieved (recovered association matches the stored association upto an efficiency of 0.9, meaning an error of 1 bit at most out of 11), when both w (the number of address decoders which fire) as well as Z (the number of associations stored) are varied.

A 3D contour plot of memory capacity ($d=11, D=256, W=4096$)

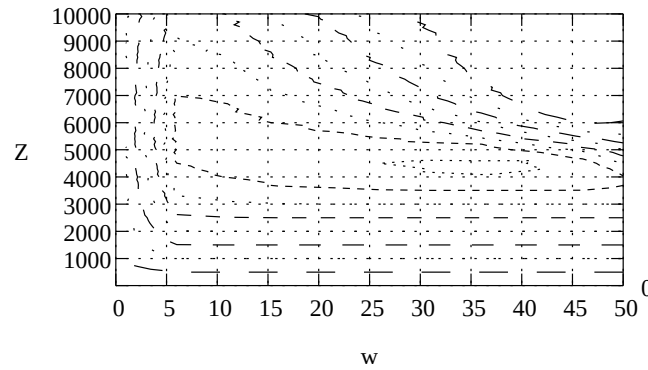


Figure 6.14: A contour plot of the memory capacity with varying w (11 of 256 code) when matching threshold is 0.9

From the contour plot in figure 6.14, we can see that the memory peak capacity occurs between 4000 and 5000 words stored. The number of associations recovered successfully is between 4000 and 5000, though it varies with w .

Figs. 6.15 and 6.16 are the corresponding graphs of the memory capacity, when exact matches are measured. From the contour we see that the peak (the innermost contour) shifts to the left when exact matching is used, as compared to when a matching threshold of 0.9 is used.

6.17.3 Capacity of the memory with varying W

Figure 6.17 is a plot of the memory capacity when W is varied. As is expected, an increased W implies an increased memory size, and so larger number of words can be successfully recovered.

A 3D plot of memory capacity ($d=11, D=256, W=4096$)

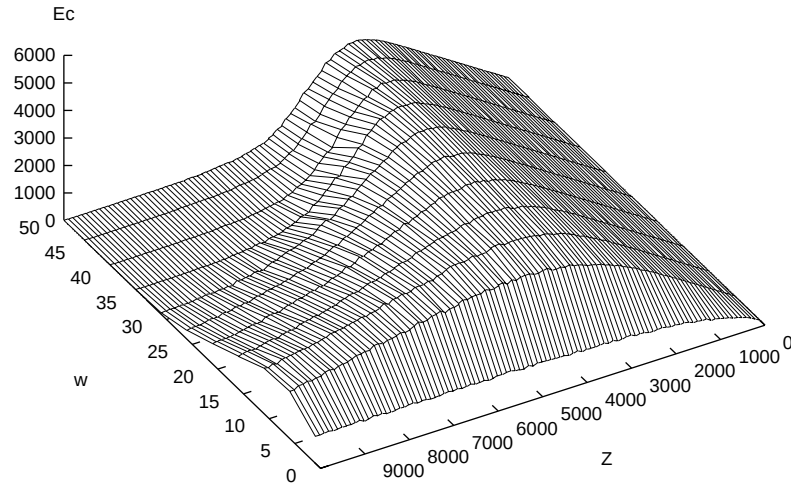


Figure 6.15: A 3D plot of the memory capacity, when exact matching is used

A 3D contour plot of memory capacity ($d=11, D=256, W=4096$)

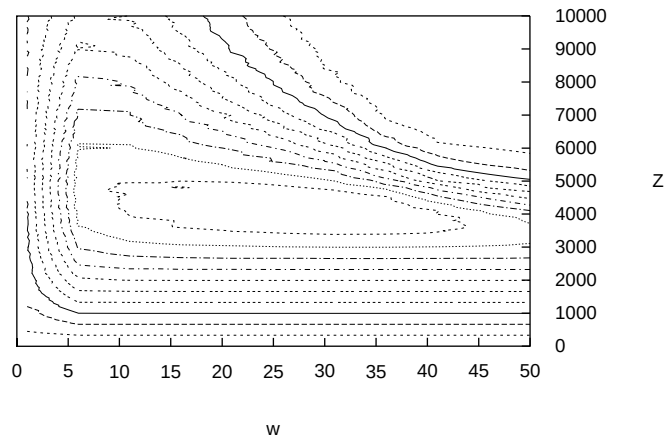
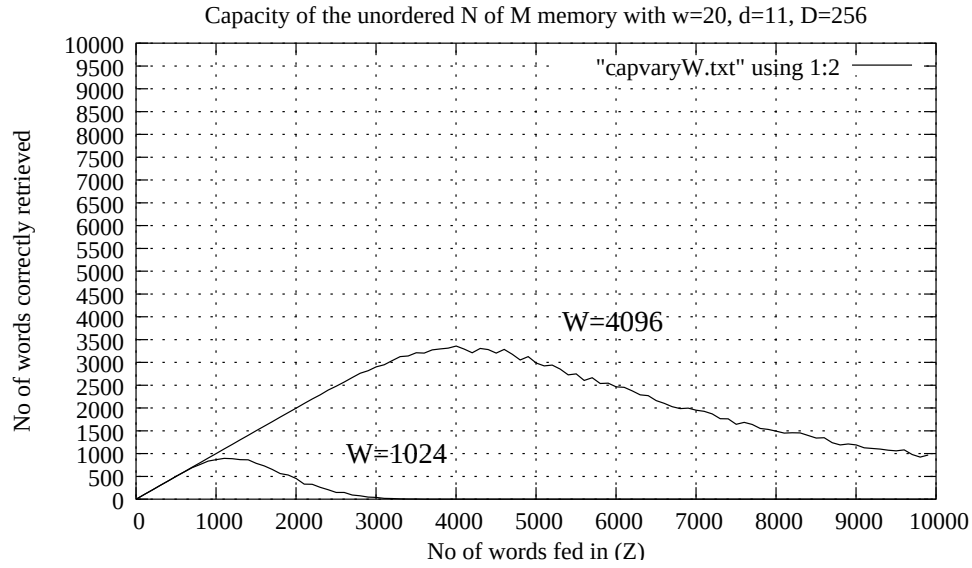
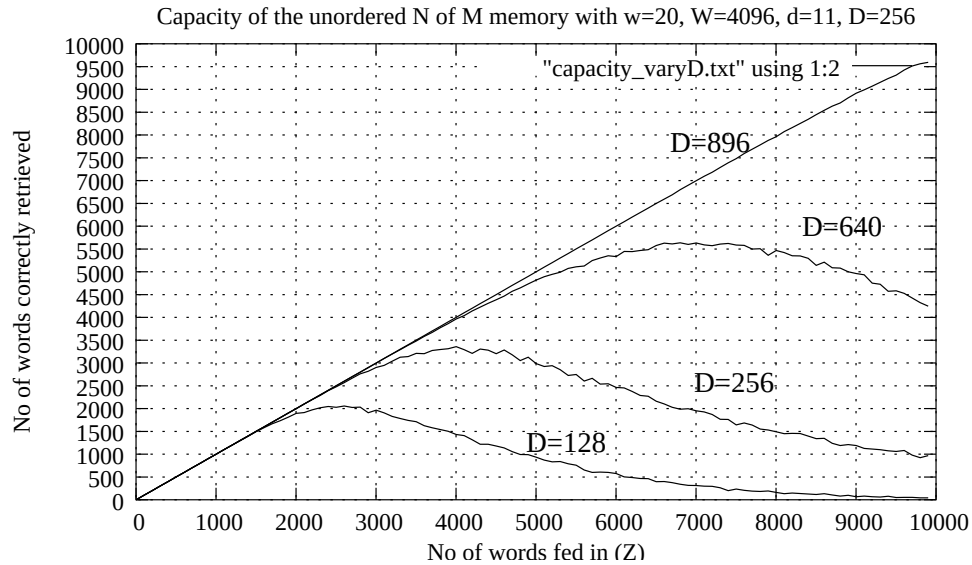


Figure 6.16: A contour plot of the memory capacity with varying w (11 of 256 code) when exact matches are used

6.17.4 Capacity of the memory with varying D

Figure 6.18 plots the capacity with varying D . As with larger W , a larger D also means a larger memory size.


 Figure 6.17: A plot of the memory capacity with varying W

 Figure 6.18: A plot of the memory capacity with varying D

6.18 The convergence property

The critical distance of the memory is measured by observing the greatest number of input bit errors in the address given for which the memory converges to the correct output, if feedback is used and the outputs are fed back into the inputs, till a stable output is reached.

6.19 Feasibility to implement in hardware

The features of this model which make it much more attractive for implementation in hardware are described in the following subsections.

6.19.1 Low average neural activity

Since the memory is sparse, the average neural activity remains low.

6.19.2 Self timing

The N of M code is self timed, as it can detect when there is an error in the code when it is not a valid codeword. This is very useful for the code

6.19.3 Unipolar weights

The need for unipolar weights in this code is quite beneficial from the view of hardware implementation, as they consume less power since less neurons are active at any one time.

6.20 Drawbacks of the model

The N of M memory model has some drawbacks. For one, the sorting presents a bottleneck to the speed of operation. Then, learning is in one pass, which makes repeated associations have no weightage, unlike in Kanerva's memory. The model can even learn to associate noisy addresses and data. Once it has learnt, the model never forgets as the weights are OR-ed and not overwritten.

6.21 Possible modification: Using an N Max of M code

Using a threshold to ensure that no neuron having insufficient activation fires is another way to increase the error tolerance of the memory. The

threshold is set to the minimum activation possible if valid words are written to the memory.

In sorting the activations to find the d neurons with maximum activations, if the neuron activation is less than the threshold, it is assumed noise and not counted.

6.22 Setting of the weights in writing

In the present model, while writing, the weights of the data memory are boolean. While writing to the memory, the weights are set by OR-ing it with 1. This means that if an address decoder neuron and a data input neuron are activated simultaneously, the weight between the two neurons is set to 1. In conventional Hebbian learning algorithms, the rule is to increase the weight between two neurons if both are active simultaneously, or decrease if they are not. In Kanerva's memory, the counters used with each hard location are equivalent to weights. The counters increment and decrement as per Hebb's rule.

The OR function has been used in this case as it is cheaper and faster to implement in hardware. The disadvantage of using the OR function to set the weights, however, is that as more items are stored into the memory, cross talk increases and the memory starts filling up. As a result it cannot recall the associations it had previously learnt, and neither can learn any new associations.

An alternative to the OR function is to increase the weights, when the two connected neurons are active simultaneously. The problem with this approach is that the weights would increase in an unbounded fashion.

New weight = old weight + 1

Another option is to take the weighed mean of the original weight and the new weight.

New weight = old weight * k + new weight * $(1-k)$

where k is a fraction between 0 and 1, which can be either constant or modulated dynamically.

This would not increase the weights in an unbounded fashion. Yet another option is to use Kanerva-style pure Hebbian learning to set the weights, in which the weights are increased as well as decreased, depending on whether the pre and post synaptic neurons are active simultaneously. Some sort of forgetting function can also be used to set the weights back to 0.

Figure 6.19 shows the capacity of the memory when each of the above approaches are used. As we see, the capacity of the N of M memory where the OR function is used, seems to be the best.

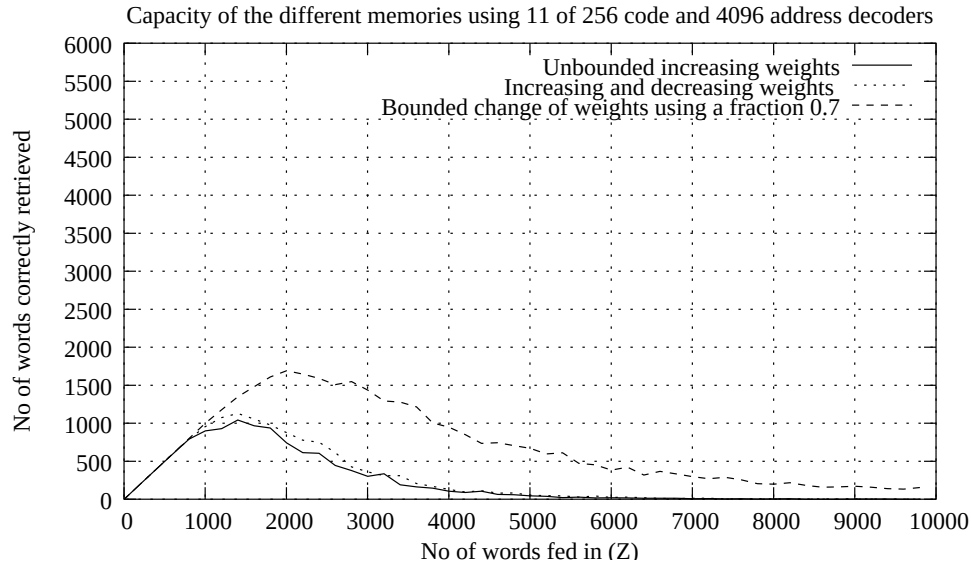


Figure 6.19: A plot of the memory capacity when different methods of varying weights are used

In all the above cases, binary(0 and 1) rather than bipolar (-1 and +1) weights had been used. Binary weights are much more suitable for hardware implementation, as they use low power.

Forgetting is gradual in the memory. Once a weight is set, it is not changed. The only way the memory can forget a word is if more words are written on top of the word such that the activation falls below a threshold. However, one drawback of this model is that it can also learn errors in a single pass.

Schwenkar and Palm[15] studied the memory capacity of autoassociative memories with binary sparsely coded patterns, when the retrieval was iterative rather than 1-step. They found that the completion capacity for one step retrieval using additive Hebbian learning was $1/(8 \ln 2)$ in the

asymptotic case, which was slightly more than the capacity on using binary Hebbian learning, which was $(\ln 2)/4$. They also introduced threshold control strategies for iterative retrieval.

6.23 Use of the memory in the learning of a sequence

In this section we explore the usefulness of the memory in the case of sequence learning, which is the storage and retrieval of a sequence of patterns. If a sequence of patterns, say A,B,C,D is presented to the memory, it can learn the sequence if the outputs of the memory are fed back to the inputs. The memory learns to associate the address-decoder encoded form of the first pattern with the second, the second with the third and so on.

A' $\rightarrow$ B
 B' $\rightarrow$ C
 C' $\rightarrow$ D

Thus we see that storing a sequence is equivalent to storing a number of associations, which are pairs of patterns in the sequence. Once the memory has been trained in this way, presenting any pattern in the sequence can enable the memory to retrieve the rest of the sequence.

In the above example, we can recall the entire sequence ABCD by presenting the pattern A at the input of the address decoder.

6.23.1 Learning recurring infinite sequences

To reproduce the infinite sequence

ABCDABCDAB....

which can be coded as $(ABCD)^*$ in finite state machine terminology, all we need to do is to store an additional sequence $D \rightarrow A$ in addition to the three sequences stored earlier, in order to complete the loop.

Thus we see that the memory can learn recurring infinite sequences. For example, $(DA)^*$, which means the infinite sequence DADADA..., all we

need is to make the memory learn to associate $A \rightarrow D$ and $D \rightarrow A$. Of course, a sequence like $A(BC)^*D$ can never be stored as the memory will not know when to terminate and will keep on producing $BCBCBC\dots$ at the outputs. Any sequence with finite number of repetitions, like $ABBB$ can also not be stored as once the memory learns to associate B with B , it will not terminate and keep producing B 's. The capacity of the memory as a sequence machine is thus limited by the fact that there is no extra memory to store the state and the state is considered the same as the output.

The only restriction on sequences that can be stored in the memory is that every pattern stored must have a unique associated pattern. If one pattern is associated to two or more different successors, the stored patterns will interfere and the memory might be unable to recall either of them due to cross talk. For example, if we store both the sequences $ABCDE$ and $MNCXY$, the memory will be unable to determine the unique successor of C . The same problem arises when we try to store a sequence such as $ABBC$, as it cannot uniquely determine the successor of B .

This restriction severely limits the number of sequences that can be stored.

6.24 The memory as a finite state machine

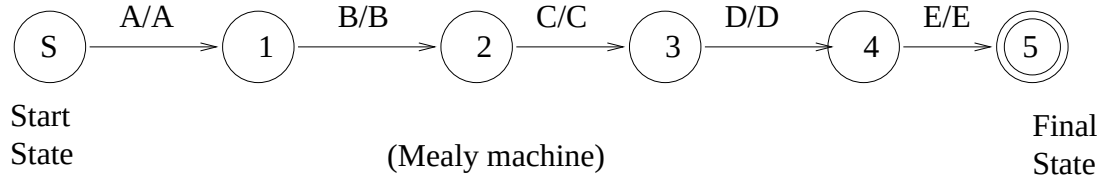
A finite state machine (FSM), as discussed in the first chapter, is an automaton which has a finite number of states and transits between different states depending on the input.

We can define this memory model with feedback as a finite state machine. The state of the system is defined to be the same as the output. Thus structurally, the model is similar to a conventional FSM, except that the state is not explicitly stored.

Figure 6.20 shows the states in a deterministic FSM for storing the sequence $ABCDE$.

The neural memory can also store the same sequence, by storing the associations $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$, and $D \rightarrow E$.

If we compare this memory to a finite state machine, we see that it can recall many sequences that can be stored and recalled by a deterministic



Deterministic finite state machine for storing the sequence ABCDE

Figure 6.20: A finite state automaton for recognising the sequence ABCDE

finite automaton, in which every state has a unique successor state. The added restriction, as discussed above, is that every pattern must have a unique successor in the sequence.

The model cannot have the functionality of a nondeterministic finite state machine, as each state must have a unique successor state for the model to be able to predict the next output accurately.

6.25 Conclusions

This chapter has explored a modification in Kanerva's memory model by using N of M codes instead of binary, and a correlation matrix rather than the counters of Kanerva's model. The properties and implementation of the resulting model have been discussed. The memory is found to have an information density of 0.25 bits per bit, which does not compare unfavourably with other memories, and is also power-efficient, which makes it a more suitable candidate for implementation in hardware.

Chapter 7

Using rank ordered N of M codes

7.1 Introduction

In the previous chapter we saw how the N of M modification to Kanerva's Sparse Distributed memory model increased the capacity of the memory and also made it plausible to implement in hardware because of the average low activity and the self error checking character of the N of M codes. In this chapter we consider a slight modification to the N of M codes using rank ordered codes, and discuss the properties of the resulting modified model.

7.2 Rank ordered codes

A rank ordered code is a code in which the order of the components matters. In the N of M neural model, a valid code is defined by a specific number of neurons of the same fascicle firing at approximately the same time. There is no importance given to the order in which the specific neurons fire. In the rank ordered N of M code, the order in which the neurons fire matters along with the number of neurons which fire. Two code words having the same neuron firing, but in a different order, would be different in rank ordered codes. Thus a word in this code has more information than one in the

unordered N of M code. The rank ordered code is inspired by the temporal coding of neurons. Since it is impractical to measure and store the exact times of the neurons firing, the next best thing is to store the sequence in which the neurons fire.

7.3 Related work

The use of rank ordered codes in the memory model was inspired by the work of Simon Thorpe and his team[35] at the Brain and Cognition Research Centre at Toulouse, France. They have successfully used rank order in spiking neurons in their software, SpikeNET. They have claimed to be able to model accurately the primate visual system and their software can do face recognition on large databases in a relatively short time. The architecture of the neural model used by them is similar to the vision system in mammals, having on-centre, off-surround neurons.

The success of the SpikeNET system in modelling large scale neural networks to perform a number of real life tasks proves that such models are commercially feasible and useful. This makes them worthy of investigation and implementation.

7.4 Why rank ordered codes

7.4.1 The case for temporal coding

Thorpe[12] argued that in certain cases, the neurons in the visual cortex have very less time to convey information to the brain, since they are typically too slow to convey more than a single spike per layer of neurons. Hence rate coding, which for long has been the traditional way neural coding was assumed, is insufficient to explain how the brain accurately recognises visual scenes in such a short time. He argued that temporal coding was thus, in some cases, the only explanation of the recognition of images by the brain in such cases.

Using ordering in codes is basically an abstraction of temporal coding. Thus ordered codes are biologically more plausible.

7.4.2 The information content of rank ordered codes

Rank order is one of the ways in which the timing of the spikes firing can be utilised, rather than simply the rate of spiking. The temporal order of the neurons firing is considered here rather than the actual times of firing, which might lose some information but still gives much more information per word than an unordered code. As we saw from Table 6.1, using the order of the firing neurons in a 10 of 256 code gives an information content of 79 bits as compared to 58 bits in the unordered code. Using a binary 256 bit code gave an information content of 1684 bits as compared with just 256 bits in the unordered case. So we see that using rank order substantially increases the information content in each word.

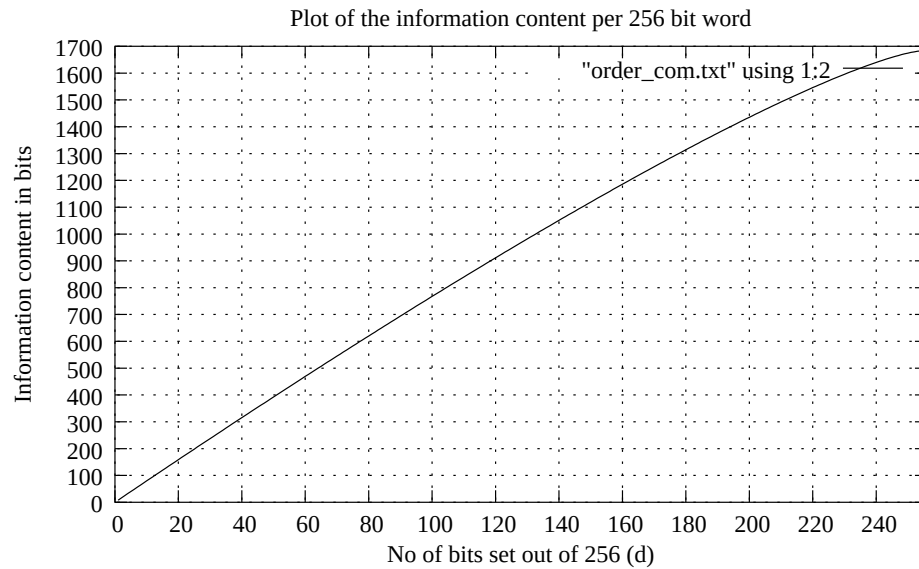


Figure 7.1: Information content of ordered N of M coded words with varying N

Figure 7.1 shows the information content of rank ordered N of M codes. If the rank of firing is considered as well, the vocabulary of the code increases by the factorial N. So the information content in each word increases monotonically with N.

7.5 Ordered N of M Address Space and distance

The N of M ordered address space here is the similar to the N of M address space, with much more points due to the various ordering combinations of the active bits in every codeword. The address space thus becomes much more richer if ordering is used. The ordering of the active bits in any valid codeword can be translated in the space by weighing the bits in order by using the sensitisation factor.

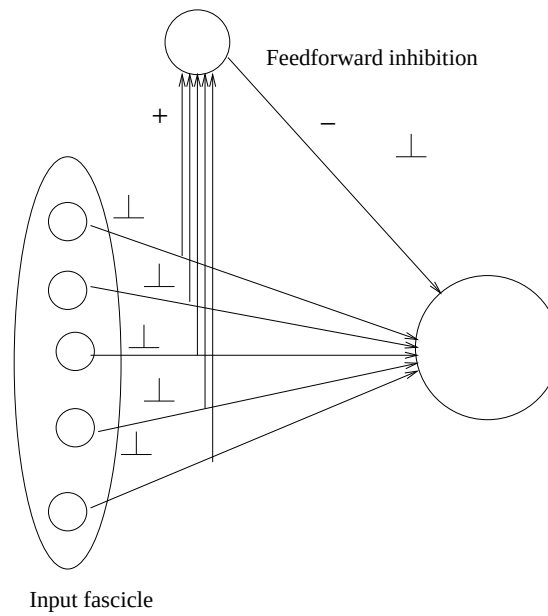
For example, if an ordered 2 of 5 code is used, a point in the space [3 1] would be translated into the coordinates [0, x, 0, 1, 0] where x is the sensitisation factor. The distance between any two points in the space is defined as the dot product of their sensitisation vectors.

7.6 Implementing rank ordered N of M codes through spiking neurons

In chapter 3, we had analysed the spiking neuron model. In the spiking model, the activation of each neuron decreases with time, as shown in the shape of the spike. If the inputs appear in a definite order, their activations would also be ordered in the same way. Here we are interested only in the relative ordering of inputs and not on the exact times of firing.

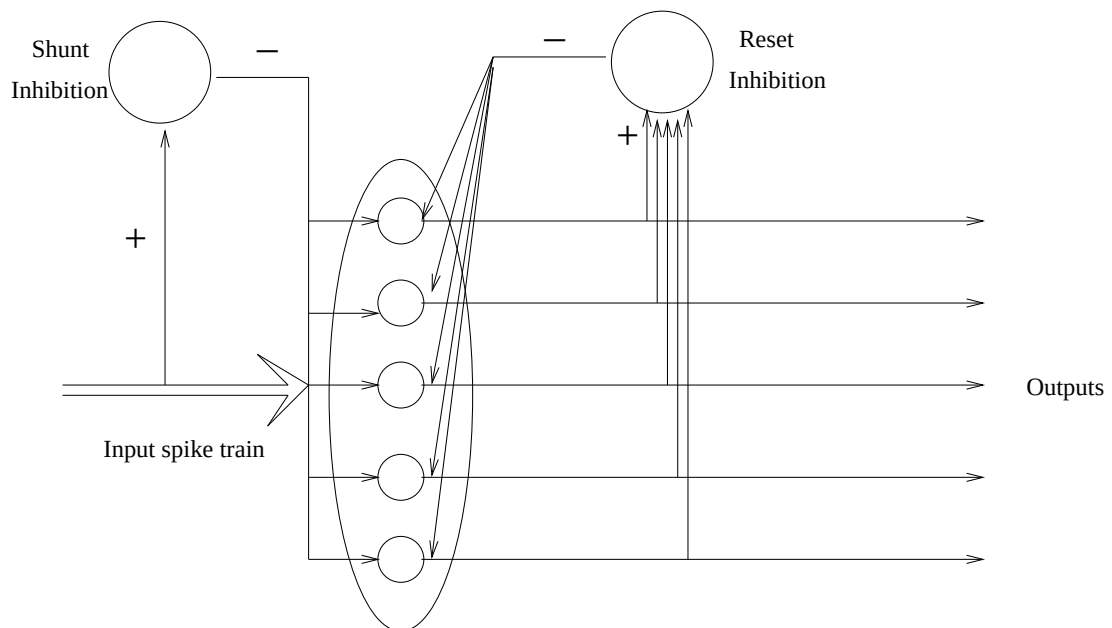
In using rank ordered codes, we want to make sure that the neurons with a higher activation fire earlier. We would also want to give more weightage to spikes which come earlier than those which come later. This can be implemented by using feed forward inhibitory neurons, as shown in the figure below.

For each successive input to any neuron in the fascicle, the feed forward inhibitory neuron also gets a spike. It sends an inhibitory spike to each neuron in the fascicle, which reduces the effect of the spike by some factor. Thus the neuron having the first spike in the fascicle gets the most increase in activation, and successive spikes decrease the ability of the neurons to respond. That is, it desensitizes the fascicle.



Imposing input ordering

Figure 7.2: Using feed-forward inhibition to impose input ordering



A fascicle to implement ordered N of M code

Figure 7.3: A fascicle to implement N of M code with rank order

The resulting structure of the neural fascicle using spiking neurons, which can produce an ordered N of M code, is shown in the figure above.

7.7 Using an abstraction of the pulsed model: sensitisation factor

The behaviour of the above neural fascicle to implement an ordered N of M code can be abstracted out. This is done by simply decreasing the increase in activation of each successive input spike to a neuron. The neuron gets progressively desensitised to input spikes. This can model rank order because this implies that the neurons firing earlier have more weightage in their spikes. Since the weights are modelled in the software model as input weights and not as output weights, it is logical to reduce the sensitivity of the incoming pulses to a neuron, with time. The exact amount of sensitisation per successive spike can be either constant, or it can be a decreasing factor, which decreases by a constant ratio. The second option will be used to abstract the effect of the feed forward (shunt) inhibition in rank ordering.

The model can be abstracted to simulate ordering of the inputs to a neuron. To simulate the decreasing of the activation increase of a neuron with each successive input spike, we assume that the activations decrease by a constant factor, which is less than 1. Let us call this constant the sensitisation factor of the fascicle, which is assumed same for the whole fascicle. So with each successive input spike, we decrease the increase in activation by a factor equal to the sensitisation factor. This in effect is equivalent to multiplying successive inputs with $1, \alpha, \alpha^2, \dots, \alpha^{(n-1)}$ where α is the sensitisation factor. Thus time can be abstracted out in this way, making the computations much faster. In effect the relative ordering is still maintained, in the same way as if real time was used.

7.7.1 Modified memory model

The N of M Kanerva memory model described in the previous chapter is modified by the method described above, to be sensitive to rank order. The architecture of the memory remains the same, with two layers, the address decoder and the data memory. The weights of the address decoder are given an arbitrarily ordered N of M code. The ordering of the weights is done by desensitising the input weights of the address decoder by the sensitisation factor. In the data memory, while writing to the memory, the

weights are set to the maximum of the previous weight and the product of the sensitisation factors of the corresponding components of the address decoder firing vector and the data input vector. The max function is used, which has the same effect as the OR function used in setting the weights in the unordered case, as described in the previous chapter.

7.8 An analysis of the rank ordered N of M coded memory model

The properties of the rank ordered N of M memory model have been studied experimentally as before. A generalised tool to simulate the memory by varying the various parameters has been built, and this is primarily used to run the experimental trials with different parameters. Figure 7.4 shows a screenshot of the tool used.

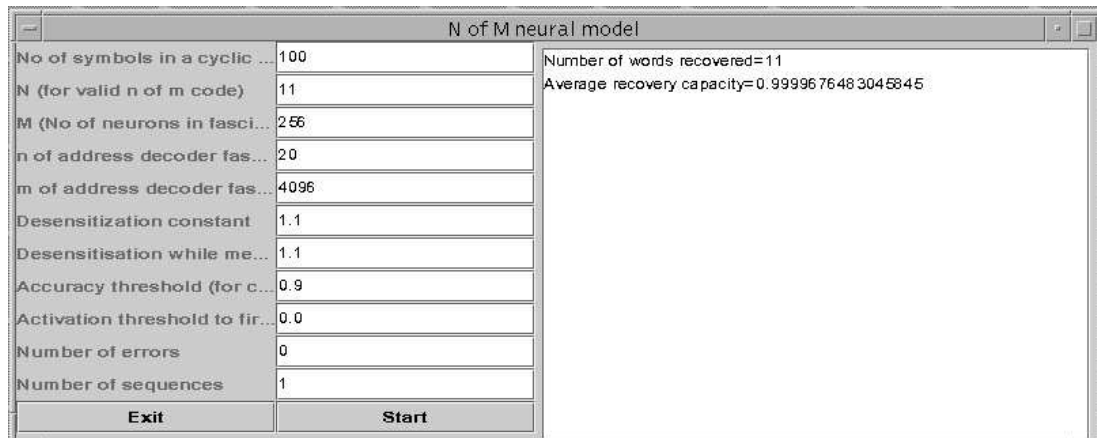


Figure 7.4: A Screenshot of the memory simulator

Most of the properties of the rank ordered N of M memory are similar to the unordered memory as shown in the last chapter. Properties such as critical distance and convergence to the stored pattern upon repeated iterations, hold good for the ordered model as in the unordered case.

7.8.1 Memory capacity

The capacity of the memory is defined, as in the unordered case, as the number of stored associations that can be correctly retrieved by the memory. So as previously done, a number of associations are stored in the memory and the number of associations correctly retrieved, on giving half of each association as input, is totalled. Here too the definition of 'correct' needs to be clarified. We could go for an exact match, which means that for an association to be decided as correctly recalled, the order of each component in the output code must be similar to the previously stored code. However, such a definition would be unreasonable in the memory, since the order itself has been approximated by the sensitisation factor.

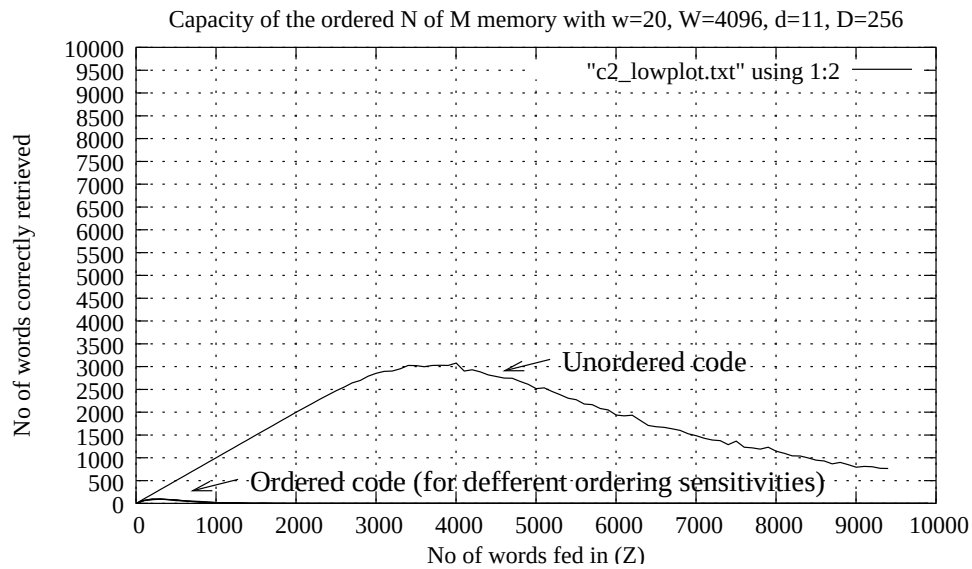


Figure 7.5: Capacity of the rank ordered N of M memory for perfect match

Figure 7.5 gives a plot of the memory capacity with different values of the sensitisation factor, when an exact match of each component is required to be correct. We see that the case when the sensitisation factor is 1.0 (which is the same as an unordered code) performs best and far better than all the ordered codes, which perform equally poorly. So insisting on exact matches is not advised when ordered codes are used.

Another option is to have a softer limit on the match in order to judge a recovered codeword as correct. The idea is to take the dot product of the sentisation factors (the distance as defined in an earlier section) of the two

vectors and divide it by the self dot product of the vector to be matched, and putting a threshold on this. For example, if the vectors to be matched are A and B, this means calculating $A.B/A.A$, and if this ratio is greater than some threshold, the words are deemed to have matched.

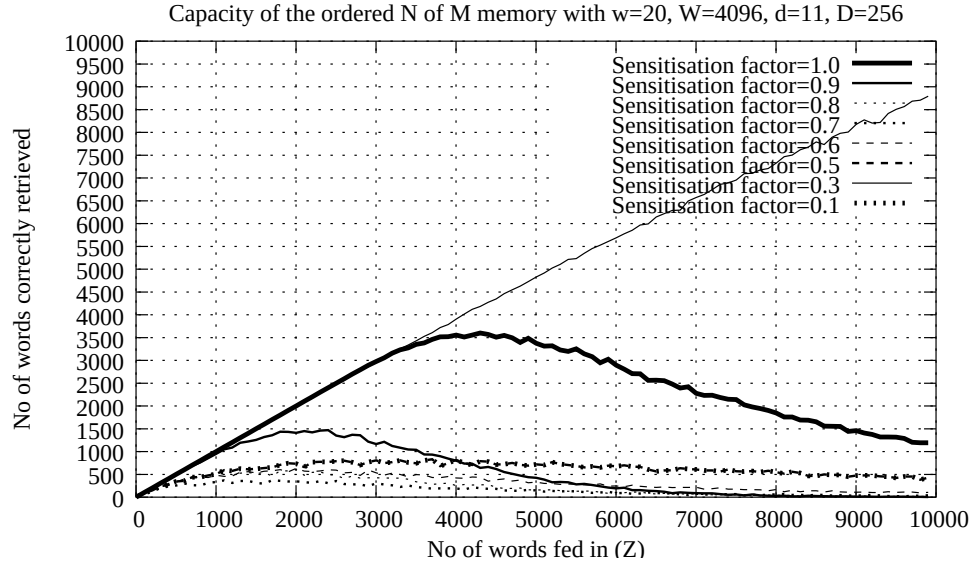


Figure 7.6: Capacity of the rank ordered N of M memory for matching threshold of 0.9

Figure 7.6 shows a plot of the memory capacity when the threshold of 0.9 is used on the ratio of the dot products. Here we get a more appreciable capacity of the ordered codes for different values of the sensitisation factors used, although it is much less than the unordered case, where the sensitisation factor is 1.0. A higher value of the sensitisation factor implies loss of ordering, and a value of 1 implies no order.

Figure 7.7 shows the same information in a condensed form. We see that for very low sensitivity values (like 0.1 or 0.3) the memory gives nearly 100 retrieval, upto a threshold of 0.9.

When the sensitisation is 0.9, the memory gives perfect recall for some words, then its performance degrades rapidly. As the sensitisation factor is decreased, the memory performance becomes worse. However, we see that the curve again rises towards a sensitivity of 0.5 and lower. Having low sensitivity implies that in the N of M code, only the first few bits need to be correct, as the system is progressively desensitised towards the later bits. As the sensitivity factor of the codes becomes lower, the importance

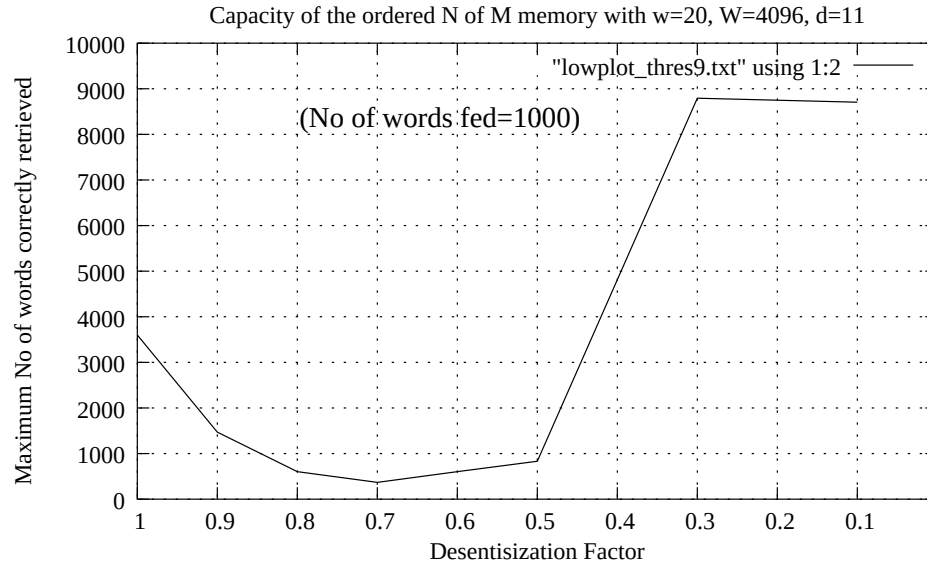


Figure 7.7: Memory capacity for ordered N of M memory with sensitivity for matching threshold of 0.9

of the first few bits in deciding the code becomes higher, and so the overall performance too increases. Fewer bits need to match for the word to be judged as recalled correctly, as long as they are towards the start.

For example, if we have a sensitivity of 0.2 and a matching threshold of 0.9, effectively only the first bit matters in an 11 of 256 code, since the relative weightage of the first bit would be $1/(1 + 0.2 + 0.04 + 0.008 + \dots)$, which is about 0.83. So if we using a threshold of 0.8 or 0.9, only the first bit needs to be correct in order for the system to classify it as a perfect match, even if most of the other bits are wrong. So it is effectively using a 1 of 256 code, due to which the capacity is much higher than at a higher value of the sensitivity factor.

From the graph, we learn that ordered codes can in general recall fewer words than unordered codes do. However, as we have seen in an earlier section, the information content of each word is higher in ordered codes than in unordered. So the information density would not compare unfavourably.

The dependence of the capacity on threshold used is illustrated by the following graphs:

Figure 7.8 shows the dependence of an ordered code with sensitivity factor=0.5, on the threshold on the dot product used to decide if two patterns

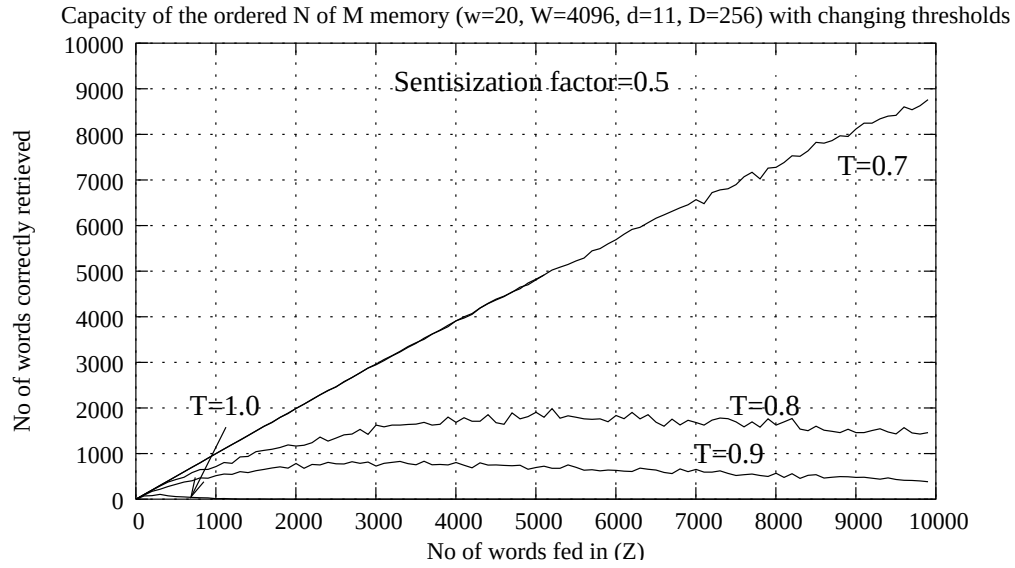


Figure 7.8: Capacity of the rank ordered N of M memory with sensitivity=0.5

are closed enough.

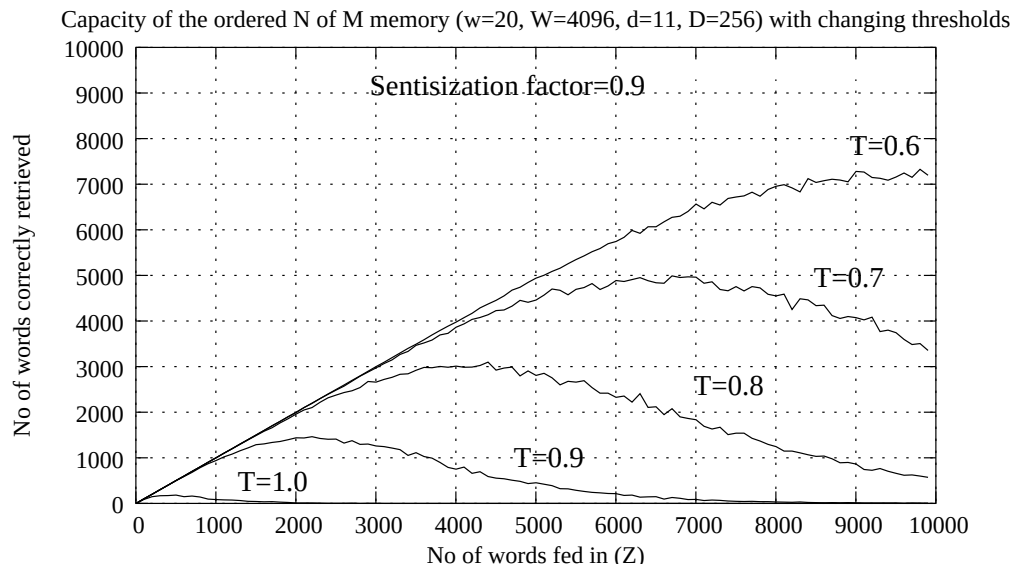


Figure 7.9: Capacity of the rank ordered N of M memory with sensitivity=0.9

Figure 7.9 shows the results of the same experiment for a sensitivity of 0.9.

Figure 7.10 combines the results of the earlier two plots and shows them in a single plot illustrating the effect of the threshold on the maximum number of words retrieved. It can be seen that unordered codes have less effect of threshold on them, than do ordered.

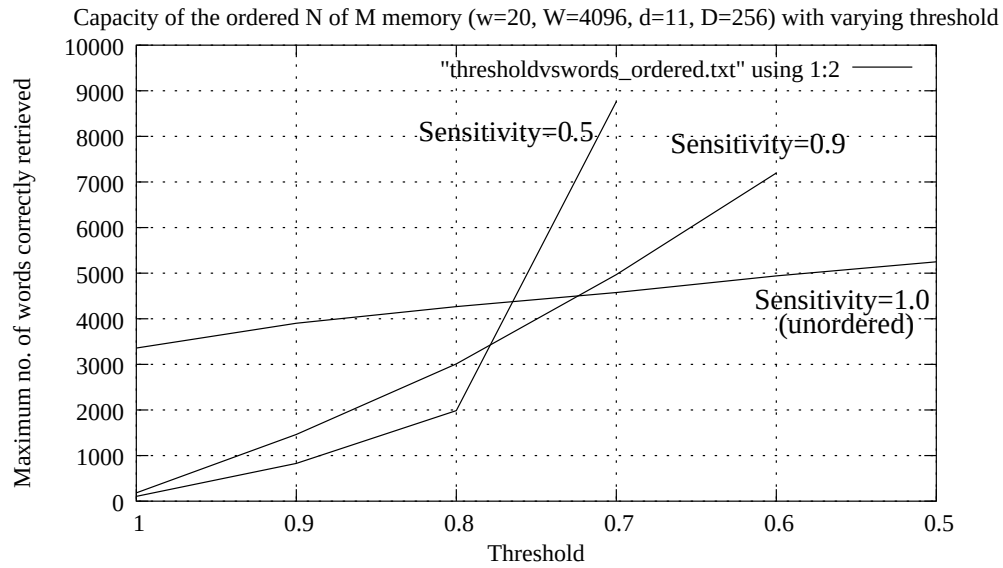


Figure 7.10: Capacity of the rank ordered N of M memory with sensitivity=0.9

7.8.2 Recovery capacity

Another way to measure the performance of the memory is by the average error, or recovery capacity. Figure 7.11 shows the performance of the mem-

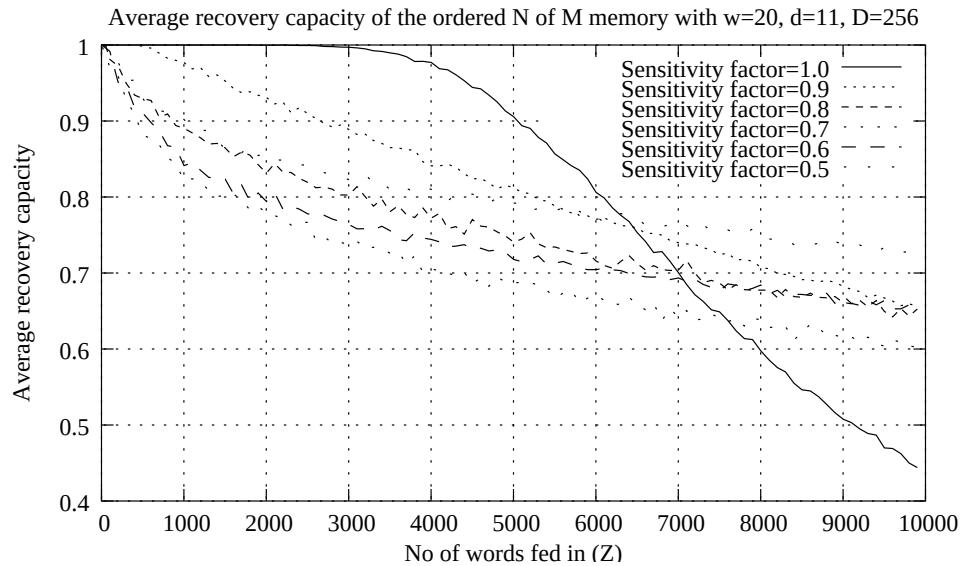


Figure 7.11: Average recovery capacity of the rank ordered memory

ory for different sensitisation factors as the recovery capacity is measured. This graphs has a much more encouraging result than the earlier graphs, which showed that ordered codes perform badly in general as compared

to unordered codes. Here we see that in the case of unordered N of M codes, we do get perfect recall (average recovery=1.0) for more words fed in. however, the average recovery also degrades much faster than the ordered codes. This proves that the performance degradation is slower when we use ordered codes as compared to unordered.

As the average recovery capacity can be claimed to be a better measure of the memory performance than the total number of words recovered, as it does not depend on the threshold used, this result is much more encouraging for the use of ordered codes.

7.9 Sequence Recognition

The ordered N of M model can recognise sequences in the same way as the unordered model. The only requirement is the presence of feedback. Any number of sequences can be stored by storing pairs of associations, upto the memory capacity. Storing a sequence is just like storing a number of associations, and so the total capacity of the memory, with respect to the number of associations that can be recalled correctly, remains unchanged. The whole stored sequence can be recalled by inputting any pattern from the middle of the sequence. This model also has the same functionality when it comes to learning sequences, as the unordered memory described in the previous chapter. So it can also behave like a deterministic finite state machine with limited functionality, as the state is not explicitly stored.

7.10 Conclusions

In this chapter we considered implementing rank ordered N of M codes in Kanerva's sparse distributed memory model. The memory using rank ordered codes was found to be less efficient as the unordered one, although additional information was conveyed per word. The average recovery capacity of the ordered memory was found to have a less degradation than the unordered memory, as more words were fed into it. Using rank ordered codes made the resulting model biologically more plausible. In the following chapter we shall consider applications of the model in learning of sequences.

Chapter 8

Learning Sequences

8.1 Introduction

In this chapter, the capability of the memory model with respect to the storage and recall of sequences of patterns is studied in detail. A generalised sequence machine is designed for context based sequential learning. The capability of the machine to recall sequences from different positions within the sequence is tested. The recall of notes of a tune as a specific application to the sequence machine is discussed.

8.2 A neural finite state machine

A finite state machine (FSM) is, by definition, an automaton capable of having a finite number of states. It transits between these states depending on the input it receives and also its present state. The state of the machine at any point of time is like an internal representation or encoded form of the past history of the machine. The machine changes state and generates outputs according to the particular input it receives and the present state it is in. The outputs are, in turn, fed back along with the next inputs. The next output, as well as the next state, of the machine is some function of the present state and the current input.

A finite state machine can learn to recognise sequences of symbols belonging to a language. There are two types of finite state machines: deterministic

and non-deterministic. Deterministic FSM's have a unique successor state for every state on giving a particular input, while a non-deterministic FSM can have more than one successor states for a given state and input. An FSM can generate valid strings of a language. It can also recognise if a string of symbols is valid in the grammar it has been trained.

To model a neural network as a finite state machine, we need to define a state, and have a way to store the state in the network. In the 6th chapter, the state was defined as the current output. This effectively meant that there was no state memory, as the next output of the network depended only on the previous output and current input. This restriction limited the capacity of the memory to recognise sequences like ABBBC, as once the network learnt to associate B with B, it went into an infinite loop.

The present model has the capability of a deterministic finite automaton only, as every state must be trained to have a unique successor.

8.2.1 The need for storing state

The proposed sequence machine should be able to recognise any finite sequence of patterns, like ABCD, ABBC, etc. as well as recurring loops, such as (CD)*. Two or more different sequences can have patterns in common, such as ABCDE and MNCOP, which have the pattern C in common. The model described in the previous chapters had the drawback of every stored pattern needing to have a unique successor in the sequence.

It is desirable for a memory with good error tolerance to be able recognise even noisy patterns, at least those with the amount of noise below some threshold. In designing this type of memory, it is necessary to incorporate some kind of state information in the memory. This is because two or more sequences of patterns can have one or more patterns in common, rather than each pattern having a unique successor. On being presented with a pattern, the memory should know which pattern sequence it belongs to, before it can generate the next output pattern. Thus we need it to hold state information, since the state is like an encoded history of the sequence. Having a state memory would eliminate the unique successor restriction that was applicable for the memory described in the previous chapter. However,

the successor state of every state still has to be unique, so the memory can at best implement only a fully functional deterministic finite state machine.

8.3 Motivation: The importance of time

Time is an essential factor in learning and cognition. It is an ordered entity which is central to many of the cognitive behaviour which form the main application space of neural networks, such as language, speech, signal processing, etc. Thus it is essential to have some representation of time in the operation of the neural network.

A sequence of patterns is, in fact, a time ordered collection of patterns.

8.4 Previous approaches to building time into the operation of a neural network

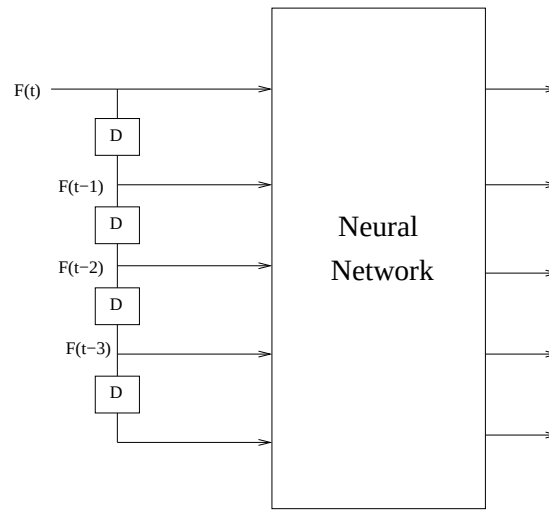
Time can be built into the operation of a neural network either implicitly, by convolving the weights with samples of the inputs, or explicitly, by giving it a representation, like explicit neurons which represent and respond to different delays. A network may have short term memory, which is the record of its outputs in definite intervals of time in the immediate past. Short term memory can make the network dynamic.

Time may be discrete or continuous and the models may be modified accordingly. In the following subsections, an overview of various neural network models to represent time is given. All the neural algorithms and architectures can be implemented with any of these approaches.

8.4.1 Time delay neural networks

These are multilayer feedforward networks whose hidden and output layer neurons are replicated across time. The input is broken up into time slices. Multiple connections between input and hidden layer represent input delayed by 1,2,3,... time units. It goes the same way between the hidden and output layer.

8.4.2 Giving spatial representation to time



Representing time explicitly using Delay lines

Figure 8.1: Using delay lines

One way to incorporate time in the operation of a neural network is by giving it a spatial representation, by using delay lines (as shown in figure 8.1) to delay the input by different time units[19], which are fed to the hidden layer. Thus the hidden layer neuron's activation is a function of the input at time t $i(t)$, $i(t-1)$, $i(t-2)$ and so on.

8.4.3 Having feedback

Another way of building time into the operation of a network is through feedback. The output of the neurons can be fed back into the inputs after some delay. A system with feedback loops is called a recurrent system.

8.4.4 State

In trying to understand the temporal dynamics of a system, we introduce the concept of the state of a system at some instant of time. State too may be represented implicitly or explicitly. The state of a system summarises information about its past behaviour.

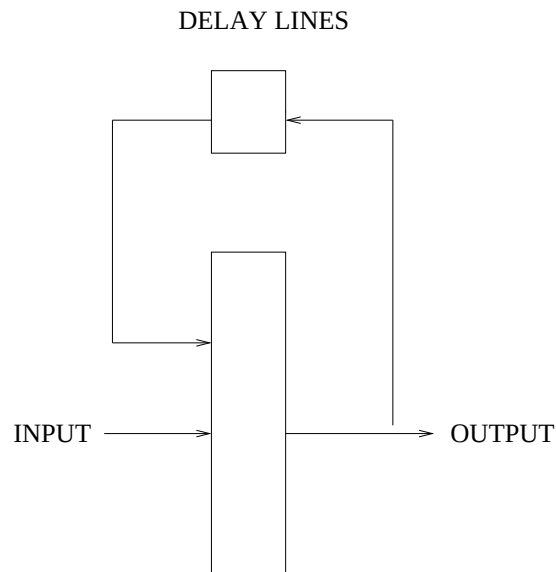


Figure 8.2: Using feedback with delay

8.4.5 Elman and Jordan Networks

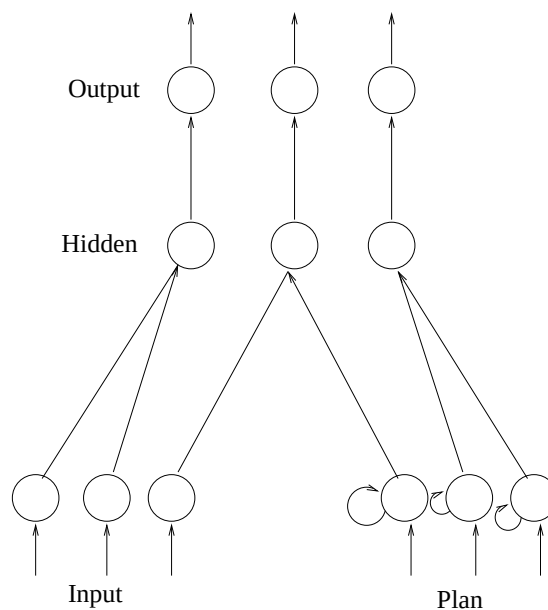


Figure 8.3: The Jordan network

Jordan and Elman proposed models for remembering incorporating time into neural networks.

Jordan[25] described a network having feedback connections which were used to associate a static pattern (plan) to an output. Jordan's network

model is shown in figure 8.3. The feedback connections allowed a network's hidden units to see their own previous output.

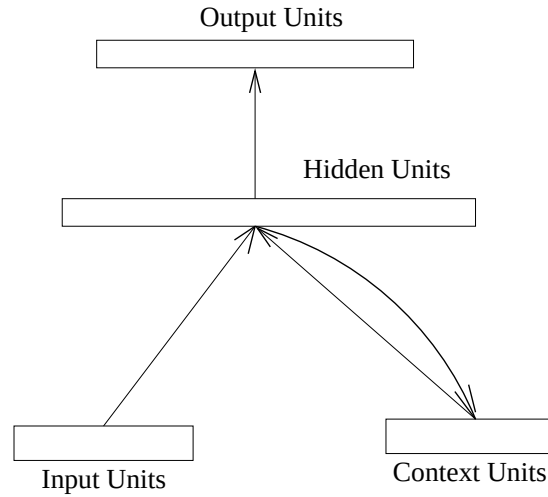


Figure 8.4: An Elman network with context layer

Elman[14] modified Jordan's model and proposed the network to be augmented at the input level by additional units, which were also hidden in the sense that they were not output. He called these units as the context units, as they stored the context of the patterns in the sequence. The hidden units received input from these context units as well as the input layer, and their output was fed back to the context units as well as the output layer. Thus these context units stored an encoded form of the past history of the sequence.

The main difference between the models proposed by Elman and Jordan was that in the Jordan network, the feedback was only from output nodes to input nodes, while in the Elman network it was from all hidden nodes and in the expanded Elman network, from the output nodes too.

8.4.6 Time series prediction

Time series prediction is the branch of neural networks concerned with prediction of the next element in a time series, where time is in discrete steps. It has got a number of applications like prediction of stock market trends, weather, etc. Traditional time series prediction is done using Markov models. Using neural networks like backpropagation in time series prediction

can lead to long training times.

The present N of M sparse distributed memory model has the properties of rough interpolation but not extrapolation. This means that it can get the point in the memory space that is closest to the actual stored value (hard location), but it cannot recognise what it has not already seen, especially since all the vectors are orthogonal. Hence the Kanerva model is not appropriate for time series prediction, where extrapolation is needed to give an estimate of the next output value given the present sequence. Only gradient based or such other neural models are suitable for this purpose.

8.5 Sequence Learning

A sequence of characters, notes in a tune, numbers or any types of patterns is like a temporal input. Hence the problem of learning of sequences is identical with that of learning of temporal patterns.

Sequence learning can either involve sequence recognition or sequence prediction. Sequence recognition involves reproducing a previous learnt sequence on being given part of the sequence as input. Sequence prediction means to predict the rest of the sequence given only a part of the sequence. As argued in the previous section, the model used is only good for sequence recognition rather than prediction.

In chapters 5,6 and 7, the neural models had been examined with respect to their ability to recognise sequences.

8.6 Learning and retrieval of music tunes

Use of neural networks in music constitutes an interesting application for sequence recognition.

We are interested in the learning, recognition and completion of a tune, rather than the prediction of the next note (which has not been learnt before) on feeding part of whole of the song. The aim is to be able to remember the whole tune, so that the whole tune can be recovered by giving the first few notes of the tune. It seems so far not much research has

been done to apply autonomous neural networks in learning of notes of a tune, which is another kind of temporal pattern.

It should be remembered that in a tune, since the number of notes are limited and often repeated, similar notes occur often and it is not necessary that there be a unique successor note for each one. Music tunes are quite often irregular, so they may not follow a set pattern either. Hence sequences such as ABBD or sequences with common patterns are to be expected. Hence the conventional associative memory won't work in this case. We need to store the context, or state, of the note in the song. Elman's memory model might be a possible way to do the same.

8.6.1 Kanerva's approach to retrieval and storage of sequences

Kanerva's original memory model was equivalent to a simple associative memory. It could successfully retrieve a sequence only if each pattern in that sequence had a unique successor. If that was not the case, the model failed to recover the next pattern in the sequence correctly. To solve this problem, Kanerva proposed storing transitions in memory. This implied storing the word $E(t)$ at the address $E(t-1)$. This supports the prediction that if $E(t-1)$ has just happened, $E(t)$ can be expected to happen next. Thus a sequence could be stored in the memory. Convergence to a stored sequence was possible as long as the starting pattern was within the critical distance of any word in the sequence. For prediction of the next pattern in the sequence in the higher order case, as when the next data could not be predicted on the basis of the current data alone, Kanerva's[26] solution was to store transitions of higher orders. This implied, for example, storing the past two outputs along with the present input in order to predict the next output.

For example, to learn the sequence ABCDE, the network would actually learn the following associations by using a time window of 2 units:

AB->C

BC->D

CD->E

To store MNCOP, the corresponding associations learnt would be like $MN- > C$, $NC- > O$ and $NO- > P$. So deciding a successor for C, with these two sequences, would no longer be a problem.

However, this model also has drawbacks, It is possible that the two sequences have the same number of consecutive patterns in common as the length of the time window, such as ABCDE and MNCDO. The successor to CD would then be a problem, since the memory reads both $CD- > E$ and $CD- > O$. However, if this method is recursively followed till all unique successors have been determined, such a problem might be solved. In this case, it would be like having 3 as the length of the time window and repeating the process: like $ABC- > D$ and $MNC- > D$ which are perfectly valid to be written and not interfere.

However, such repeated readings and writings for only one sequence can be time consuming. Elman's method of using the context layer is preferable to this, as it is simpler and perhaps more efficient.

In the following section, Elman's context layer has been used to modify the original neural memory layer to enable it to recognise sequences.

8.7 Structure and role of the context layer

There are a few alternatives to set the weights for the context layer. One way is to fix the weights randomly, so that the outputs of the neurons form the context. Another way is to make the context layer learn the associations, either by normal Hebbian correlation matrix learning or alternatively, by using RAM based or weightless nodes or counters, as in Kanerva's model.

To store the sequence ABCDE in a model with context, the associations learnt are as follows:

A->B

AB->C

ABC->D

ABCD->E

The function of the context layer is to ensure that the next output depends on the previous history of the sequence as well as the present input.

8.8 A modified rank ordered N of M model to learn sequences

The ordered N of M model is modified to store sequences by using Elman's model. A second, context layer is introduced along with the input layer. This layer takes its input from the previous hidden (address decoder) layer's output. The output from the context layer feed into the address decoder neurons. The output (data memory) layer is not effected.

The reason why this way is better than feeding the context layer from the output layer as in Jordan network is that the hidden layer encodes all the past history while the output layer encodes only the most recent history.

However, instead of having a separate context layer, we can have additional neurons along with the output layer to store the context. Those additional neurons would not be seen at the output, but would feed their outputs to the address decoder layer. Such a system would be equivalent to having a separate context layer.

Thus the context layer stores an encoded form of the past history of the network. It starts from 0, and gradually fills up as the sequence gets bigger. However, problems occur when a second sequence has to be stored. Some consistent mechanism has to be explored for this case, as also the case when a sequence has to be restored from the middle rather than from the beginning.

8.9 The context weight factor

It is possible to dynamically control the influence of the context on the output. This can be done by introducing a parameter called context weight factor. The activation increase in the address decoder layer as a result of context layer firing is governed by this factor. So it can be modulated externally or internally. When the factor is set to 0, the model resembles the old (without context) memory. As it is increased, the weightage given to history keeps increasing proportionately.

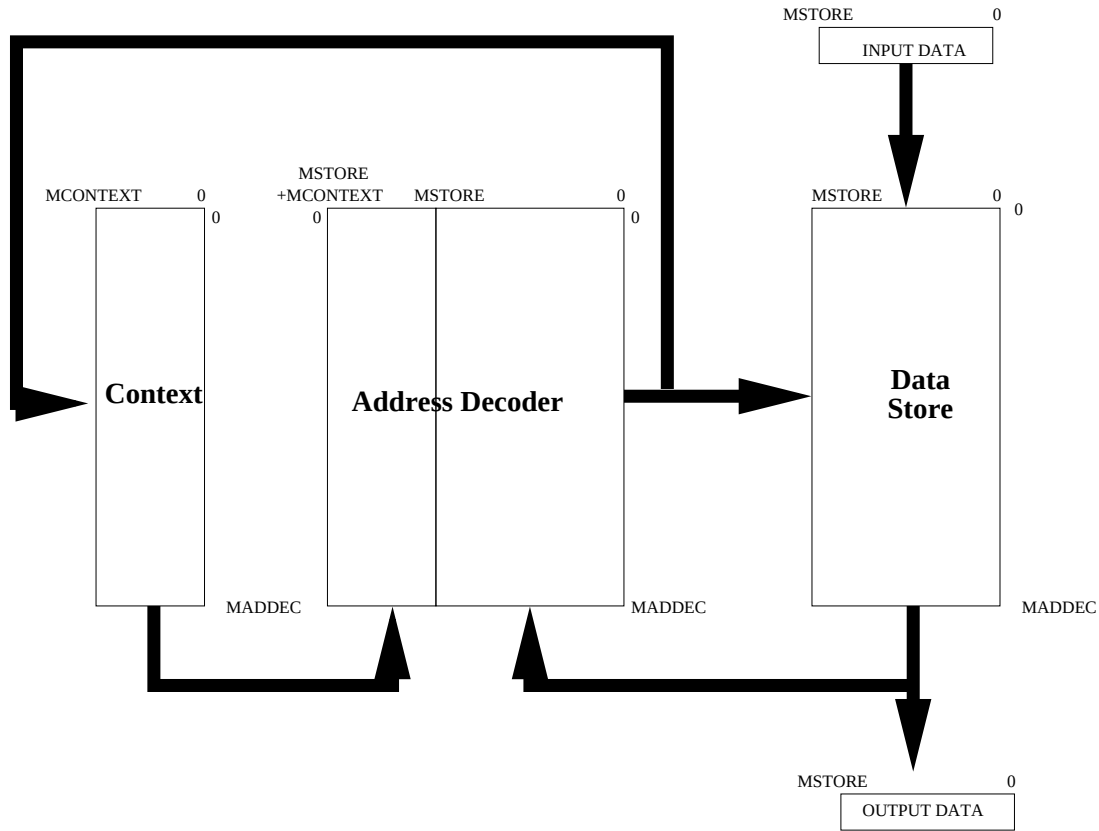


Figure 8.5: The modified N of M Kanerva memory with context

8.10 The sequence machine

The sequence machine consists of a letter to N of M code encoder, the neural memory and an N of M code to letter decoder. The letter sequence is fed first through the encoder, where each letter is converted to the N of M code before writing to or reading from the memory. The final output is then decoded to give back the letter outputs, if any.

The N of M encoder and decoder can be implemented in spiking neurons by using a 1 of 26 code fascicle (for the letters of the alphabet) and an N of M fascicle. The connection weights can be randomly set.

The structure of the sequence machine is shown in Figure 8.6. The previous rank-order recognition memory model is modified to include feedback. The outputs are fed back to form the next addresses. So there is just one input in the modified network: the data input. The data output feeds back to form the next address input.

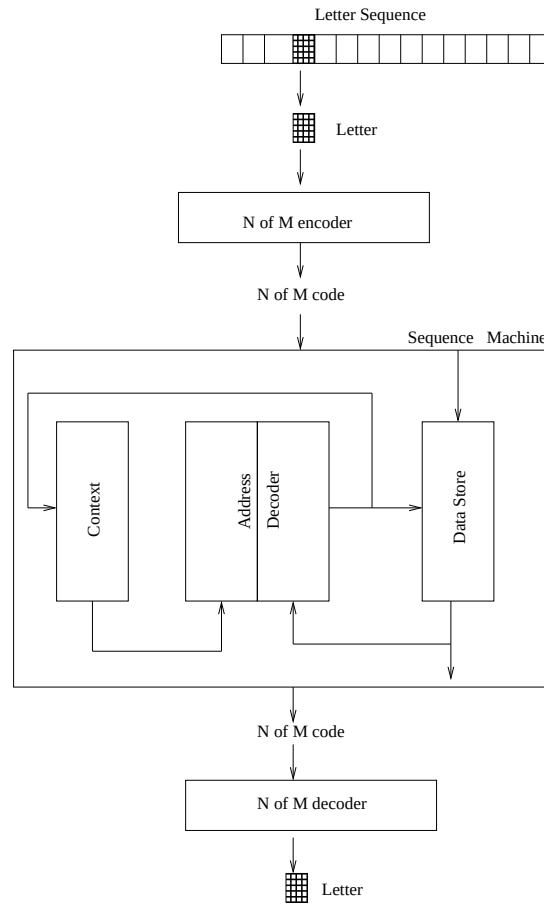


Figure 8.6: Structure of the sequence machine

There is an address decoder layer, as in the previous model. It receives input from the data output neurons, which are fed back to it. There is a new layer, called context layer. The neurons in this layer have fixed weights. The address decoder outputs feed into this fascicle. The context outputs, in turn, are inputs to the address decoder layer, along with the fed back data output neurons.

The way it works is this: every new letter is at first encoded by a separate fixed weight N of M encoding layer. Then the encoded form is fed into the data memory layer, along with any output from the address decoder the N of M output is passed in reverse through the same encoding mechanism to decode it to see if it matches any letter. However, this should actually be any letter which it has previously seen. That letter is then output and the output is then fed back and the context and the address decoder layers are updated.

8.11 Online and offline learning

Learning in the sequence machine can take place in two ways: online or offline.

8.11.1 Offline learning

Offline learning is the more common mode of learning in neural networks. In offline learning, the writing(learning) and reading(recall) modes are separated. In the writing mode, the input sequence of letters is first presented to the memory and learnt. Only when the writing of each association in the sequence has completed, the mode may be changed.

In the reading mode, any of the input patterns previously written is presented to the memory, which then reads the rest of the sequence from the memory.

All the input sequences are learnt in this way.

For example, in writing (learning) the sequence 'ABC' to the memory, the network proceeds in the following way:

<i>Input</i>	<i>Address decoder output(prev.o/p,context)</i>	<i>Data memory output(input,addec o/p)</i>	<i>Context layer output(addec)</i>	<i>Assoc. written</i>
A	$ad(0, 0)$	$dm(A, 0) = A$	$c(0) = 0$	None
B	$ad(A, 0) = A'$	$dm(B, A') = B$	$c(A') = A''$	$A' - > B$
C	$ad(B, A'') = A'''B'$	$dm(C, A'''B') = C$	$c(A'''B')$	$A'''B' - > C$

Table 8.1: Offline writing of sequence ABC

Now in reading (recalling) the sequence from the memory on giving just A at the input, we have

<i>Input</i>	<i>Address decoder output(prev.o/p,context)</i>	<i>Data memory output(input,addec o/p)</i>	<i>Context layer output(addec)</i>	<i>Assoc. read</i>
A	$ad(0, 0) = 0$	$dm(A, 0) = A$	$c(0) = 0$	None
0	$ad(A, 0) = A'$	$dm(0, A') = B$	$c(A') = A''$	$A' - > B$
0	$ad(B, A'') = A'''B'$	$dm(0, A'''B') = C$	$c(A'''B')$	$A'''B' - > C$

Table 8.2: Offline reading out of the sequence ABC on giving A as input

Thus we see that in the offline case, any stored sequence can be recalled in the same way.

Then the letters at the beginning or middle of the sequence to be recalled are presented, and the memory is expected to be able to complete the sequence from the letters already presented. Thus it performs the tasks of sequence recognition and completion.

The memory will have no problem in storing sequences not having unique successors for each pattern, such as ABBC. Because of the context layer, the memory would be able to remember the context of each character, so the next character read from the memory would depend on the present character as well as the machine state (or context).

8.11.2 Online learning

Online learning means that the network learns as the input sequence is presented. There are no separate modes for reading and writing to the memory. If the network has seen something it learnt previously, it outputs the same immediately. Before the next character in the sequence is input, the network goes in a loop to output any letters in the sequence it saw previously.

This loop terminates if the output becomes 0 at any point. If the loop terminates, the address decoder and the context outputs become frozen.

For example, in storing a sequence ABC, the memory proceeds as follows:

<i>I/P</i>	<i>Address decoder output(prev.o/p,context)</i>	<i>Data memory output(i/p,addec o/p)</i>	<i>Context layer output(addec)</i>	<i>Assoc w/r</i>
A	$ad(0,0) = 0$	$dm(A, 0) = A$	$c(0)=0$	None
0	$ad(A,0) = A'$	$dm(0,A') = 0$	$c(A')=A''$	None
B	A'	$dm(B, A') = B$	A''	$A' - > B$ w
0	$ad(B,A'')=A'''B'$	$dm(0,A'''B')=A''''B''$	$c(A'''B')=A'''''B'''$	None
C	$A'''B'$	$dm(C, A'''B')=C$	$A'''''B''''$	$A'''B' - > C$ w
0	$ad(C,A'''''B''''')=A''''''B'''''$	$dm(0,A''''''B''''')=0$	$c(A''''''B''''')$	None

Table 8.3: Online reading/writing of the sequence ABC

8.12 Analysis of the sequence machine

The sequence machine is meant for storing any sequence of patterns. The aim is to optimise its capacity, to enable it to store and reliably retrieve the maximum number or the longest length of sequences, and its functionality, so as to enable it to store as many types of sequences as possible (such as ABBC, XYCDE etc.), as well as converging and diverging sequences.

The modulation of the effect of context by means of the context sensitivity factor presents a way to study the effect of the past history on retrieval efficiency. When the context sensitivity factor is 0, the model is identical to the no-context model considered in previous chapters. Hence the functionality should be the same. As the sensitivity is increased, the encoded form of the history become more and more important. For large values of the sensitivity, the output predicted by the machine disregards the current input completely.

Where the context sensitivity is not 0, the address decoder now produces an encoded combination of the past history and the present output feedback. The prediction of the next letter output by the context machine depends on two factors: the history prediction and the present output prediction. The final prediction is a combination of these two predictions, with the relative weightage of the predictions modulated by the context factor. The combination used here is simply summing of the activations of the neurons. In an N of M coded world, the summing of two point vectors does not produce a point which is the vector sum of the two, but rather it produces an entirely new point in the space, which can be orthogonal to both of those points. This implies that the modulation of the context factor can have different and unpredictable results. Using a binary code for the vectors, however, does not face this problem as a summing of the vectors indeed produces a point which is the vector sum of those two, and the modulation directly effects the distance between them.

The N of M model is basically similar to Kanerva's model, which had been described in Chapter 5. Kanerva's model had the property of an access circle for every hard location, within which vectors in the space converged to the stored data, and without which they diverged. This property also presents problems in sequence recognition, since there is no way to bring

a vector which is outside the access circle to the inside. A proper coding scheme and definition of the distance metric would solve this problem.

The threshold used to identify the patterns in the code to letter decoder is also an important parameter, as the danger of getting false identification of letters exists. The threshold has to be chosen in such a way that a letter is not wrongly identified when the code is not similar to any of the stored letters, nor is a sufficiently similar letter not identified.

8.12.1 Testing

The above model was tested using simple characters as patterns. The following sequences of patterns (as letters of the alphabet) were written to the model:

"abcde"

"rsctu"

These sequences were chosen due to the ambiguity in choosing the successor of "c": in the first sequence it should be "d", while in the second, it should be "t".

In order to test retrieval, the first few characters of the non-overlapping input patterns are put into the memory. In this case, 'r' was input to the memory and the system was expected to be able to retrieve the whole sequence "rsctu".

The outputs of the simulation for a few trial runs with different context factors and sensitivity are given below. The input parameters were the sensitivity and the context factor. For example, "java offline 1 0" means that the context factor is 1 and the sensitivity is 0.

In all of the following tests, the offline learning version is used, with clear separation between the learning and recall phases.

```
amu23-bosej/single> java single_offline 1 0
Writing=a It recalls=a
Writing=b It recalls=a
Writing=c It recalls=a
```

```
Writing=d It recalls=a
Writing=e It recalls=a
```

```
Writing=r It recalls=a
Writing=s It recalls=a
Writing=c It recalls=a
Writing=t It recalls=a
Writing=u It recalls=a
```

```
Reading=a
Reading=a
Reading=a
```

```
Reading=a ... (infinite loop)
```

In this case, since sensitivity is 0, only the first input (a) matters, which it keeps recalling in an infinite loop.

```
amu23-bosej/single> java single_offline 1 1
```

```
Writing=a It recalls=a
Writing=b It recalls=b
Writing=c It recalls=c
Writing=d It recalls=d
Writing=e It recalls=e
```

```
Writing=r It recalls=r
Writing=s It recalls=s
Writing=c It recalls=c
Writing=t It recalls=t
Writing=u It recalls=u
```

```
Reading=r
Reading=s
Reading=c
Reading=t
Reading=u
```

Here we see that the recall is correct.

```
amu23-bosej/single> java single_offline 0 1
```

```
Writing=a It recalls=a
```

```
Writing=b It recalls=b
```

```
Writing=c It recalls=c
```

```
Writing=d It recalls=d
```

```
Writing=e It recalls=e
```

```
Writing=r It recalls=r
```

```
Writing=s It recalls=s
```

```
Writing=c It recalls=c
```

```
Writing=t It recalls=t
```

```
Writing=u It recalls=u
```

```
Reading=r
```

```
Reading=s
```

```
Reading=c
```

Since the context factor is 0 in this case, this is equivalent to the case with no context. Here the ambiguity between d and t stops the memory from recalling a single pattern correctly.

```
amu23-bosej/single> java single_offline 0.001 1
```

```
Writing=a It recalls=a
```

```
Writing=b It recalls=b
```

```
Writing=c It recalls=c
```

```
Writing=d It recalls=d
```

```
Writing=e It recalls=e
```

```
Writing=r It recalls=r
```

```
Writing=s It recalls=s
```

```
Writing=c It recalls=c
```

```
Writing=t It recalls=t
```

```
Writing=u It recalls=u
```

```
Reading=r
Reading=s
Reading=c
Reading=t
Reading=u
```

```
amu23-bosej/single> java single_offline 10000 1
```

```
Writing=a It recalls=a
Writing=b It recalls=b
Writing=c It recalls=c
Writing=d It recalls=d
Writing=e It recalls=e
```

```
Writing=r It recalls=r
Writing=s It recalls=s
Writing=c It recalls=c
Writing=t It recalls=t
Writing=u It recalls=u
```

```
Reading=r
Reading=s
Reading=c
Reading=t
Reading=u
```

Here we see that having a very small or big value of the context factor has no effect on the output and it still retrieves correctly.

```
amu23-bosej/single> java single_offline 0.8 0.001
```

```
Writing=a It recalls=a
Writing=b It recalls=b
Writing=c It recalls=c
Writing=d It recalls=d
Writing=e It recalls=e
```

```
Writing=r It recalls=r
```

```

Writing=s It recalls=s
Writing=c It recalls=c
Writing=t It recalls=t
Writing=u It recalls=u

```

```

Reading=r
Reading=s
Reading=c
Reading=t
Reading=u

```

This is a repeat run for an ordered code. Here too the recall is perfect.

```

amu23-bosej/single> java single_offline 0 2
Writing=a It recalls=
Writing=b It recalls=
Writing=c It recalls=
Writing=d It recalls=
Writing=e It recalls=

Writing=r It recalls=
Writing=s It recalls=
Writing=c It recalls=
Writing=t It recalls=
Writing=u It recalls=

```

Here we see that having a sensitivity of 2 recalls nothing correctly.

8.13 Preliminary results for sequence recall

It was found that it is not possible to retrieve the stored sequence except from the beginning, as the memory learns to associate the context as well as the previous output, with the next output. This is regardless of how many words had been written into the sequence, or the value of the weight factor. Offline learning invariably gave much better results than online learning.

The value of the context weight factor did not seem to matter much to the output. Though this is a not too encouraging result, especially for the music case, more experimentation and different models might yield better results.

8.14 Recalling the sequence from the middle

The present memory, it seems, can recall the whole sequence, but only if the first character of the sequence is presented. Ideally, presenting any character from the middle of the sequence should also be enough to get the next character. The problem with the present model is that since the weightage factor of the context layer is constant, the context information builds up steadily from the starting of the sequence. As a result, presenting a character from the middle of the sequence would not retrieve the rest of the sequence, as the context is missing. The problem arises as all the vectors are orthogonal to each other in the N of M coded Kanerva memory. One way to solve this problem would be to use dynamic context sensitivity factor modulation, as described in the next section.

It is possible to recall the rest of the sequence from any point because we know what should be the average activation of the same so we can just sense if the context layer is sending a strong enough and clear signal or, if not, we simply ignore the contribution from the context and take into account only the data memory contribution. However the drawback is that it cannot work if both layers predict different letters as next in the sequence. It may also be possible to modulate the weightage given to the context layer dynamically. It is also worth exploring the feasibility of the same first, by experimenting with dynamical neural systems and seeing the stability of a propagating wave front as it cycles through the feedback loop.

8.15 Having dynamic context weightage

One option to have in the index sensitivity to the context layer would be to set it dynamically. At any stage during the reading of a sequence, we have three options to choose the next output in the sequence: one is to choose

the output according to the prediction of the address decoder along and ignore the contribution from the context layer. Another way is to use only the context layer prediction and ignore the address decoder instead. The third and balanced way is to modulate the sensitivity to the context dynamically, according to which prediction gives the closest match. However, this modulation would be done only in the case where ambiguity about the next output exists.

8.16 Application areas

The sequence machine is a scalable, autonomous machine which can learn associations in one shot. A few of its possible applications are given below:

8.16.1 As an encoder

The network can be used as an encoder of an input pattern. It encodes the input pattern as an N of M code. The encoding is fixed and determined by the weight matrix, which is composed of random weights.

8.16.2 As a pattern matching machine

Suitable applications include character and image recognition, music, robotic guides, etc As

8.16.3 As a robust and error tolerant memory

Possible applications include a variety of character recognition applications. Some of the ideas in the development of the sequence machine might find use in other areas. The application space for the same has yet to be explored.

8.17 Drawbacks of the sequence machine

As shown in an earlier section, having an N of M code is not the ideal choice of coding for the machine, as a linear combination of two points in the space

does not give a point which can be predicted. Due to this, modulation of the context sensitivity can lead to unpredictable results. The need is to have a code which has the concept of distance and addition defined in a proper way. However, having an N of M code has its advantages, as described in earlier chapters.

The machine can now work as a Deterministic Finite Automaton (DFA) only, as each state must have a fixed successor. Having a nondeterministic character leads to enhanced functionality. NFA's have the capacity to recognise context free languages. A string from a language like $a(bc)^*d$, such as *abcabcd*, should be able to be recognised by an ideal sequence machine.

8.18 Conclusion

This chapter contains the main contribution of this research. It is the development of a sequence machine which is simultaneously scalable, low power and thus capable of implementation in hardware, robust, self error-checking, and capable of storing and retrieving sequences of patterns. Thus it combines the advantages of all models previously considered. It is theoretically possible to store and retrieve any sequence if the memory is big enough and if the sequence is retrieved from the beginning. Thus it can implement a generic finite state machine. More experimentation needs to be done to find if the whole sequence can be read by presenting a character from anywhere in the sequence, like the middle.

Chapter 9

Neurons as Dynamical Systems

9.1 Introduction

This chapter mainly deals with the dynamics of populations of neurons. It deals with issues related to the dynamics of propagation of a wave of spikes over a layer of spiking neurons with recurrent connections. It also discusses emergent behaviour in neurons, and modelling of such behaviour.

9.2 Motivation

The model developed in the previous chapters was based on rank ordered codes, which implied that the order of firing of the spikes conveyed the information. If the ordering gets distorted or lost, the information is also lost. It is important to ensure that the order is not distorted, when the wave of spikes goes through multiple layers of neurons. That is why it is necessary to discuss the dynamics of the same in some detail.

The abstractions used in the earlier models discussed in this research (to incorporate rank order, N of M firings, sorting the activations to get the N neurons with maximum activation) can be validated only if it is possible to get equivalent behaviour with low level spiking neural models. A consideration of the dynamics of such neurons is therefore necessary to understand their behaviour. Neurons have emergent behaviour, which is difficult to

predict unless they are modelled as populations. Hence one way to study the dynamics would be by modelling them in some artificial environment.

9.3 Related work

The dynamics of neural populations have long been a subject of interest. Milton[31] has discussed issues related with the dynamics of neural populations. Mass[42] and Gerstner[43] have analysed the properties of populations of spiking neurons. Synfire chains[1], proposed by Abeles, is a model to produce synchronised waves of firings in spiking neural populations.

Many artificial models have been built to study the dynamics of autonomous interacting neurons. A Life, proposed by Chris Langton[9], is a field of study related to modelling of populations of artificial living systems.

9.4 A feedback control system

We have explored the use of feedback in neural models in earlier chapters, especially the sequence machine. A neural network with feedback is an example of a feedback control system. These systems are found in many areas of engineering design, and even in parts of our brains. For example, the cerebellum in the brain has been claimed to be a type of feedback control system.

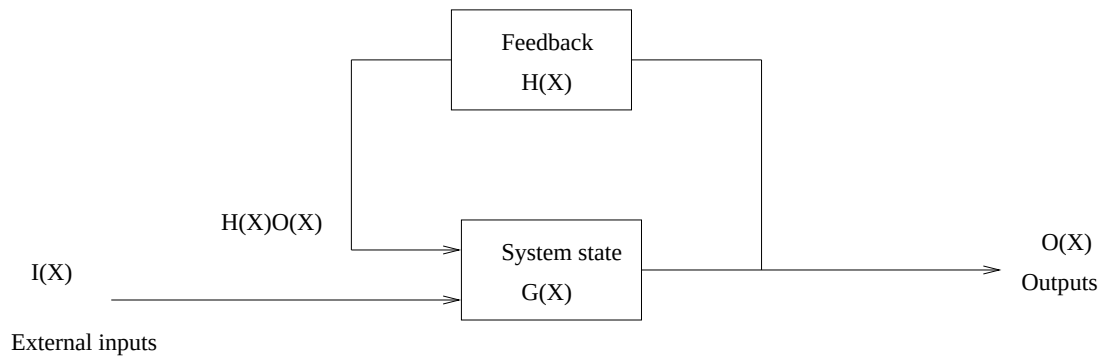


Figure 9.1: A feedback control system block diagram

As the name indicates, a feedback control system is controlled by feedback. It is a closed loop system. The feedback signal is a function of the output

signal. The system works in this way: the outputs are fed back and compared with the desired outputs to give the error signal, which is fed along with the input to the process to get the next output.

The feedback may be error correcting, adjusting the system state or parameters so that the error is not repeated. The feedback can be used to bias the system in any direction. There is generally a time delay before the present outputs can be fed back to the inputs. A finite state machine with feedback is another example of a feedback control system.

As mentioned earlier, the sequence machine described in the last chapter is a feedback control system. The output is a function of the present input and feedback from the previous output. We are interested in the dynamics of the neural feedback control system.

Traditional feedback control systems are of two types: linear and nonlinear. They are analysed by frequency and time domain methods, such as impulse response.

9.5 Propagation of spike waves around a feedback loop

In the N of M coded memory implemented through spiking neurons, information is conveyed through a wave of spike firings. The inputs and outputs of each fascicle are waves of firings. N of the total M neurons in the layer fire spikes at approximately the same time to produce a valid codeword.

9.5.1 An ordered N of M fascicle

The firing of exactly N of the M neurons and the progressive desensitisation of the neurons to successive input spikes can be implemented through feed-forward and feedback inhibition respectively, as described in earlier chapters and also shown in Figure 9.3.

In ordered N of M codes, the temporal ordering of the spikes is taken into account. In a feedback loop, these spike waves propagate again and again

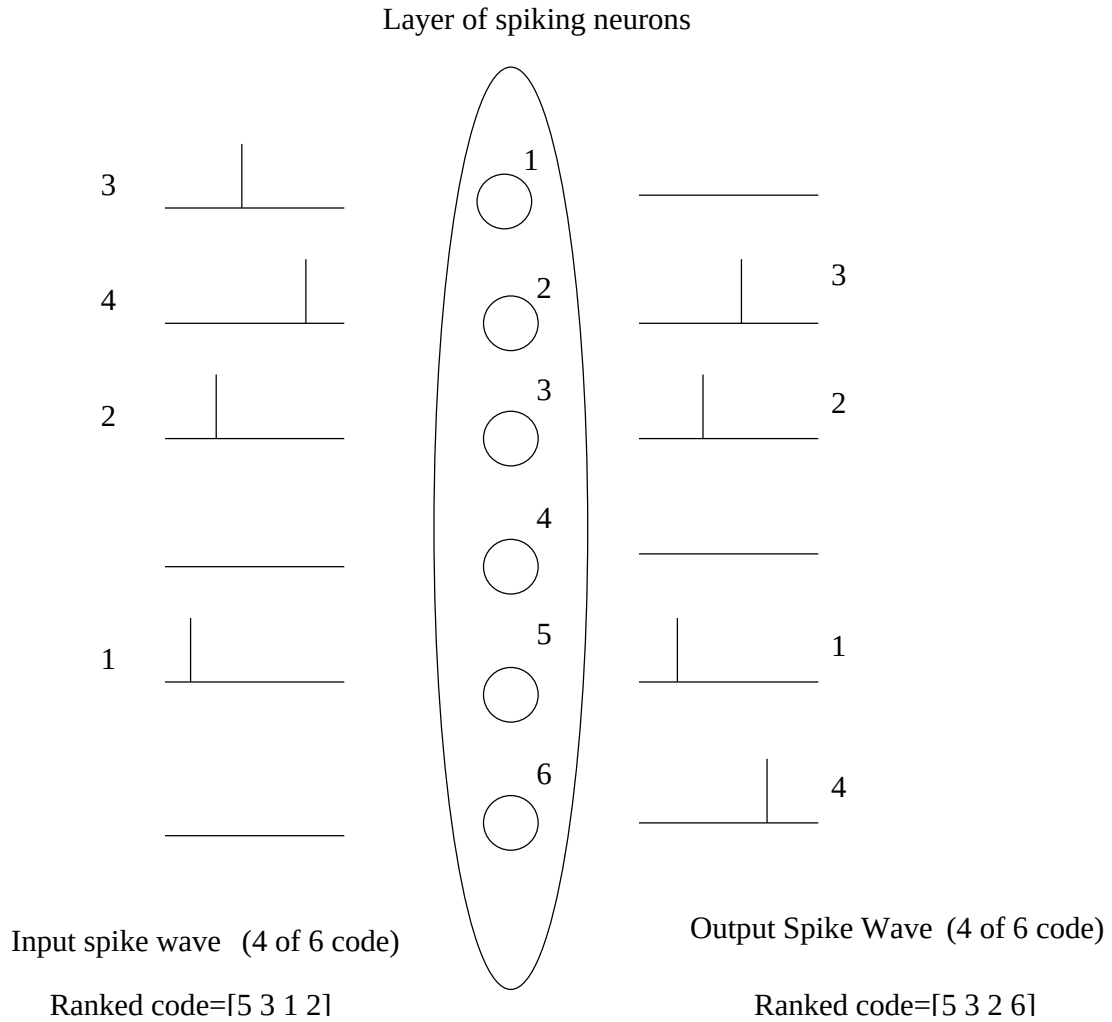


Figure 9.2: A wave of input and output spike firings in a fascicle

through the loop through successive iterations. There are two time constants which characterise the system: one represents the time t , within which the wave of spikes is distributed. The other represents the time separation of two different waves of spikes, represented as T .

In the neural feedback system, it is necessary to ensure the following

- Two different spike waves do not interfere. This means that there should be sufficient temporal separation between two waves of spikes and this should not decrease as the waves propagate around the loop.
- The relative ordering of the firing spikes is maintained as the spike wave propagates around the loop

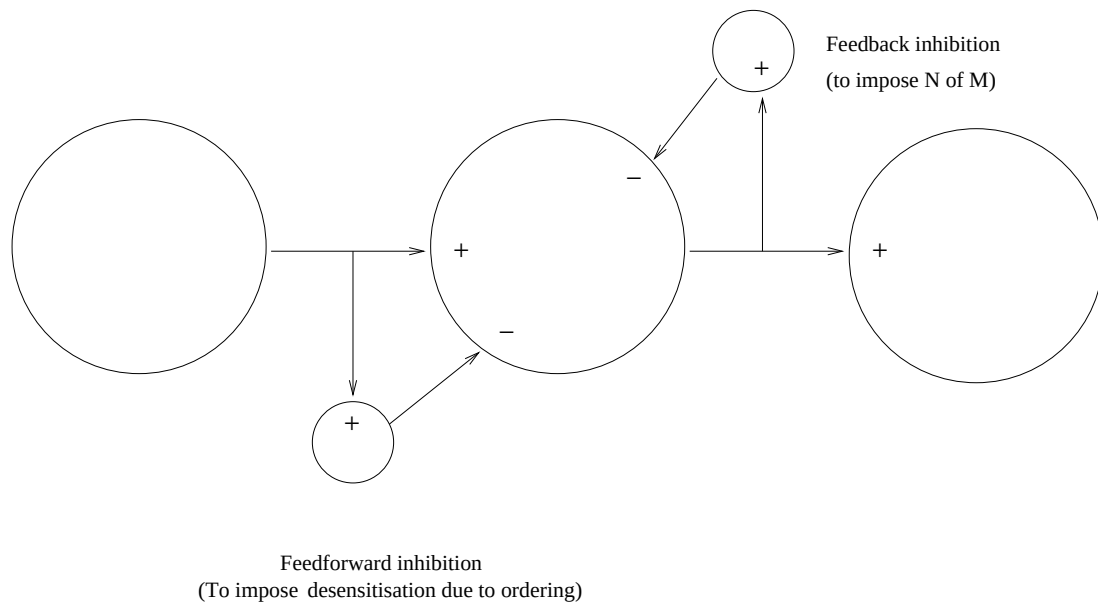


Figure 9.3: A fascicle to implement ordered N of M code

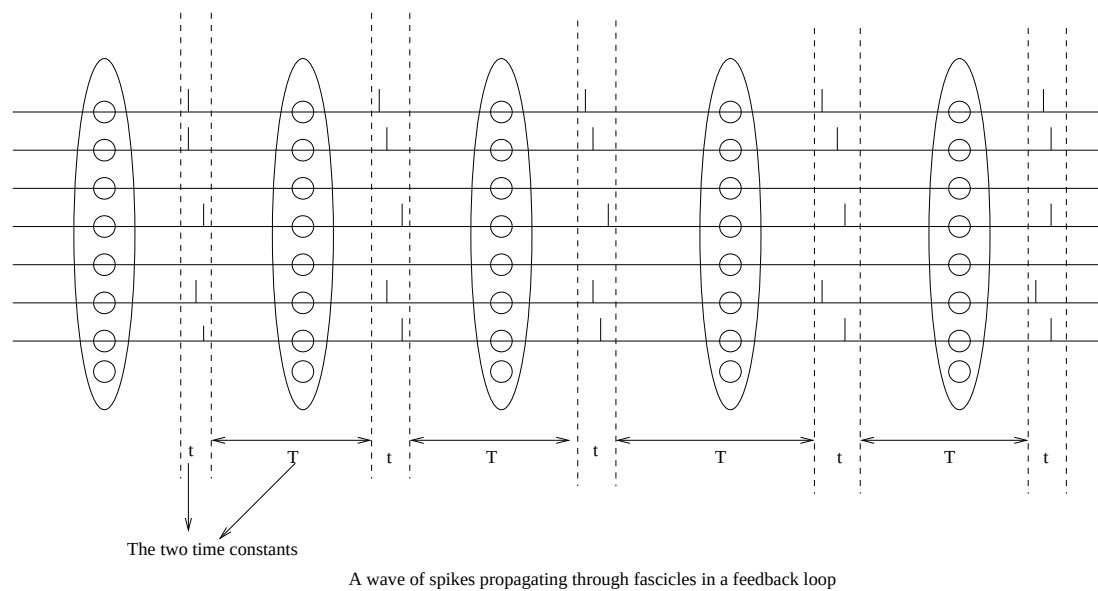


Figure 9.4: A propagating spike wave and the two time constants

A neural wave of firings propagating through two fascicles in a feedback loop is equivalent to having to pass through infinite fascicles, if the feedback loop is 'unrolled'. Figure 9.4 shows the two time constants and the propagating wave of spikes around the feedback loop, unrolled.

9.6 Anti dispersion measures

The problem with having a wave of spikes propagating round a feedback loop is that it tends to diverge with time. Due to various factors such as noise due to cross talk or unequal time delays in propagation through the fascicle, the spikes tend to lose the relative timing, thus corrupting the ordered code. There may also be crosstalk between two waves of spikes.

We have to ensure that the two time constants, as mentioned above, are properly chosen so that they have an appreciable difference, so that the wavefront can maintain its identity as it propagates. Also we need to minimise the possibility of dispersion explicitly by introducing appropriate inhibition to prevent the wavefronts from interfering.

One such measure would be to have an extra inhibitory feedback loop having an extra fascicle.

As shown Figure 9.5, each fascicle has a corresponding inhibitory fascicle which is identical to it in all respects. Inhibition from this fascicle would serve to maintain firing order and control dispersion.

9.7 Stabilisation of neural states and oscillation

It has been found that when such a feedback loop is used in a low level implementation of the ordered N of M neural model, the network indeed settles into stable states after fluctuating for some time. However, there may be multiple stable states and the network may oscillate between those states.

9.8 A generalised neural system

This section deals with building a generalised spiking N of M neural system which is more generalised than the pendulum model described in chapter 6.

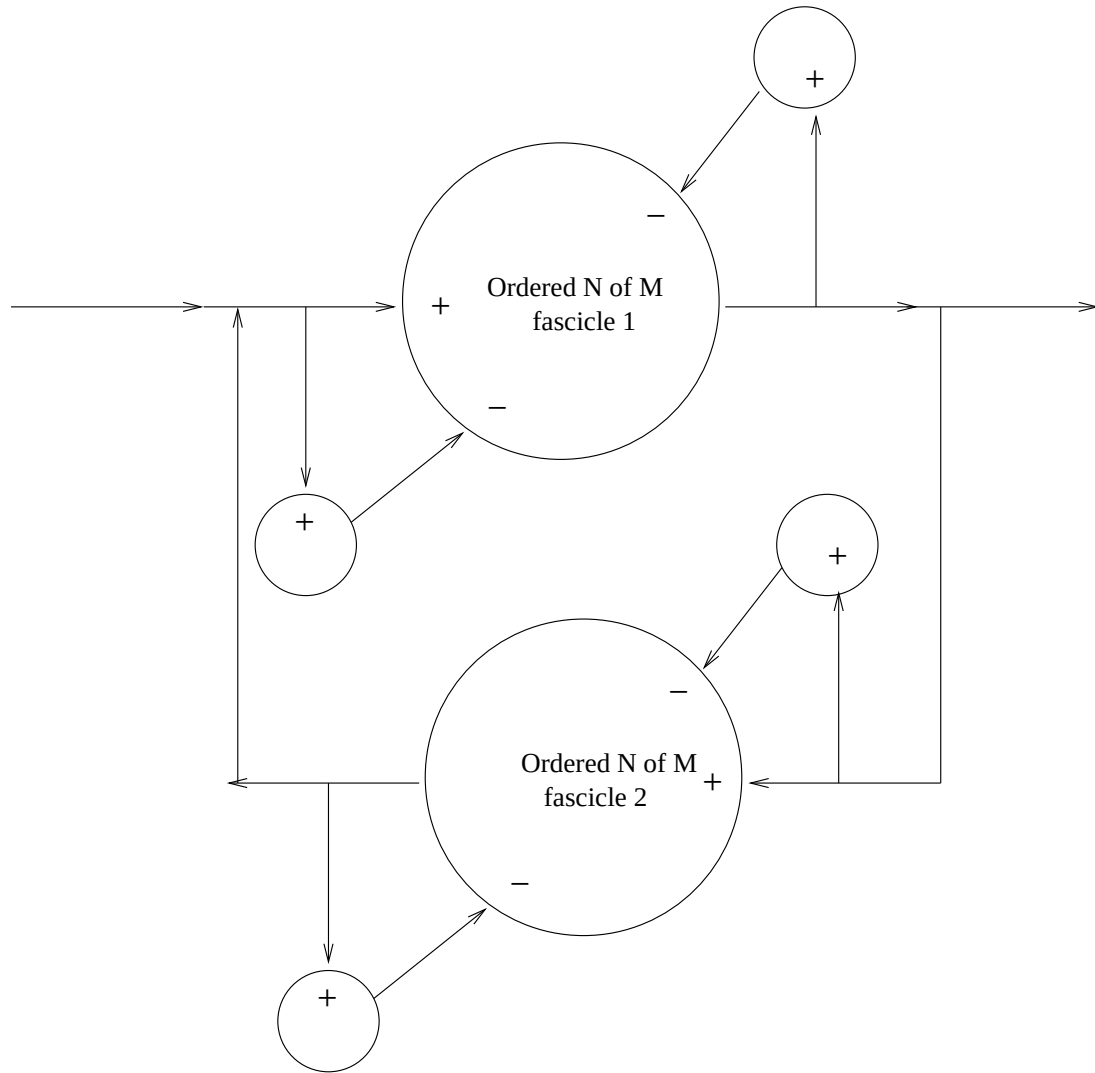


Figure 9.5: Using a second inhibitory fascicle to control wave dispersion

Spiking neural systems have some additional properties than in the pendulum model. They are weightless and have no inertia, unlike in the pendulum model where neurons had mass and momentum. Besides, there are a few other effects that can be modelled, such as habituation. Habituation increases the threshold of the neuron which fires frequently, which in effect reduces its sensitivity.

Furber's technical report[17] of a rank order fascicle processor examines the dynamics in detail.

9.9 Neural Engineering

This research falls in the sphere of a wider field called Neural Engineering, which is the application of engineering to models of neural systems, in an attempt to understand how reliable systems can be built out of unreliable component neurons.

9.10 Emergent behaviour

Large autonomous dynamical systems of neurons interact with each other. These interactions cause emergent behaviour. This behaviour is very difficult to analyse and predict without actual modelling. Hence modelling such systems is the only way we can study such behaviour.

9.10.1 A life: the modelling of emergent properties of autonomous systems

A life is a comparatively new field of study, being less than 15 years old. The term was originally coined by Chris Langton[9]. This field is entirely devoted to the study and modelling of artificial life systems in a virtual environment. Usually the environment is modelled as a grid of cells. A number of 'animals', which are autonomous systems occupying one or more squares in the grid. External factors such as 'food', 'sunlight' etc can be modelled, and the animals have to compete with each other to survive. For the backend of each autonomous animal, genetic or other evolutionary algorithms, cellular automata, etc. could be used rather than conventional neural models. Each of these systems are autonomous and have primitive sensory and motor organs.

One of the biggest commercial implementations of A life is the game Creatures. It was originally developed by Steve Grand[18]. It basically consists of systems of artificial beings called norms. Being a commercial implementation, a lot of effort has gone towards increasing the level of sophistication.

9.10.2 A virtual environment to model autonomous dynamical systems to observe emergent properties

This is an A-life model, inspired by Conway's game of life, cellular automata, and mobile robotics. This environment can be used for modelling autonomous systems and studying their emergent properties.

As described in the preceding section, the virtual environment would be modelled as a grid. It would have autonomous cellular systems called 'animals'. Each animal would have some neural sensors to interface with the environment. These animals would have primitive sensory, motor and reproductive capabilities.

Reproduction would be asexual, with an animal capable of laying or fertilising a one-celled egg. Each such task costs the animal energy.

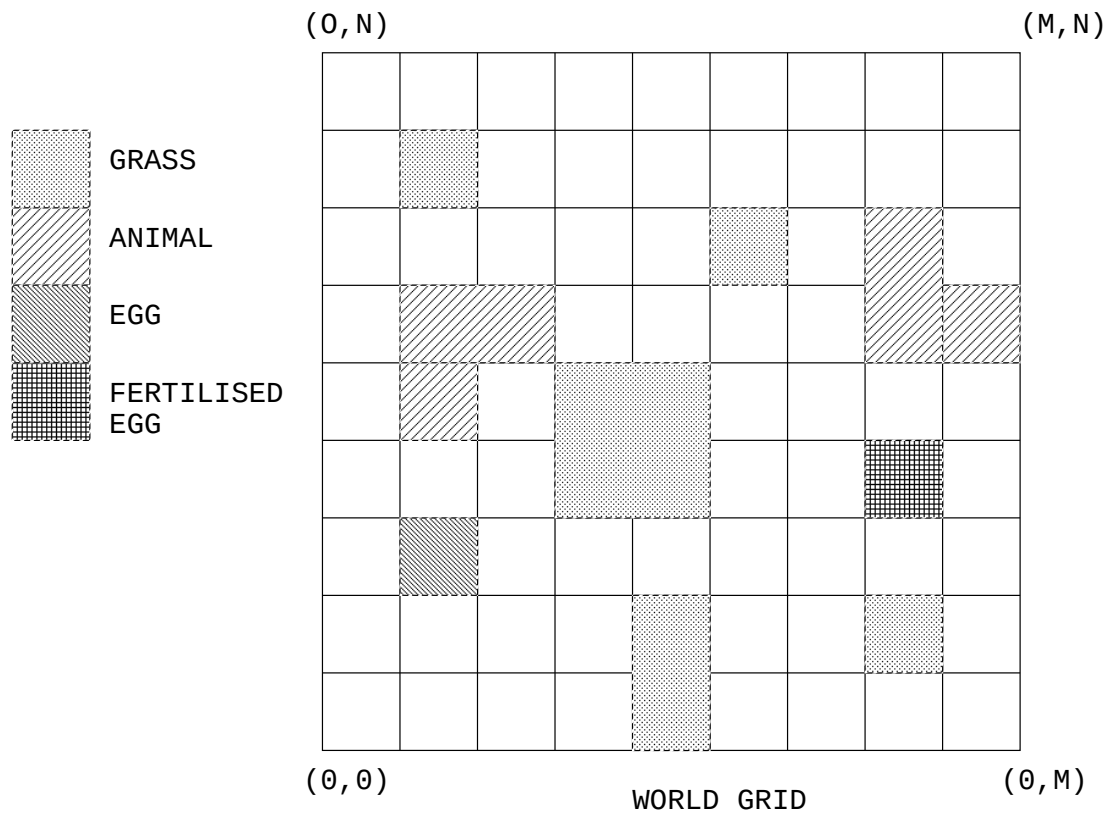


Figure 9.6: Virtual environment

Some of the cells in the grid would contain food, which would replenish the energy level of the animal.

The basic motor actions the animal is capable of doing are:

- Lay egg
- Fertilise egg (simple asexual reproduction)
- Eat
- Move (up, down, left or right) one cell

The basic sensory actions would be as follows:

- Detect food
- Detect unfertilised egg
- Detect another animal in one of its neighbouring cells

More complex function like sleep, pain, happiness etc. can be modelled in the same way if necessary. The object of the animal is to survive for the longest time.

The autonomous neural models learn and evolve as they live in this virtual world. The animal which evolves the best algorithm survives the longest.

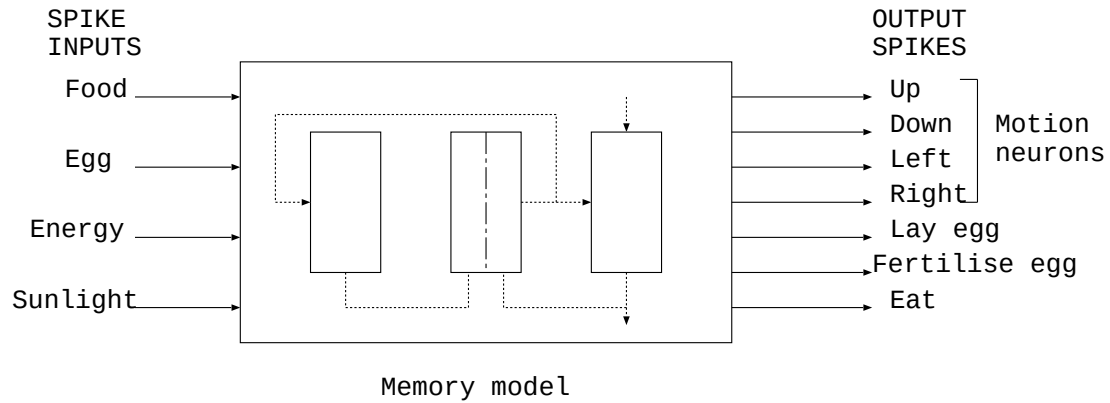


Figure 9.7: Virtual environment implementation with neurons

All communication of the animal with the outside world would take place through spikes generated by sensory neurons. Another set of output motor neurons generate spikes, which, in turn, tell the animal to take appropriate action. The world grid has a number of species of these animals, which interact with each other, compete to survive and advance their species. If necessary, further complexity may be introduced by having different competing species.

Such an environment can be used in testing dynamics and emergent properties of low-level neural models. This tool would be an easy to visualise, generic environment for testing any neural algorithm or architecture, which would be at the backend of the animal and would define its actions, as well as taking input from the outside world.

9.10.3 Mobile robotics: A possible application

One application of such a virtual environment can be modelling robots in a real environment. The autonomous neural systems can be trained in the virtual environment with obstacles etc. to model the real world as closely as possible. The best performing algorithm can be used to control the motion of a mobile robot in a real environment like a room.

9.11 Conclusion

In this chapter, we have discussed some issues involved regarding the dynamics of neural systems, and analysed our model to check if it is valid when real neurons are considered rather than the abstraction which had been used previously. The next section is the concluding section and will summarise the major contributions made in this thesis and contain pointers for future work.

Chapter 10

Conclusions and future work

10.1 Introduction

This chapter outlines the major avenues of future work and tries to touch upon some possible applications for the memory model.

10.2 Summary of this research

This research is mainly concerned with building software models for scalable neural systems, which can be implemented in hardware. The developed model encodes data using a rank-ordered N of M code. It is based on Kanerva's Sparse Distributed memories. The Elman network model is used to incorporate state information by having some extra neurons which represent state. The basic model is a two-layered neural network with recurrent connections. The idea is to decode the input pattern into high dimensional space, where it is much more likely to be linearly separable. Hebbian learning takes place in one of the two layers, while the weights are fixed in the other layer, which is called the address decoder as it casts or decodes the input address pattern into high dimensional address space. The second of the two layers is a correlation matrix memory. The model uses a time abstracted form of ordered code with spiking neurons and learns in a single pass.

A summary of the work done in the context of this research is listed below.

1. Experimental Analysis (capacity and error tolerance) of Kanerva's SDM model and related associative memories
2. Modified Kanerva's model using Rank Ordered codes
3. Effect of sensitivity of ordering, input errors, matching threshold and input parameters on the information content of the memory studied through experiments
4. Improving the model: using state neurons and recurrent connections
5. Sequence recognition: context based sequential learning
6. Testing with both low and high level models, thus validating the high level abstractions used
7. Proposal for a generalised neural asynchronous finite state machine

10.3 Experimental results

The main results of numerical simulations conducted to study the properties of the neural model are summarised below.

- Capacity of the memory degrades (forgetting due to cross talk) gracefully and not abruptly, as the number of words fed in is increased.
- Using an ordered code is useful only if exact matches (at least in the order) are not required, although this statement is applicable only for low desensitisation effects due to ordering. In general, capacity of the memory when ordered codes are used is substantially less than capacity with unordered codes.
- The size of the high dimensional space used to cast the input pattern (as in the number of address decoders) effects the memory capacity, as does the vocabulary of the pattern space (the number of words that are different)
- The choice of the parameters (N of M) of the code used affects the memory information density.
- Having a sparse model with binary weights using the OR function to set the weights during writing to the memory (in the unordered case) leads to the maximum information capacity.

- Using sensitisation factors of 0.5 and lower give best results, as ordering of two code words becomes more important than the individual bits of the patterns chosen. Also the recovery capacities of ordered codes degrade more gracefully than unordered codes
- As N (in the N of M code) is decreased, the memory capacity increases but the information in each word decreases too.
- Increasing the number of bit-errors in the input during recall of stored associations degrades the memory capacity significantly.
- Offline learning in the retrieval of sequences works much better than online learning.
- Like in Kanervas model, there is a critical distance (3 bits for the 11 of 256 unordered code with 4096 address decoder neurons), beyond which input patterns never converge.

10.4 Contributions made in this thesis

The primary contribution of this thesis is in developing a memory model which can store and recognise sequences of patterns, and is also scalable, capable of learning on the fly and also stores the history of a sequence. However, the model has not yet been successful in retrieving the whole sequence from the middle of the sequence. It can only recall from the beginning. Possible applications to the same have been analysed and alternative avenues explored.

The main achievement of this work has been the development of a context based autonomous memory, which uses localised Hebbian learning, and use of it to learn and recall sequences of patterns. Work has also been done in exploring models of associative neural memories, and their information capacities, and trying to develop a scalable model that can be implemented in hardware, is capable of both one-pass online and offline learning, and is neurobiologically feasible (can be implemented by low level spiking neurons as well).

10.5 Future work: Towards better memory models

It might be possible to develop better learning algorithms for increasing the capacity of associative memories by experimenting with a number of alternative techniques. A few of such possibilities are outlined below.

10.5.1 Using lateral connections

For the learning to be online, it is necessary for the learning algorithm to be unsupervised. It may be worthwhile to experiment with memory models using lateral connections. Such winner-take-all systems might solve the sorting bottleneck so that the neuron with the maximum activity in the fascicle should automatically fire first.

10.5.2 Using stochastic neural models

Setting the weight to 1 with a probability equal to some measure rather than setting the weight to a fraction might be explored with.

10.5.3 Using other algorithms

Experimentation with other neural architectures or algorithms such as self organising maps, genetic algorithms or support vector machines may also be investigated. The possibility of using genetic algorithms or other such evolutionary techniques in the memory needs to be further explored.

10.5.4 Experimenting with feedforward and feedback loops

More experimenting needs to be done with feedback and feedforward loops, to ensure that different spike trains propagating around the loop do not impinge on one another when the dynamics of real neurons are taken into account. In a large scale network it would be very essential to synchronise

the firing trains of different components of the fascicles which fire asynchronously, so that the identity of the train and order of firing of the neurons in the fascicles are preserved.

10.5.5 Experimenting with supervised learning

A combination of supervised and unsupervised learning algorithms might be useful in giving better prediction of the next pattern in a sequence. Such a semi-supervised algorithm needs to be experimented upon.

10.5.6 Efficiency of hardware implementation

The neural architecture or learning algorithm used in the model may need to be modified to make it more efficient or faster when it is implemented in hardware. Further experimentation may be done regarding this.

10.5.7 Some alternatives for sequence learning

The present model can retrieve any sequence when the first pattern of the sequence is provided but is not so efficient in retrieving the rest of the sequence given a pattern at the middle of the sequence. Further experimentation is needed to find a way of modulating the weight given to the context of the pattern in the sequence as compared to the pattern itself, while determining which pattern should follow in the sequence. A separation of the learning and recall modes, rather than online learning, might be necessary for this to be possible. The system should be able to automatically decide upon predicting the next character when it is getting an ambiguous signal or the pattern and the context of the pattern predict entirely different outputs.

10.6 Future work: Possible applications

The application space of the neural model is enormous. There are a number of possibilities for commercial exploitation. A few applications of the model are discussed in the following paragraphs.

10.6.1 Exploitable features

The model has a few features, which can be exploited to develop appropriate applications. The novelty of the model lies in its scalability, speed of learning and retrieval, which may be done in real time, asynchronous operation, low average power throughput required, sparseness of the memory space involved, capacity to model a generic finite state machine, robustness and resistance to degradation, distributed processing, error tolerance, ease of hardware implementation and the resulting speedup due to parallel processing in hardware, etc. The training and recall phases can be either separated or done together depending on the application. Multiple units of the two-layer model may be used to give it additional flexibility. Any or all of these features may be exploited in applications.

10.6.2 Pattern recognition

The model is, first of all, a pattern associator. Character or image recognition comes to mind as the first possible application, as they are standard applications in the neural domain. It can learn to predict the next pattern in a given sequence given the first few patterns in the sequence. It may be possible for the model to predict the next output in a time series. Such a thing would be of use in things like the prediction of weather and stock markets. Autonomous time series prediction is thus, one of the possible applications of the model.

10.6.3 Online learning

On line learning is another characteristic of the model, which can be exploited to make useful applications, such as online authentication by handwriting or speech recognition.

10.6.4 High-end tasks

The possibility of building of large scale systems in hardware opens up new avenues of applications, in robust, high performance, calculation intensive

systems involving millions of neurons. This can be of use in data classification or clustering, motif discovery etc.

10.6.5 Robot navigation

Sequence storage, retrieval and prediction can have a number of uses, as in genome mapping in DNA sequences, or in sequencing or remembering of notes in a tune.. Mobile robotics is another possible characteristic of the model. Modelling of the best course for navigation of a robot in a real or virtual world can help in creation of more intelligent robotic systems. As this model learns online, it can be of relevance in helping the robot overcome obstacles or find its way through a maze. It can also be used in modelling financial markets or other such time series.

10.6.6 Modelling biology

Modelling biology can open up many avenues of application in medicine, neuroscience, behavioural psychology and many other fields. For example, modelling of the primate visual system can have a number of applications in artificial vision.

The autonomous learning nature of the model can have a few applications as well. Emergent behaviour of neurons in real time large scale systems can be studied using this model as an experimental bed. This can thus become a test bed for testing various neural connectionist or evolutionary algorithms or architectures. The model can have applications in neuroscience, as the possibility of modelling millions of neurons can help in appreciating the function of parts of the brain by modelling the corresponding neural structures, and thus get a better understanding of the working of the brain. It may be used in modelling or predicting the behaviour of a few organisms which have much simpler brains, such as ants, bees, or frogs.

As discussed earlier, the sparse distributed memory has structural similarity to the cerebellum in the brain. This makes it possible to model its working. Despite being one of the most well researched areas of the brain, the exact working of the cerebellum still not understood. It is said to be some sort of a feedback control system, which takes inputs from the environment

through sensory organs and sends output spikes to the motor neurons. It is typically very slow in function, and the inputs frequently change by the time it processes the input stimuli. So it has to predict if the changes in advance. An interesting application would be to model the cerebellum using a SDM with spiking neurons and see if it can model accurately the cerebellar function. A standard test would be tracking a point on the screen. The neural model used has a time lag, so it needs to predict accurately.

10.6.7 Encryption

The way the model encodes patterns in memory can be utilised to have an encryption algorithm, with the random generator keys as the public keys.

10.7 Conclusion

Thus, this model can have a number of applications, not just limited to the ones discussed above. Further work is possible in any of these avenues.

Bibliography

- [1] Moshe Abeles. *Corticonics*. Cambridge University Press, 1991.
- [2] Paulo J. L. Adeodato and John G. Taylor. Analysis of the storage capacity of ram-based neural networks. *Neural Networks*, 14, 2001.
- [3] Igor Aleksander and Helen Morton. *An introduction to Neural Computing*. Thomson Computer Press, 1995.
- [4] Kenneth W. Regan Arun Jagota, Giri Narasimhan. Information capacity of binary weights associative memories. *Neurocomputing*, 19, 1998.
- [5] J Austin and M Turner. Matching performance of binary correlation matrix memories. *Neural Networks*, 10, 1997.
- [6] Jim Austin. Adam: A distributed associative memory for scene analysis. *Proceedings of First International Conference on Neural Networks Ed. M Caudhill and C Butler, San Diego*, 4, June 1987.
- [7] D Casasent and B Tefler. High capacity pattern recognition associative processors. *Neural Networks*, 5, 1992.
- [8] Philip A. Chou. The capacity of the kanerva associative memory. *IEEE Transactions on Information Theory*, 35, 1989.
- [9] Editor Christopher G. Langton. *Artificial Life*. Addison Wesley, 1989.
- [10] O P Buneman D J Willshaw and H C Longuet-Higgins. Non holographic associative memory. *Nature*, June 1969.
- [11] Simon Davidson. *On the application of Neural Networks to Symbol Systems*. PhD thesis, University of Sheffield, UK, 1999.
- [12] A. Delorme and S. Thorpe. Spikenet: an event driven simulation package for modeling large networks of spiking neurons. *Neural Networks*, 14, 2003.

- [13] H. Gutfreund D.J. Amit and H. Sompolinsky. Statistical mechanics of neural networks near saturation. *Ann. Phys.(USA)*, 173, 1987.
- [14] Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14, 1990.
- [15] F T Sommer F Schwenker and G Palm. Iterative retrieval of sparsely coded associative memory patterns. *Neural Networks*, 9, 1996.
- [16] Rob de Ruyter van Steveninck Fred Rieke, David Warland and William Bialek. *Spikes: Exploring the neural code*. MIT Press, 1999.
- [17] S. B. Furber. A rank-order fascicle processor. Technical report, University of Manchester, 2003.
- [18] Steve Grand. *Creation: Life and how to make it*. Phoenix, 2000.
- [19] Simon Haykin. *Neural networks: A comprehensive foundation*. Pearson Education, inc, 2nd edition, 1999.
- [20] D. O. Hebb. The organisation of behaviour: a neuropsychological theory. *Bulletin of Mathematical Biophysics*, 1949.
- [21] Richard Henson. *Short term associative memories*. MSc Thesis, University of Edinburgh, UK, 1993.
- [22] A. L. Hodgkin and A. F. Huxley. A quantitative description of ion currents and its applications to conduction and excitation in nerve membranes. *Journal of Physiology*, 117, 1952.
- [23] J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences, USA*, 79, 1982.
- [24] W.V.Thomas Igor Alexander and P.A.Bowden. Wisard: a radical step forward in image recognition. *Sensor Review*, 4, 1984.
- [25] M. I. Jordan. Serial order: a parallel distributed processing approach. Technical Report 86044, Institute for Cognitive Science, University of California San Diego, 1986.
- [26] Pentti Kanerva. *Sparse Distributed Memory*. MIT Press, 1988.
- [27] Teuvo Kohonen. Correlation matrix memories. *IEEE Transactions on Computers*, 21(4), April 1972.

- [28] David Lomas. *Improving Automated Postal Address Recognition*. MSc Thesis, University of York, UK, 1996.
- [29] W. S. McCulloch and W. Pitts. A logical calculus of the ideas immanent in neural activity. *Bulletin of Mathematical Biophysics*, 5, 1943.
- [30] Claude Meunier and Jean-Pierre Nadal. Sparsely coded neural networks. *The Handbook of Brain Theory and Neural Networks*, Arbib M. A. Ed. (MIT Press, 1995), 1995.
- [31] John Milton. *Dynamics of small neural populations, Vol.7, CRM Monograph Series*. American Mathematical Society, 1996.
- [32] J P Nadal and G Toulouse. Information storage in sparsely coded memory nets. *Network: Information in neural systems*, 1990.
- [33] Roger Penrose. *The Emperor's New Mind*. Oxford University Press, 1989.
- [34] F. Rosenblatt. The perceptron: a probabilistic model for information storage and organisation in the brain. *Psychological Review*, 65, 1958.
- [35] R. Van Rullen S. Thorpe, A. Delorme. Spike based strategies for rapid processing. *Neural Networks*, 14, 2001.
- [36] J.M. Cumpstey S.B. Furber, W.J. Bainbridge and S. Temple. Sparse distributed memory using n of m codes. *Submitted for publication to Neural Networks*, 2001.
- [37] C. E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27, July 1948.
- [38] Yoshiyuki Kabashima Toshiyuki Tanaka, Shinsuke Kakiya. Capacity analysis of bidirectional associative memory. *Proc. ICONIP 2000, Taejeon, Korea*, 2, 2000.
- [39] John von Neumann. The principles of large-scale computing machines. *Ann. Hist. Comp*, 3(3), 1946.
- [40] Richard C. Wilson and Edwin R. Hancock. Storage capacity of the exponential correlation associative memory. *International Conference on Pattern Recognition (ICPR'00), Barcelona, Spain*, 2, 2000.
- [41] D.A.Edwards W.J.Bainbridge, W.B.Toms and S.B.Furber. Delay insensitive, point-to-point interconnect using m-of-n codes. *Proceedings of Async 2003, Vancouver*, May 2003.

- [42] Christopher M. Bishop Wolfgang Maass. *Pulsed Neural Networks*. MIT Press, 1999.
- [43] Werner M. Kistler Wulfram Gerstner. *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press, 1st edition, 2002.